

算法竞赛模板与技巧手册

默认 `gnu++20`、公开调用 1-based、区间 `[1, r]`；例外在条目前单独标明。

目录

- [1. 数据结构](#)
 - **并查集**：并查集 (DSU) · 可撤销并查集 · 带权并查集 · 后继并查集
 - **静态区间**：树状数组 · 稀疏表 (ST 表)
 - **线段树**：线段树 · 懒标记线段树 · 可持久化线段树 · 动态开点线段树 · 吉司机线段树 (Segment Tree Beats)
 - **平衡树**：无旋 Treap (FHQ) · 隐式 Treap · 有序多重集 (PBDS)
 - **凸包与直线容器**：李超线段树 · 凸包优化 DP
 - **高级树结构**：笛卡尔树 · 动态树 (LCT)
 - **Top-K 维护**：按不同键维护 Top-K
- [2. 图论](#)
 - **图论基础**：单源最短路 (Dijkstra) · 0-1 最短路 (BFS) · 全源最短路 (Floyd) · 拓扑排序 · 最小生成树 (Kruskal)
 - **二分图度序列构造** (Gale–Ryser) · 增量 APSP (只加边)
 - **连通性**：强连通分量 (SCC) · 二元可满足性 (2-SAT) · 桥与割点 · 边双连通分量 (EBCC) · 点双连通分量与圆方树
 - **欧拉路与匹配**：欧拉路径 · 二分图最大匹配 · 二分图最大权匹配 (Hungarian)
 - **网络流**：最大流 (Dinic) · 最小费用流 · 有上下界网络流
 - **图论建模与转化**：差分约束 · 函数图 · 优化建图 · 增量容量最大流 · 严格次小生成树
 - **阈值激活事件堆** · 异或最小生成树 · 最大权闭合子图 · 隐式完全图 Boruvka · 指定拓扑序的稀疏约束
- [3. 树上算法](#)
 - **树上基础**：树的直径 · 树的重心 · 最小生成树重构树 (Kruskal)
 - **树链与路径**：最近公共祖先 (LCA) · 重链剖分 (HLD) · 树上差分 · 虚树 · 倍增
 - **树分治**：树上启发式合并 (DSU on Tree) · 点分治
 - **树上技巧**：换根 DP · 启发式合并 (Small-to-Large) · 长链剖分优化深度 DP · 树上仿射约束传播
- [4. 动态规划](#)
 - **序列 DP**：最长上升子序列 (LIS)
 - **背包模板**：多重背包 (单调队列优化)
 - **DP 技巧**：数位 DP · 轮廓线 DP · 中位数阈值 ± 1 变换 · 稀疏事件线性 DP (矩阵幂) · 排列 LCS $\rightarrow$ LIS
 - **背包** · 单调队列优化 DP · 区间 DP · 树上背包 · 子集和 DP (SOS DP)
 - **分治优化 DP** · 状压 DP / Gray code · 四边形不等式优化 (Knuth) · 带权二分 (WQS) · 位集优化 (Bitset)
- [5. 数学](#)

- **模运算与组合**: 模整数 (MInt) · 扩展欧几里得 / 中国剩余定理 (CRT) · 组合数 · 卢卡斯定理 (Lucas) · 拉格朗日插值
- **筛法与分解**: 线性筛 · 莫比乌斯 / 欧拉函数前缀和 · 米勒-拉宾素性测试 · 波拉德-ρ 大整数分解 · 整除分块
- **类欧几里得与离散对数**: 类欧几里得 (Floor Sum) · 离散对数 (BSGS)
- **矩阵**: 矩阵快速幂 (模质数)
- **亚线性筛**: 杜教筛 (ϕ / μ 前缀和) · 亚线性筛 (Min_25)
- **数学技巧**: 约数触发更新 · 半环矩阵幂 · 同余按 gcd 分类计数 · 二进制逐位提升 (2-adic) · 欧拉降幂幂塔
- ↪: 欧几里得批量跳跃
- [6. 多项式](#)
 - **卷积**: 快速傅里叶变换 (FFT, 整数卷积) · 数论变换 (NTT, 模 998244353) · 快速沃尔什变换 (FWT, OR / AND / XOR 卷积)
 - **形式幂级数**: 形式幂级数: 逆 / $\ln$ / $\exp$
- [7. 线性代数](#)
 - **高斯消元与异或线性基**: 高斯消元 (Gauss-Jordan) · 异或线性基 (u64) · 二元域多右端项线性方程组 (GF(2))
- [8. 字符串](#)
 - **字符串匹配**: 前缀函数匹配 (KMP) · Z 函数 / 扩展 KMP · 滚动哈希 · 子序列自动机 (next-occurrence)
 - **字符串自动机**: 字典树 (Trie) · AC 自动机 · 后缀自动机 (SAM)
 - **字符串序**: 后缀数组 (SA) · 最小循环同构
 - **回文**: 回文半径 (Manacher) · 回文自动机 (Eertree)
- [9. 计算几何](#)
 - **几何基础**: 点 (Point) · 直线 (Line) · 线段 (Segment) · 凸包 (Andrew 单调链) · 点在多边形内 (射线法)
 - ↪: 最近点对 (分治)
 - **高级几何**: 旋转卡壳 (凸多边形直径) · 圆 (外接圆 / 两圆交点) · 半平面交 (直线左侧, 需有界) · 闵可夫斯基和
 - **扫描线应用**: 矩形面积并
- [10. 离线算法与经典技巧](#)
 - **搜索与分治**: 二分答案 · 离线分治 (CDQ) · 整体二分 · 线段树分治 · 双指针 / 滑动窗口
 - ↪: 折半搜索
 - **离线与事件排序**: 莫队 · 离线倒序 · 坐标压缩 · 扫描线 · 重置点分段
 - ↪: 预留坐标
 - **根号分治与重轻**: 根号分治 / 序列分块 · 度数定向 / 重轻点 · 时间分块 · 频次排序记忆化
 - **贡献与转化**: 对数段技巧 (Log Trick) · 前缀和 / 差分 · 贡献法 / 动态维护贡献 · 有限群惰性标记 · 曼哈顿距离符号变换 (L1)
 - ↪: 势能剪枝线段树

- **贪心**：反悔贪心 · 单调状态空间 Top-K · 最后出现位置单调栈 · 相邻交换排序 · 区间-点贪心匹配
- **可行性 Oracle 构造** · 加权中位数
- **计数、位运算与几何**：容斥 · 二进制对齐块分解 · 固定半径空间哈希 · 布尔格对称链
- [11. 公式与博弈](#)
 - **公式与博弈**：常用恒等式 · 博弈 (SG / Nim)

1. 数据结构

并查集 (DSU)

1-based.

```
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> fa, sz;
    int cnt;

    DSU(int n = 0) : fa(n + 1), sz(n + 1, 1), cnt(n) {
        iota(fa.begin(), fa.end(), 0);
    }

    int find(int x) {
        return x == fa[x] ? x : fa[x] = find(fa[x]);
    }

    bool same(int x, int y) {
        return find(x) == find(y);
    }

    bool merge(int x, int y) {
        x = find(x);
        y = find(y);
        if(x == y) return false;
        if(sz[x] < sz[y]) swap(x, y);
        fa[y] = x;
        sz[x] += sz[y];
        cnt--;
        return true;
    }

    int size(int x) {
        return sz[find(x)];
    }

    int groups() const {
        return cnt;
    }
};
```

可撤销并查集

1-based; 不做路径压缩, 按 size 合并。

snapshot() 记录栈位置, rollback(snap) 恢复。

```
#include <bits/stdc++.h>
using namespace std;

struct RollbackDSU {
    vector<int> fa, sz;
    vector<array<int, 3>> his;
    int cnt;

    RollbackDSU(int n = 0) : fa(n + 1), sz(n + 1, 1), cnt(n) {
        iota(fa.begin(), fa.end(), 0);
    }

    int find(int x) const {
        while(x != fa[x]) x = fa[x];
        return x;
    }

    bool same(int x, int y) const {
        return find(x) == find(y);
    }

    int size(int x) const {
        return sz[find(x)];
    }

    int groups() const {
        return cnt;
    }

    int snapshot() const {
        return int(his.size());
    }

    bool merge(int x, int y) {
        x = find(x);
        y = find(y);
        if(x == y) return false;
        if(sz[x] < sz[y]) swap(x, y);
        his.push_back({x, y, sz[x]});
        fa[y] = x;
        sz[x] += sz[y];
        cnt--;
        return true;
    }

    void rollback(int t) {
        while(int(his.size()) > t) {
```

```

        auto [x, y, oldSize] = his.back();
        his.pop_back();
        fa[y] = y;
        sz[x] = oldSize;
        cnt++;
    }
}
};

```

带权并查集

1-based; `merge(x,y,w)` 加入 `value[y]-value[x]=w`, 返回约束是否相容; `diff(x,y)` 同方向且要求已连通。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

template <class T = i64>
struct WeightedDSU {
    vector<int> fa, sz;
    vector<T> d;
    int cnt;

    WeightedDSU(int n = 0) : fa(n + 1), sz(n + 1, 1), d(n + 1), cnt(n) {
        iota(fa.begin(), fa.end(), 0);
    }

    int find(int x) {
        if(x == fa[x]) return x;
        int y = fa[x];
        fa[x] = find(y);
        d[x] += d[y];
        return fa[x];
    }

    bool same(int x, int y) {
        return find(x) == find(y);
    }

    int groups() const {
        return cnt;
    }

    T diff(int x, int y) {
        int a = find(x);
        int b = find(y);
        assert(a == b);
        return d[y] - d[x];
    }

    // value[y] - value[x] = w

```

```

bool merge(int x, int y, T w) {
    int a = find(x);
    int b = find(y);
    if(a == b) return d[y] - d[x] == w;

    T t = w + d[x] - d[y];
    if(sz[a] >= sz[b]) {
        fa[b] = a;
        d[b] = t;
        sz[a] += sz[b];
    } else {
        fa[a] = b;
        d[a] = -t;
        sz[b] += sz[a];
    }
    cnt--;
    return true;
}
};

```

后继并查集

1-based; 删除 `x` 后, `next(x)` 返回不小于 `x` 的首个未删除位置; `n+1` 表示无答案。每个位置只删一次。

```

#include <bits/stdc++.h>
using namespace std;

struct SuccessorDSU {
    vector<int> fa;

    SuccessorDSU(int n = 0) : fa(n + 2) {
        iota(fa.begin(), fa.end(), 0);
    }

    int find(int x) {
        return x == fa[x] ? x : fa[x] = find(fa[x]);
    }

    int next(int x) {
        return find(x);
    }

    void erase(int x) {
        fa[x] = find(x + 1);
    }
};

```

树状数组

1-based; `query(x)` 为 $[1, x]$, `rangeQuery(l, r)` 为闭区间; `select(k)` 返回最小前缀和 $\geq k$ 的位置, 不存在返回 `n+1`。 `select` 要求前缀和单调 (典型为各位置频次非负)。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

template <class T = i64>
struct Fenwick {
    int n;
    vector<T> tr;

    Fenwick(int n_ = 0) : n(n_), tr(n_ + 1) {}

    void add(int x, T v) {
        for(; x <= n; x += x & -x) tr[x] += v;
    }

    T query(int x) const {
        T res{};
        for(; x > 0; x -= x & -x) res += tr[x];
        return res;
    }

    T rangeQuery(int l, int r) const {
        return query(r) - query(l - 1);
    }

    int select(T k) const {
        int x = 0;
        T cur{};
        int step = int(bit_floor(unsigned(n)));
        for(; step; step >>= 1) {
            if(x + step <= n && cur + tr[x + step] < k) {
                x += step;
                cur += tr[x];
            }
        }
        return x + 1;
    }
};

```

稀疏表 (ST 表)

1-based, `query(l,r)` 为闭区间 `[l,r]`; `a[0]` 不使用, `op` 需满足结合、交换、幂等。

```

#include <bits/stdc++.h>
using namespace std;

template <class T, class F>
struct SparseTable {
    int n;
    vector<vector<T>> st;
    F op;

```

```

SparseTable(const vector<T>& a, F f) : n(int(a.size()) - 1), op(move(f)) {
    int m = int(bit_width(unsigned(n)));
    st.assign(m, vector<T>(n + 1));
    st[0] = a;
    for(int k = 1; k < m; k++) {
        for(int i = 1; i + (1 << k) - 1 <= n; i++) {
            st[k][i] = this->op(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
        }
    }
}

T query(int l, int r) const {
    int k = int(bit_width(unsigned(r - l + 1))) - 1;
    return op(st[k][l], st[k][r - (1 << k) + 1]);
}
};

```

线段树

1-based 闭区间；输入 `a[0]` 不使用；`Info()` 为单位元，`+` 满足结合律。`findFirst/findLast` 要求 `pred` 对按顺序累积的 `Info` 具有单调性。

```

#include <bits/stdc++.h>
using namespace std;

template <class Info>
struct SegmentTree {
    int n;
    vector<Info> tr;

    SegmentTree(int n_ = 0) : n(n_), tr(2 * n_) {}
    SegmentTree(const vector<Info>& a) : SegmentTree(int(a.size()) - 1) {
        for(int i = 1; i <= n; i++) tr[n + i - 1] = a[i];
        for(int p = n - 1; p; p--) pull(p);
    }

    void pull(int p) {
        tr[p] = tr[p << 1] + tr[p << 1 | 1];
    }

    void modify(int p, const Info& v) {
        tr[p += n - 1] = v;
        while(p > 1) {
            p >>= 1;
            pull(p);
        }
    }

    Info rangeQuery(int l, int r) const {
        Info a, b;
        for(l += n - 1, r += n; l < r; l >>= 1, r >>= 1) {

```

```

        if(l & 1) a = a + tr[l++];
        if(r & 1) b = tr[--r] + b;
    }
    return a + b;
}

template <class F>
int findFirst(int l, int r, F pred) const {
    int a[64], b[64], ca = 0, cb = 0;
    for(l += n - 1, r += n; l < r; l >>= 1, r >>= 1) {
        if(l & 1) a[ca++] = l++;
        if(r & 1) b[cb++] = --r;
    }
    Info cur;
    auto work = [&](int p) {
        Info nxt = cur + tr[p];
        if(!pred(nxt)) {
            cur = nxt;
            return -1;
        }
        while(p < n) {
            p <<= 1;
            nxt = cur + tr[p];
            if(!pred(nxt)) {
                cur = nxt;
                p |= 1;
            }
        }
        return p - n + 1;
    };
    for(int i = 0; i < ca; i++) {
        int p = work(a[i]);
        if(p != -1) return p;
    }
    for(int i = cb - 1; i >= 0; i--) {
        int p = work(b[i]);
        if(p != -1) return p;
    }
    return -1;
}

```

```

template <class F>
int findLast(int l, int r, F pred) const {
    int a[64], b[64], ca = 0, cb = 0;
    for(l += n - 1, r += n; l < r; l >>= 1, r >>= 1) {
        if(l & 1) a[ca++] = l++;
        if(r & 1) b[cb++] = --r;
    }
    Info cur;
    auto work = [&](int p) {
        Info nxt = tr[p] + cur;
        if(!pred(nxt)) {
            cur = nxt;

```

```

        return -1;
    }
    while(p < n) {
        p = p << 1 | 1;
        nxt = tr[p] + cur;
        if(!pred(nxt)) {
            cur = nxt;
            p ^= 1;
        }
    }
    return p - n + 1;
};

for(int i = 0; i < cb; i++) {
    int p = work(b[i]);
    if(p != -1) return p;
}
for(int i = ca - 1; i >= 0; i--) {
    int p = work(a[i]);
    if(p != -1) return p;
}
return -1;
}
};

```

懒标记线段树

1-based 闭区间；输入 `a[0]` 不使用；`Info()/Tag()` 为单位元；`Info.apply(v)` 应用标记，`Tag.apply(v)` 把新操作 `v` 接在旧操作之后。`findFirst/findLast` 要求 `pred` 对按顺序累积的 `Info` 具有单调性。

```

#include <bits/stdc++.h>
using namespace std;

template <class Info, class Tag>
struct LazySegmentTree {
    int n = 0;
    vector<Info> tr;
    vector<Tag> lz;

    LazySegmentTree(int n_ = 0, const Info& v = Info()) {
        init(n_, v);
    }
    template <class T>
    LazySegmentTree(const vector<T>& a) {
        init(a);
    }

    void init(int n_, const Info& v = Info()) {
        init(vector<Info>(n_ + 1, v));
    }
    template <class T>
    void init(const vector<T>& a) {
        n = int(a.size()) - 1;
    }
};

```

```

        tr.assign(4 * max(1, n), Info());
        lz.assign(4 * max(1, n), Tag());
        if(n) build(1, 1, n, a);
    }
    template <class T>
    void build(int p, int l, int r, const vector<T>& a) {
        if(l == r) {
            tr[p] = a[l];
            return;
        }
        int m = (l + r) >> 1;
        build(p << 1, l, m, a);
        build(p << 1 | 1, m + 1, r, a);
        pull(p);
    }

    void pull(int p) {
        tr[p] = tr[p << 1] + tr[p << 1 | 1];
    }

    void apply(int p, const Tag& v) {
        tr[p].apply(v);
        lz[p].apply(v);
    }

    void push(int p) {
        apply(p << 1, lz[p]);
        apply(p << 1 | 1, lz[p]);
        lz[p] = Tag();
    }

    void modify(int p, int l, int r, int x, const Info& v) {
        if(l == r) {
            tr[p] = v;
            return;
        }
        push(p);
        int m = (l + r) >> 1;
        if(x <= m) modify(p << 1, l, m, x, v);
        else modify(p << 1 | 1, m + 1, r, x, v);
        pull(p);
    }

    void modify(int x, const Info& v) {
        modify(1, 1, n, x, v);
    }

    Info rangeQuery(int p, int l, int r, int x, int y) {
        if(x <= l && r <= y) return tr[p];
        push(p);
        int m = (l + r) >> 1;
        if(y <= m) return rangeQuery(p << 1, l, m, x, y);
        if(x > m) return rangeQuery(p << 1 | 1, m + 1, r, x, y);
        return rangeQuery(p << 1, l, m, x, y)
            + rangeQuery(p << 1 | 1, m + 1, r, x, y);
    }

```

```

Info rangeQuery(int l, int r) {
    return rangeQuery(1, 1, n, l, r);
}

void rangeApply(int p, int l, int r, int x, int y, const Tag& v) {
    if(x <= l && r <= y) {
        apply(p, v);
        return;
    }
    push(p);
    int m = (l + r) >> 1;
    if(x <= m) rangeApply(p << 1, l, m, x, y, v);
    if(y > m) rangeApply(p << 1 | 1, m + 1, r, x, y, v);
    pull(p);
}

void rangeApply(int l, int r, const Tag& v) {
    rangeApply(1, 1, n, l, r, v);
}

template <class F>
int findFirst(int p, int l, int r, int x, int y, Info& cur, F& pred) {
    if(r < x || y < l) return -1;
    if(x <= l && r <= y) {
        Info nxt = cur + tr[p];
        if(!pred(nxt)) {
            cur = nxt;
            return -1;
        }
        if(l == r) return l;
    }
    push(p);
    int m = (l + r) >> 1;
    int res = findFirst(p << 1, l, m, x, y, cur, pred);
    if(res == -1) res = findFirst(p << 1 | 1, m + 1, r, x, y, cur, pred);
    return res;
}

template <class F>
int findFirst(int l, int r, F pred) {
    Info cur;
    return findFirst(1, 1, n, l, r, cur, pred);
}

template <class F>
int findLast(int p, int l, int r, int x, int y, Info& cur, F& pred) {
    if(r < x || y < l) return -1;
    if(x <= l && r <= y) {
        Info nxt = tr[p] + cur;
        if(!pred(nxt)) {
            cur = nxt;
            return -1;
        }
        if(l == r) return l;
    }
}

```

```

        push(p);
        int m = (l + r) >> 1;
        int res = findLast(p << 1 | 1, m + 1, r, x, y, cur, pred);
        if(res == -1) res = findLast(p << 1, 1, m, x, y, cur, pred);
        return res;
    }
    template <class F>
    int findLast(int l, int r, F pred) {
        Info cur;
        return findLast(1, 1, n, l, r, cur, pred);
    }
};

```

可持久化线段树

1-based 闭区间, 0 号根为空; `modify(prev,p,v)` 在叶子执行 `Info.apply(v)` 并返回新根。版本差查询/定位要求 `Info` 支持 `-`。

```

#include <bits/stdc++.h>
using namespace std;

template <class Info>
struct PersistentSegmentTree {
    struct Node {
        int ls = 0, rs = 0;
        Info val;
    };

    int n;
    vector<Node> tr;

    PersistentSegmentTree(int n_ = 0, int q = 0) : n(n_), tr(1) {
        if(n && q) tr.reserve(size_t(q) * (bit_width(unsigned(n)) + 1) + 1);
    }

    int clone(int p) {
        tr.push_back(tr[p]);
        return int(tr.size()) - 1;
    }

    void pull(int p) {
        tr[p].val = tr[tr[p].ls].val + tr[tr[p].rs].val;
    }

    int modify(int p, int l, int r, int x, const Info& v) {
        int u = clone(p);
        if(l == r) {
            tr[u].val.apply(v);
            return u;
        }
        int m = (l + r) >> 1;
        if(x <= m) tr[u].ls = modify(tr[p].ls, l, m, x, v);
        else tr[u].rs = modify(tr[p].rs, m + 1, r, x, v);
    }
};

```

```

    pull(u);
    return u;
}

int modify(int root, int x, const Info& v) {
    return modify(root, 1, n, x, v);
}

Info rangeQuery(int p, int l, int r, int x, int y) const {
    if(!p) return Info();
    if(x <= l && r <= y) return tr[p].val;
    int m = (l + r) >> 1;
    if(y <= m) return rangeQuery(tr[p].ls, l, m, x, y);
    if(x > m) return rangeQuery(tr[p].rs, m + 1, r, x, y);
    return rangeQuery(tr[p].ls, l, m, x, y)
        + rangeQuery(tr[p].rs, m + 1, r, x, y);
}

Info rangeQuery(int root, int l, int r) const {
    return rangeQuery(root, 1, n, l, r);
}

Info rangeQuery(int a, int b, int l, int r, int x, int y) const {
    if(x <= l && r <= y) return tr[b].val - tr[a].val;
    int m = (l + r) >> 1;
    if(y <= m) return rangeQuery(tr[a].ls, tr[b].ls, l, m, x, y);
    if(x > m) return rangeQuery(tr[a].rs, tr[b].rs, m + 1, r, x, y);
    return rangeQuery(tr[a].ls, tr[b].ls, l, m, x, y)
        + rangeQuery(tr[a].rs, tr[b].rs, m + 1, r, x, y);
}

Info rangeQuery(int a, int b, int l, int r) const {
    return rangeQuery(a, b, 1, n, l, r);
}

template <class F>
int findFirst(int a, int b, int l, int r, int x, int y, Info& cur, F& pred) const {
    if(r < x || y < l) return -1;
    if(x <= l && r <= y) {
        Info nxt = cur + (tr[b].val - tr[a].val);
        if(!pred(nxt)) {
            cur = nxt;
            return -1;
        }
        if(l == r) return l;
    }
    int m = (l + r) >> 1;
    int p = findFirst(tr[a].ls, tr[b].ls, l, m, x, y, cur, pred);
    if(p == -1) p = findFirst(tr[a].rs, tr[b].rs, m + 1, r, x, y, cur, pred);
    return p;
}

template <class F>
int findFirst(int a, int b, int l, int r, F pred) const {
    Info cur;
    return findFirst(a, b, 1, n, l, r, cur, pred);
}

```

```

template <class F>
int findLast(int a, int b, int l, int r, int x, int y, Info& cur, F& pred) const {
    if(r < x || y < l) return -1;
    if(x <= l && r <= y) {
        Info nxt = (tr[b].val - tr[a].val) + cur;
        if(!pred(nxt)) {
            cur = nxt;
            return -1;
        }
        if(l == r) return l;
    }
    int m = (l + r) >> 1;
    int p = findLast(tr[a].rs, tr[b].rs, m + 1, r, x, y, cur, pred);
    if(p == -1) p = findLast(tr[a].ls, tr[b].ls, l, m, x, y, cur, pred);
    return p;
}
template <class F>
int findLast(int a, int b, int l, int r, F pred) const {
    Info cur;
    return findLast(a, b, 1, n, l, r, cur, pred);
}
};

```

动态开点线段树

坐标域为闭区间 $[L, R]$ ；根初始为 0；`modify(root, p, v)` 单点赋值并更新根。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

template <class Info>
struct DynamicSegmentTree {
    struct Node {
        int ls = 0, rs = 0;
        Info val;
    };

    i64 L, R;
    vector<Node> tr;

    DynamicSegmentTree(i64 l, i64 r) : L(l), R(r), tr(1) {}

    int newNode() {
        tr.push_back({});
        return int(tr.size()) - 1;
    }

    void pull(int p) {
        tr[p].val = tr[tr[p].ls].val + tr[tr[p].rs].val;
    }
}

```

```

int modify(int p, i64 l, i64 r, i64 x, const Info& v) {
    if(!p) p = newNode();
    if(l == r) {
        tr[p].val = v;
        return p;
    }
    i64 m = l + (r - l) / 2;
    if(x <= m) tr[p].ls = modify(tr[p].ls, l, m, x, v);
    else tr[p].rs = modify(tr[p].rs, m + 1, r, x, v);
    pull(p);
    return p;
}

void modify(int& root, i64 x, const Info& v) {
    root = modify(root, L, R, x, v);
}

Info rangeQuery(int p, i64 l, i64 r, i64 x, i64 y) const {
    if(!p) return Info();
    if(x <= l && r <= y) return tr[p].val;
    i64 m = l + (r - l) / 2;
    if(y <= m) return rangeQuery(tr[p].ls, l, m, x, y);
    if(x > m) return rangeQuery(tr[p].rs, m + 1, r, x, y);
    return rangeQuery(tr[p].ls, l, m, x, y)
        + rangeQuery(tr[p].rs, m + 1, r, x, y);
}

Info rangeQuery(int root, i64 l, i64 r) const {
    return rangeQuery(root, L, R, l, r);
}

};

```

吉司机线段树 (Segment Tree Beats)

1-based 闭区间; 输入 `a[0]` 不使用; 支持 `rangeChmin(l,r,x)`、`rangeSum(l,r)`、`rangeMax(l,r)`。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct SegmentTreeBeats {
    static constexpr i64 negInf = numeric_limits<i64>::lowest();
    struct Node {
        i64 sum = 0;
        i64 mx = negInf, se = negInf;
        int cnt = 0;
    };
};

int n;
vector<Node> tr;

SegmentTreeBeats(const vector<i64>& a) : n(int(a.size()) - 1), tr(4 * n) {

```

```

        build(1, 1, n, a);
    }

    void build(int p, int l, int r, const vector<i64>& a) {
        if(l == r) {
            tr[p] = {a[l], a[l], negInf, 1};
            return;
        }
        int m = (l + r) >> 1;
        build(p << 1, l, m, a);
        build(p << 1 | 1, m + 1, r, a);
        pull(p);
    }

    void pull(int p) {
        auto& a = tr[p << 1];
        auto& b = tr[p << 1 | 1];
        tr[p].sum = a.sum + b.sum;
        if(a.mx == b.mx) {
            tr[p].mx = a.mx;
            tr[p].se = max(a.se, b.se);
            tr[p].cnt = a.cnt + b.cnt;
        } else if(a.mx > b.mx) {
            tr[p].mx = a.mx;
            tr[p].se = max(a.se, b.mx);
            tr[p].cnt = a.cnt;
        } else {
            tr[p].mx = b.mx;
            tr[p].se = max(a.mx, b.se);
            tr[p].cnt = b.cnt;
        }
    }

    void applyChmin(int p, i64 x) {
        if(x >= tr[p].mx) return;
        tr[p].sum -= (tr[p].mx - x) * tr[p].cnt;
        tr[p].mx = x;
    }

    void push(int p) {
        applyChmin(p << 1, tr[p].mx);
        applyChmin(p << 1 | 1, tr[p].mx);
    }

    void rangeChmin(int p, int l, int r, int x, int y, i64 v) {
        if(r < x || y < l || v >= tr[p].mx) return;
        if(x <= l && r <= y && v > tr[p].se) {
            applyChmin(p, v);
            return;
        }
        push(p);
        int m = (l + r) >> 1;
        rangeChmin(p << 1, l, m, x, y, v);
        rangeChmin(p << 1 | 1, m + 1, r, x, y, v);
    }

```

```

    pull(p);
}
void rangeChmin(int l, int r, i64 v) {
    rangeChmin(1, 1, n, l, r, v);
}

i64 rangeSum(int p, int l, int r, int x, int y) {
    if(x <= l && r <= y) return tr[p].sum;
    push(p);
    int m = (l + r) >> 1;
    i64 res = 0;
    if(x <= m) res += rangeSum(p << 1, l, m, x, y);
    if(y > m) res += rangeSum(p << 1 | 1, m + 1, r, x, y);
    return res;
}
i64 rangeSum(int l, int r) {
    return rangeSum(1, 1, n, l, r);
}

i64 rangeMax(int p, int l, int r, int x, int y) {
    if(x <= l && r <= y) return tr[p].mx;
    push(p);
    int m = (l + r) >> 1;
    i64 res = negInf;
    if(x <= m) res = max(res, rangeMax(p << 1, l, m, x, y));
    if(y > m) res = max(res, rangeMax(p << 1 | 1, m + 1, r, x, y));
    return res;
}
i64 rangeMax(int l, int r) {
    return rangeMax(1, 1, n, l, r);
}
};

```

无旋 Treap (FHQ)

有序多重集合; `rank/kth` 为 1-based; `prev/next` 返回 `optional<int>`。

```

#include <bits/stdc++.h>
using namespace std;

struct FHQTreap {
    struct Node {
        int ls = 0, rs = 0;
        int val = 0, sz = 0;
        uint32_t pri = 0;
    };

    vector<Node> tr;
    int root = 0;
    mt19937 rng;

```

```

    FHQTreap(int q = 0, uint32_t seed =
uint32_t(chrono::steady_clock::now().time_since_epoch().count()))
    : tr(1), rng(seed) {
        tr.reserve(q + 1);
    }

    int size(int p) const {
        return tr[p].sz;
    }
    int size() const {
        return size(root);
    }
    int newNode(int v) {
        tr.push_back({0, 0, v, 1, rng()});
        return int(tr.size()) - 1;
    }
    void pull(int p) {
        tr[p].sz = size(tr[p].ls) + size(tr[p].rs) + 1;
    }

    void splitLess(int p, int v, int& x, int& y) {
        if(!p) {
            x = y = 0;
            return;
        }
        if(tr[p].val < v) {
            x = p;
            splitLess(tr[p].rs, v, tr[p].rs, y);
        } else {
            y = p;
            splitLess(tr[p].ls, v, x, tr[p].ls);
        }
        pull(p);
    }
    void splitLE(int p, int v, int& x, int& y) {
        if(!p) {
            x = y = 0;
            return;
        }
        if(tr[p].val <= v) {
            x = p;
            splitLE(tr[p].rs, v, tr[p].rs, y);
        } else {
            y = p;
            splitLE(tr[p].ls, v, x, tr[p].ls);
        }
        pull(p);
    }
    int merge(int x, int y) {
        if(!x || !y) return x | y;
        if(tr[x].pri > tr[y].pri) {
            tr[x].rs = merge(tr[x].rs, y);
            pull(x);
        }
    }

```

```

        return x;
    }
    tr[y].ls = merge(x, tr[y].ls);
    pull(y);
    return y;
}

void insert(int v) {
    int x, y;
    splitLE(root, v, x, y);
    root = merge(merge(x, newNode(v)), y);
}

bool erase(int v) {
    int x, y, z;
    splitLess(root, v, x, y);
    splitLE(y, v, y, z);
    bool ok = y != 0;
    if(y) y = merge(tr[y].ls, tr[y].rs);
    root = merge(x, merge(y, z));
    return ok;
}

int rank(int v) {
    int x, y;
    splitLess(root, v, x, y);
    int res = size(x) + 1;
    root = merge(x, y);
    return res;
}

int kth(int k) const {
    int p = root;
    while(p) {
        int s = size(tr[p].ls);
        if(k == s + 1) return tr[p].val;
        if(k <= s) p = tr[p].ls;
        else {
            k -= s + 1;
            p = tr[p].rs;
        }
    }
    assert(false);
    return 0;
}

optional<int> prev(int v) {
    int x, y;
    splitLess(root, v, x, y);
    int p = x;
    while(p && tr[p].rs) p = tr[p].rs;
    optional<int> res;
    if(p) res = tr[p].val;
    root = merge(x, y);
    return res;
}

optional<int> next(int v) {

```

```

        int x, y;
        splitLE(root, v, x, y);
        int p = y;
        while(p && tr[p].ls) p = tr[p].ls;
        optional<int> res;
        if(p) res = tr[p].val;
        root = merge(x, y);
        return res;
    }
};

```

隐式 Treap

序列接口 1-based 闭区间; `insertAfter(root, pos, mid)` 的 `pos` 是前缀长度。 `compose(old, new)` 表示旧操作后追加 new; 翻转时同步 `reverseInfo`。

```

#include <bits/stdc++.h>
using namespace std;

template <class Policy>
struct ImplicitTreap {
    using Val = typename Policy::Val;
    using Info = typename Policy::Info;
    using Tag = typename Policy::Tag;
    struct Node {
        int ls = 0, rs = 0, sz = 1;
        uint32_t pri = 0;
        Val val{};
        Info info{};
        Tag tag{};
        bool rev = false;
    };

    vector<Node> tr;
    mt19937 rng;

    ImplicitTreap(int q = 0, uint32_t seed =
uint32_t(chrono::steady_clock::now().time_since_epoch().count()))
        : tr(1), rng(seed) {
        tr.reserve(q + 1);
        tr[0].sz = 0;
        tr[0].info = Policy::identity();
        tr[0].tag = Policy::tagIdentity();
    }

    int size(int p) const {
        return tr[p].sz;
    }

    Info info(int p) const {
        return p ? tr[p].info : Policy::identity();
    }

    int newNode(const Val& v) {

```

```

    tr.push_back({});
    int p = int(tr.size()) - 1;
    tr[p].pri = rng();
    tr[p].val = v;
    tr[p].info = Policy::make(v);
    tr[p].tag = Policy::tagIdentity();
    return p;
}

void pull(int p) {
    tr[p].sz = size(tr[p].ls) + size(tr[p].rs) + 1;
    tr[p].info = Policy::merge(
        Policy::merge(info(tr[p].ls), Policy::make(tr[p].val)), info(tr[p].rs));
}

void applyTag(int p, const Tag& tag) {
    if(!p || Policy::tagEmpty(tag)) return;
    Policy::apply(tr[p].val, tr[p].info, tr[p].sz, tag);
    Policy::compose(tr[p].tag, tag);
}

void applyRev(int p) {
    if(!p) return;
    swap(tr[p].ls, tr[p].rs);
    tr[p].rev ^= 1;
    Policy::reverseInfo(tr[p].info);
}

void push(int p) {
    if(tr[p].rev) {
        applyRev(tr[p].ls);
        applyRev(tr[p].rs);
        tr[p].rev = false;
    }
    if(!Policy::tagEmpty(tr[p].tag)) {
        applyTag(tr[p].ls, tr[p].tag);
        applyTag(tr[p].rs, tr[p].tag);
        tr[p].tag = Policy::tagIdentity();
    }
}

pair<int, int> split(int p, int k) {
    if(!p) return {0, 0};
    push(p);
    if(size(tr[p].ls) >= k) {
        auto [x, y] = split(tr[p].ls, k);
        tr[p].ls = y;
        pull(p);
        return {x, p};
    }
    auto [x, y] = split(tr[p].rs, k - size(tr[p].ls) - 1);
    tr[p].rs = x;
    pull(p);
    return {p, y};
}

int merge(int x, int y) {
    if(!x || !y) return x | y;

```

```

        if(tr[x].pri > tr[y].pri) {
            push(x);
            tr[x].rs = merge(tr[x].rs, y);
            pull(x);
            return x;
        }
        push(y);
        tr[y].ls = merge(x, tr[y].ls);
        pull(y);
        return y;
    }
    int build(const vector<Val>& a) {
        int root = 0;
        for(const Val& v : a) root = merge(root, newNode(v));
        return root;
    }
    int kth(int root, int k) {
        int p = root;
        while(p) {
            push(p);
            int s = size(tr[p].ls);
            if(k == s + 1) return p;
            if(k <= s) p = tr[p].ls;
            else {
                k -= s + 1;
                p = tr[p].rs;
            }
        }
        return 0;
    }
    tuple<int, int, int> splitRange(int root, int l, int r) {
        auto [a, bc] = split(root, l - 1);
        auto [b, c] = split(bc, r - l + 1);
        return {a, b, c};
    }
    Info rangeQuery(int& root, int l, int r) {
        auto [a, b, c] = splitRange(root, l, r);
        Info res = info(b);
        root = merge(a, merge(b, c));
        return res;
    }
    void rangeApply(int& root, int l, int r, const Tag& tag) {
        auto [a, b, c] = splitRange(root, l, r);
        applyTag(b, tag);
        root = merge(a, merge(b, c));
    }
    void rangeReverse(int& root, int l, int r) {
        auto [a, b, c] = splitRange(root, l, r);
        applyRev(b);
        root = merge(a, merge(b, c));
    }
    int eraseRange(int& root, int l, int r) {
        auto [a, b, c] = splitRange(root, l, r);

```

```

        root = merge(a, c);
        return b;
    }
    void insertAfter(int& root, int pos, int mid) {
        auto [a, b] = split(root, pos);
        root = merge(merge(a, mid), b);
    }
};

```

有序多重集 (PBDS)

GNU only; `kth` 为 1-based, `orderOfKey(x)` 统计 `<x`。

```

#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;

template <class T>
struct OrderedMultiset {
    using Key = pair<T, int>;
    tree<Key, null_type, less<Key>, rb_tree_tag, tree_order_statistics_node_update> tr;
    int uid = 0;

    int size() const {
        return int(tr.size());
    }
    void insert(const T& x) {
        tr.insert({x, uid++});
    }
    bool erase(const T& x) {
        auto it = tr.lower_bound({x, numeric_limits<int>::min()});
        if(it == tr.end() || it->first != x) return false;
        tr.erase(it);
        return true;
    }
    int orderOfKey(const T& x) const {
        return int(tr.order_of_key({x, numeric_limits<int>::min()}));
    }
    int count(const T& x) const {
        return int(tr.order_of_key({x, numeric_limits<int>::max()})) - orderOfKey(x);
    }
    optional<T> kth(int k) const {
        if(k < 1 || k > size()) return nullopt;
        return tr.find_by_order(k - 1)->first;
    }
};

```

李超线段树

整数闭域 $[L, R]$ ；加入整域直线并查询最小值，返回 `__int128`；最大值整体取负。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct LiChaoTree {
    using i128 = __int128;
    static constexpr i128 inf = i128(1) << 120;
    struct Line {
        i64 k = 0, b = 0;
        i128 get(i64 x) const {
            return i128(k) * x + b;
        }
    };
    struct Node {
        int ls = 0, rs = 0;
        Line line;
        bool has = false;
    };

    i64 L, R;
    vector<Node> tr;
    int root = 0;

    LiChaoTree(i64 l, i64 r) : L(l), R(r), tr(1) {}

    int newNode(Line line) {
        tr.push_back({0, 0, line, true});
        return int(tr.size()) - 1;
    }

    int addLine(int p, i64 l, i64 r, Line line) {
        if(!p) return newNode(line);
        i64 m = l + (r - l) / 2;
        bool left = line.get(l) < tr[p].line.get(l);
        bool mid = line.get(m) < tr[p].line.get(m);
        if(mid) swap(line, tr[p].line);
        if(l == r) return p;
        if(left != mid) {
            int child = addLine(tr[p].ls, l, m, line);
            tr[p].ls = child;
        } else {
            int child = addLine(tr[p].rs, m + 1, r, line);
            tr[p].rs = child;
        }
        return p;
    }

    void addLine(i64 k, i64 b) {
        root = addLine(root, L, R, {k, b});
    }

    i128 query(int p, i64 l, i64 r, i64 x) const {
        if(!p) return inf;
        i128 res = tr[p].line.get(x);
    }

```

```

        if(l == r) return res;
        i64 m = l + (r - l) / 2;
        if(x <= m) return min(res, query(tr[p].ls, l, m, x));
        return min(res, query(tr[p].rs, m + 1, r, x));
    }
    i128 query(i64 x) const {
        return query(root, L, R, x);
    }
};

```

凸包优化 DP

候选点按 x 非递减加入；`query(q)` 求 `min dot(p,q)`，要求 `q.y>0`。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct ConvexHullDP {
    using i128 = __int128;
    struct Point {
        i64 x, y;
        Point operator-(const Point& p) const {
            return {x - p.x, y - p.y};
        }
    };
    vector<Point> hull;

    static i128 cross(Point a, Point b) {
        return i128(a.x) * b.y - i128(a.y) * b.x;
    }
    static i128 dot(Point a, Point b) {
        return i128(a.x) * b.x + i128(a.y) * b.y;
    }
    void addPoint(Point p) {
        if(!hull.empty() && hull.back().x == p.x) {
            if(hull.back().y <= p.y) return;
            hull.pop_back();
        }
        while(hull.size() >= 2
            && cross(hull.back() - hull[hull.size() - 2], p - hull.back()) <= 0) {
            hull.pop_back();
        }
        hull.push_back(p);
    }
    i128 query(Point q) const {
        int l = 0, r = int(hull.size()) - 1;
        while(l < r) {
            int m = (l + r) >> 1;
            if(dot(hull[m], q) <= dot(hull[m + 1], q)) r = m;
            else l = m + 1;
        }
    }
};

```

```

    }
    return dot(hull[1], q);
}
};

```

笛卡尔树

1-based 最小笛卡尔树，`a[0]` 不使用，0 表示无父子；相等值保留靠左者为祖先。

```

#include <bits/stdc++.h>
using namespace std;

template<class T, class Compare = less<T>>
struct CartesianTree {
    int root = 0;
    vector<int> parent, left, right;

    CartesianTree(const vector<T>& a, Compare cmp = Compare())
        : parent(a.size()), left(a.size()), right(a.size()) {
        vector<int> st;
        for(int i = 1; i < int(a.size()); i++) {
            int last = 0;
            while(!st.empty() && cmp(a[i], a[st.back()])) {
                last = st.back();
                st.pop_back();
            }
            if(!st.empty()) {
                parent[i] = st.back();
                right[st.back()] = i;
            }
            if(last) {
                parent[last] = i;
                left[i] = last;
            }
            st.push_back(i);
        }
        if(!st.empty()) root = st.front();
    }
};

```

动态树 (LCT)

1-based 动态森林；路径包含端点。`link/cut` 返回是否成功；维护路径异或。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct LinkCutTree {
    struct Node {

```

```

    int ch[2][{}], fa = 0;
    i64 val = 0, sum = 0;
    bool rev = false;
};

vector<Node> tr;

LinkCutTree(int n = 0) : tr(n + 1) {}

bool isRoot(int x) const {
    int f = tr[x].fa;
    return !f || (tr[f].ch[0] != x && tr[f].ch[1] != x);
}

void pull(int x) {
    tr[x].sum = tr[tr[x].ch[0]].sum ^ tr[x].val ^ tr[tr[x].ch[1]].sum;
}

void applyRev(int x) {
    if(!x) return;
    swap(tr[x].ch[0], tr[x].ch[1]);
    tr[x].rev ^= 1;
}

void push(int x) {
    if(!tr[x].rev) return;
    applyRev(tr[x].ch[0]);
    applyRev(tr[x].ch[1]);
    tr[x].rev = false;
}

void rotate(int x) {
    int y = tr[x].fa, z = tr[y].fa;
    int k = tr[y].ch[1] == x;
    if(!isRoot(y)) tr[z].ch[tr[z].ch[1] == y] = x;
    tr[x].fa = z;
    tr[y].ch[k] = tr[x].ch[k ^ 1];
    if(tr[x].ch[k ^ 1]) tr[tr[x].ch[k ^ 1]].fa = y;
    tr[x].ch[k ^ 1] = y;
    tr[y].fa = x;
    pull(y);
    pull(x);
}

void splay(int x) {
    vector<int> st{x};
    for(int y = x; !isRoot(y); y = tr[y].fa) st.push_back(tr[y].fa);
    while(!st.empty()) {
        push(st.back());
        st.pop_back();
    }
    while(!isRoot(x)) {
        int y = tr[x].fa, z = tr[y].fa;
        if(!isRoot(y)) {
            if((tr[y].ch[1] == x) == (tr[z].ch[1] == y)) rotate(y);
            else rotate(x);
        }
        rotate(x);
    }
}

```

```

int access(int x) {
    int y = 0;
    for(int z = x; z; z = tr[z].fa) {
        splay(z);
        tr[z].ch[1] = y;
        pull(z);
        y = z;
    }
    splay(x);
    return y;
}

void makeRoot(int x) {
    access(x);
    applyRev(x);
}

int findRoot(int x) {
    access(x);
    while(tr[x].ch[0]) {
        push(x);
        x = tr[x].ch[0];
    }
    splay(x);
    return x;
}

bool connected(int x, int y) {
    return x == y || findRoot(x) == findRoot(y);
}

bool link(int x, int y) {
    makeRoot(x);
    if(findRoot(y) == x) return false;
    tr[x].fa = y;
    return true;
}

bool cut(int x, int y) {
    makeRoot(x);
    access(y);
    if(tr[y].ch[0] != x || tr[x].ch[1]) return false;
    tr[y].ch[0] = tr[x].fa = 0;
    pull(y);
    return true;
}

void setValue(int x, i64 value) {
    access(x);
    tr[x].val = value;
    pull(x);
}

i64 pathXor(int x, int y) {
    makeRoot(x);
    access(y);
    return tr[y].sum;
}
};

```

按不同键维护 Top-K

维护“不同 key 各自最优值”的前 K 大；`get(...)` 可排除至多 2 个禁用 key。无候选返回 `-INF`；需要“0 即无答案”语义时先 `add(-1, 0)`。key 为 `-1` 时需改哨兵。

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

// 不同 key 的前 K 大值；空槽 key = -1、val = -INF（key 可能为 -1 时需改哨兵）
template <int K>
struct TopK {
    static constexpr i64 INF = numeric_limits<i64>::max() / 4;

    struct Node {
        i64 val = -INF;
        int key = -1;
    };

    array<Node, K> a;

    void norm() {
        sort(a.begin(), a.end(), [](const Node& x, const Node& y) {
            return x.val > y.val;
        });
    }

    // 插入 (key, val)：同 key 保留较大值；否则尝试替换第 K 小
    void add(int key, i64 val) {
        for(auto& x : a) {
            if(x.key == key) {
                x.val = max(x.val, val);
                norm();
                return;
            }
        }
        if(val > a[K - 1].val) {
            a[K - 1] = {val, key};
            norm();
        }
    }

    // 返回未被禁用 key 的最大 val；无候选时返回 -INF
    i64 get(int ban1 = -1, int ban2 = -1) const {
        for(const auto& x : a) {
            if(x.key != ban1 && x.key != ban2) {
                return x.val;
            }
        }
        return -INF;
    }
};
```

```
};
```

2. 图论

单源最短路 (Dijkstra)

点 1-based; 非负边权; 不可达为 `inf`。

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct Dijkstra {
    struct Edge {
        int to;
        i64 w;
    };
    static constexpr i64 inf = numeric_limits<i64>::max() / 4;
    vector<vector<Edge>> adj;
    Dijkstra(int n = 0) : adj(n + 1) {}
    void addEdge(int u, int v, i64 w) {
        adj[u].push_back({v, w});
    }
    vector<i64> run(int s) const {
        vector<i64> dis(adj.size(), inf);
        priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<>> q;
        dis[s] = 0;
        q.push({0, s});
        while(!q.empty()) {
            auto [distance, u] = q.top();
            q.pop();
            if(distance != dis[u]) {
                continue;
            }
            for(auto [v, w] : adj[u]) {
                if(dis[v] > distance + w) {
                    dis[v] = distance + w;
                    q.push({dis[v], v});
                }
            }
        }
        return dis;
    }
};
```

0-1 最短路 (BFS)

点 1-based; 边权只能为 0/1。

```
#include <bits/stdc++.h>
```

```

using namespace std;

struct ZeroOneBFS {
    struct Edge {
        int to, w;
    };
    static constexpr int inf = numeric_limits<int>::max() / 4;
    vector<vector<Edge>> adj;
    ZeroOneBFS(int n = 0) : adj(n + 1) {}
    void addEdge(int u, int v, int w) {
        adj[u].push_back({v, w});
    }
    vector<int> run(int s) const {
        vector<int> dis(adj.size(), inf);
        deque<int> q;
        dis[s] = 0;
        q.push_back(s);
        while(!q.empty()) {
            int u = q.front();
            q.pop_front();
            for(auto [v, w] : adj[u]) {
                if(dis[v] > dis[u] + w) {
                    dis[v] = dis[u] + w;
                    if(w) {
                        q.push_back(v);
                    } else {
                        q.push_front(v);
                    }
                }
            }
        }
        return dis;
    }
};

```

全源最短路 (Floyd)

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

void floyd(vector<vector<i64>>& dis, i64 inf) {
    int n = int(dis.size()) - 1;
    for(int k = 1; k <= n; k++) {
        for(int i = 1; i <= n; i++) {
            if(dis[i][k] == inf) {
                continue;
            }
            for(int j = 1; j <= n; j++) {
                if(dis[k][j] == inf) {
                    continue;
                }
            }
        }
    }
}

```

```

        }
        dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
    }
}
}
}
}

```

拓扑排序

点 1-based, `adj[0]` 不使用; 有环返回空数组。

```

#include <bits/stdc++.h>
using namespace std;

vector<int> topologicalSort(const vector<vector<int>>& adj) {
    vector<int> deg(adj.size());
    int n = int(adj.size()) - 1;
    for(int u = 1; u <= n; u++) {
        for(int v : adj[u]) {
            deg[v]++;
        }
    }
    queue<int> q;
    for(int i = 1; i <= n; i++) {
        if(!deg[i]) {
            q.push(i);
        }
    }
    vector<int> order;
    while(!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);
        for(int v : adj[u]) {
            if(--deg[v] == 0) {
                q.push(v);
            }
        }
    }
    if(int(order.size()) != n) {
        return {};
    }
    return order;
}

```

最小生成树 (Kruskal)

返回最小生成森林权值和原边编号。

```

#include <bits/stdc++.h>
using namespace std;

```

```

using i64 = long long;

struct Kruskal {
    struct Edge {
        int u, v;
        i64 w;
    };
    static pair<i64, vector<int>> run(int n, const vector<Edge>& edges) {
        vector<int> p(n + 1), sz(n + 1, 1), id(edges.size());
        iota(p.begin(), p.end(), 0);
        iota(id.begin(), id.end(), 0);
        sort(id.begin(), id.end(), [&](int i, int j) {
            return edges[i].w < edges[j].w;
        });
        auto find = [&](int x) {
            while(x != p[x]) {
                x = p[x] = p[p[x]];
            }
            return x;
        };
        i64 sum = 0;
        vector<int> chosen;
        for(int i : id) {
            int x = find(edges[i].u);
            int y = find(edges[i].v);
            if(x == y) {
                continue;
            }
            if(sz[x] < sz[y]) {
                swap(x, y);
            }
            p[y] = x;
            sz[x] += sz[y];
            sum += edges[i].w;
            chosen.push_back(i + 1);
        }
        return {sum, chosen};
    }
};

```

二分图度序列构造 (Gale-Ryser)

左点 $1..n$ 、右点 $1..m$ ；`row[i] / col[j]` 为两侧度数，和必须相等；`run` 返回是否可行，可行时 `mat[i][j]=1` 表示连边。模板不处理预先固定/禁用的格子。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

// Gale-Ryser 构造：逐行取剩余度数最大的若干列（按剩余度分组，从组内右侧取）

```

```

struct BipartiteDegreeSequence {
    static bool run(int n, int m, const vector<int>& row, const vector<int>& col,
                    vector<vector<int>>& mat) {
        mat.assign(n + 1, vector<int>(m + 1, 0));
        vector<int> csum(m + 1);
        i64 total = 0;
        for(int i = 1; i <= n; i++) {
            if(row[i] < 0 || row[i] > m) {
                return false;
            }
            total += row[i];
        }
        for(int j = 1; j <= m; j++) {
            if(col[j] < 0 || col[j] > n) {
                return false;
            }
            csum[j] = col[j];
            total -= col[j];
        }
        if(total != 0) {
            return false;
        }

        vector<int> ord(m);
        iota(ord.begin(), ord.end(), 1);
        sort(ord.begin(), ord.end(), [&](int a, int b) {
            return csum[a] > csum[b];
        });

        for(int i = 1; i <= n; i++) {
            int cnt = row[i];
            for(int lo = 0, hi = 0; lo < m && cnt > 0; lo = hi) {
                if(csum[ord[lo]] == 0) {
                    break;
                }
                while(hi < m && csum[ord[hi]] == csum[ord[lo]]) {
                    hi++;
                }
                for(int k = hi - 1; k >= lo && cnt > 0; k--) {
                    mat[i][ord[k]] = 1;
                    csum[ord[k]]--;
                    cnt--;
                }
            }
            if(cnt > 0) {
                return false;
            }
        }
        for(int j = 1; j <= m; j++) {
            if(csum[j] != 0) {
                return false;
            }
        }
    }
}

```

```

        return true;
    }
};

```

增量 APSP (只加边)

点 $1..n$; `addEdge(u,v,w)` 无向、`addDirected` 有向, 每条边 $O(n^2)$ 松弛, 查询直接读 `dis[i][j]`; 不可达为 `INF`。只支持加边, n 通常 ≤ 500 。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct IncrementalAPSP {
    static constexpr i64 INF = numeric_limits<i64>::max() / 4;

    int n;
    vector<vector<i64>> dis;

    IncrementalAPSP(int n_) : n(n_), dis(n_ + 1, vector<i64>(n_ + 1, INF)) {
        for(int i = 1; i <= n; i++) {
            dis[i][i] = 0;
        }
    }

    void addEdge(int u, int v, i64 w) {
        relax(u, v, w);
        relax(v, u, w);
    }

    void addDirected(int u, int v, i64 w) {
        relax(u, v, w);
    }

    void relax(int u, int v, i64 w) {
        for(int i = 1; i <= n; i++) {
            if(dis[i][u] == INF) {
                continue;
            }
            for(int j = 1; j <= n; j++) {
                if(dis[v][j] == INF) {
                    continue;
                }
                if(dis[i][j] > dis[i][u] + w + dis[v][j]) {
                    dis[i][j] = dis[i][u] + w + dis[v][j];
                }
            }
        }
    }
};

```

强连通分量 (SCC)

点 1-based; `id` 是 0-based 内部分量编号。

```
#include <bits/stdc++.h>
using namespace std;

struct SCC {
    int n, timer = 0, count = 0;
    const vector<vector<int>>& adj;
    vector<int> dfn, low, stack, id;
    vector<bool> inStack;
    vector<vector<int>> components;

    SCC(const vector<vector<int>>& graph)
        : n(int(graph.size()) - 1), adj(graph), dfn(graph.size()), low(graph.size()),
          id(graph.size(), -1), inStack(graph.size()) {
        for(int u = 1; u <= n; u++) {
            if(!dfn[u]) {
                dfs(u);
            }
        }
    }

    void dfs(int u) {
        dfn[u] = low[u] = ++timer;
        stack.push_back(u);
        inStack[u] = true;
        for(int v : adj[u]) {
            if(!dfn[v]) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if(inStack[v]) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if(low[u] != dfn[u]) {
            return;
        }
        components.push_back({});
        while(true) {
            int v = stack.back();
            stack.pop_back();
            inStack[v] = false;
            id[v] = count;
            components.back().push_back(v);
            if(v == u) {
                break;
            }
        }
        count++;
    }
};
```

二元可满足性 (2-SAT)

变量 1-based; Tarjan 已内置; `solve()` 不可行返回 false。

```
#include <bits/stdc++.h>
using namespace std;

struct TwoSAT {
    int n;
    vector<vector<int>> adj;
    vector<bool> answer;
    vector<int> dfn, low, stack, id;
    vector<bool> inStack;
    int timer = 0, count = 0;

    TwoSAT(int n_ = 0)
        : n(n_), adj(2 * n_ + 2), answer(n_ + 1),
          dfn(2 * n_ + 2), low(2 * n_ + 2), id(2 * n_ + 2, -1),
          inStack(2 * n_ + 2) {}
    int literal(int x, bool value) const {
        return 2 * x + int(value);
    }
    void imply(int a, int b) {
        adj[a].push_back(b);
    }
    void addClause(int x, bool xValue, int y, bool yValue) {
        int a = literal(x, xValue);
        int b = literal(y, yValue);
        imply(a ^ 1, b);
        imply(b ^ 1, a);
    }
    void setValue(int x, bool value) {
        addClause(x, value, x, value);
    }
    bool solve() {
        for(int u = 2; u <= 2 * n + 1; u++) {
            if(!dfn[u]) {
                dfs(u);
            }
        }
        for(int i = 1; i <= n; i++) {
            if(id[2 * i] == id[2 * i + 1]) {
                return false;
            }
            answer[i] = id[2 * i] > id[2 * i + 1];
        }
        return true;
    }

private:
    void dfs(int u) {
        dfn[u] = low[u] = ++timer;
```

```

        stack.push_back(u);
        inStack[u] = true;
        for(int v : adj[u]) {
            if(!dfn[v]) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if(inStack[v]) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if(low[u] != dfn[u]) {
            return;
        }
        while(true) {
            int v = stack.back();
            stack.pop_back();
            inStack[v] = false;
            id[v] = count;
            if(v == u) {
                break;
            }
        }
        count++;
    }
};

```

桥与割点

点、边均 1-based；支持重边、自环、非连通图。

```

#include <bits/stdc++.h>
using namespace std;

struct BridgeArticulation {
    int n, timer = 0;
    vector<pair<int, int>> edges;
    vector<vector<pair<int, int>>> adj;
    vector<int> dfn, low;
    vector<bool> isBridge, isArticulation;

    BridgeArticulation(int n_ = 0)
        : n(n_), edges(1), adj(n_ + 1), dfn(n_ + 1), low(n_ + 1),
          isArticulation(n_ + 1) {}

    int addEdge(int u, int v) {
        int id = int(edges.size());
        edges.push_back({u, v});
        adj[u].push_back({v, id});
        adj[v].push_back({u, id});
        return id;
    }

    void work() {
        isBridge.assign(edges.size(), false);
    }
};

```

```

        for(int u = 1; u <= n; u++) {
            if(!dfn[u]) {
                dfs(u, 0);
            }
        }
    }

    void dfs(int u, int parentEdge) {
        dfn[u] = low[u] = ++timer;
        int child = 0;
        for(auto [v, id] : adj[u]) {
            if(id == parentEdge) {
                continue;
            }
            if(!dfn[v]) {
                child++;
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if(low[v] > dfn[u]) {
                    isBridge[id] = true;
                }
                if(parentEdge != 0 && low[v] >= dfn[u]) {
                    isArticulation[u] = true;
                }
            } else {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if(parentEdge == 0 && child > 1) {
            isArticulation[u] = true;
        }
    }
};

```

边双连通分量 (EBCC)

点、边、分量均 1-based; 割桥代码内置; `id[u]` 为删桥后的连通块。

```

#include <bits/stdc++.h>
using namespace std;

struct BridgeArticulation {
    int n, timer = 0;
    vector<pair<int, int>> edges;
    vector<vector<pair<int, int>>> adj;
    vector<int> dfn, low;
    vector<bool> isBridge, isArticulation;

    BridgeArticulation(int n_ = 0)
        : n(n_), edges(1), adj(n_ + 1), dfn(n_ + 1), low(n_ + 1),
          isArticulation(n_ + 1) {}
    int addEdge(int u, int v) {
        int id = int(edges.size());

```

```

        edges.push_back({u, v});
        adj[u].push_back({v, id});
        adj[v].push_back({u, id});
        return id;
    }

    void work() {
        isBridge.assign(edges.size(), false);
        for(int u = 1; u <= n; u++) {
            if(!dfn[u]) {
                dfs(u, 0);
            }
        }
    }

    void dfs(int u, int parentEdge) {
        dfn[u] = low[u] = ++timer;
        int child = 0;
        for(auto [v, id] : adj[u]) {
            if(id == parentEdge) {
                continue;
            }
            if(!dfn[v]) {
                child++;
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if(low[v] > dfn[u]) {
                    isBridge[id] = true;
                }
                if(parentEdge != 0 && low[v] >= dfn[u]) {
                    isArticulation[u] = true;
                }
            } else {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if(parentEdge == 0 && child > 1) {
            isArticulation[u] = true;
        }
    }
};

struct EdgeBCC {
    BridgeArticulation bridge;
    vector<int> id;
    vector<vector<int>> components;

    EdgeBCC(int n = 0) : bridge(n), id(n + 1), components(1) {}
    int addEdge(int u, int v) {
        return bridge.addEdge(u, v);
    }

    void work() {
        bridge.work();
        for(int s = 1; s <= bridge.n; s++) {
            if(id[s] != 0) {

```



```

        dfn[u] = low[u] = ++timer;
        components.push_back({u});
    } else {
        dfs(u, 0);
    }
}
blockCutTree.assign(n + components.size(), {});
for(int i = 1; i < int(components.size()); i++) {
    int componentNode = n + i;
    for(int u : components[i]) {
        blockCutTree[u].push_back(componentNode);
        blockCutTree[componentNode].push_back(u);
    }
}
}

void dfs(int u, int parentEdge) {
    dfn[u] = low[u] = ++timer;
    int child = 0;
    for(auto [v, id] : adj[u]) {
        if(id == parentEdge) {
            continue;
        }
        if(!dfn[v]) {
            child++;
            edgeStack.push_back(id);
            dfs(v, id);
            low[u] = min(low[u], low[v]);
            if(low[v] >= dfn[u]) {
                if(parentEdge != 0 || child > 1) {
                    isArticulation[u] = true;
                }
                vector<int> vertices;
                while(true) {
                    int edge = edgeStack.back();
                    edgeStack.pop_back();
                    vertices.push_back(edges[edge].first);
                    vertices.push_back(edges[edge].second);
                    if(edge == id) {
                        break;
                    }
                }
                sort(vertices.begin(), vertices.end());
                vertices.erase(unique(vertices.begin(), vertices.end()),
vertices.end());
                components.push_back(move(vertices));
            }
        } else if(dfn[v] < dfn[u]) {
            edgeStack.push_back(id);
            low[u] = min(low[u], dfn[v]);
        }
    }
}

};

```

欧拉路径

点 1-based; 成功返回点序列, 失败返回 `nullopt`。

```
#include <bits/stdc++.h>
using namespace std;

struct EulerTrail {
    static optional<vector<int>> directed(int n, const vector<pair<int, int>>& edges) {
        if(n == 0) {
            return edges.empty() ? optional(vector<int>{}) : nullopt;
        }
        vector<vector<pair<int, int>>> adj(n + 1);
        vector<int> in(n + 1), out(n + 1);
        for(int i = 0; i < int(edges.size()); i++) {
            auto [u, v] = edges[i];
            adj[u].push_back({v, i});
            out[u]++;
            in[v]++;
        }
        int start = -1;
        int end = -1;
        for(int u = 1; u <= n; u++) {
            if(out[u] - in[u] == 1) {
                if(start != -1) {
                    return nullopt;
                }
                start = u;
            } else if(in[u] - out[u] == 1) {
                if(end != -1) {
                    return nullopt;
                }
                end = u;
            } else if(in[u] != out[u]) {
                return nullopt;
            }
        }
        if((start == -1) != (end == -1)) {
            return nullopt;
        }
        if(start == -1) {
            start = 1;
            for(int u = 1; u <= n; u++) {
                if(out[u]) {
                    start = u;
                    break;
                }
            }
        }
        return hierholzer(adj, int(edges.size()), start);
    }
    static optional<vector<int>> undirected(int n, const vector<pair<int, int>>& edges) {
```

```

if(n == 0) {
    return edges.empty() ? optional<vector<int>{}> : nullopt;
}
vector<vector<pair<int, int>>> adj(n + 1);
vector<int> degree(n + 1);
for(int i = 0; i < int(edges.size()); i++) {
    auto [u, v] = edges[i];
    adj[u].push_back({v, i});
    adj[v].push_back({u, i});
    degree[u]++;
    degree[v]++;
}
vector<int> odd;
for(int u = 1; u <= n; u++) {
    if(degree[u] & 1) {
        odd.push_back(u);
    }
}
if(odd.size() != 0 && odd.size() != 2) {
    return nullopt;
}
int start = odd.empty() ? 1 : odd[0];
if(odd.empty()) {
    for(int u = 1; u <= n; u++) {
        if(degree[u]) {
            start = u;
            break;
        }
    }
}
return hierholzer(adj, int(edges.size()), start);
}

```

private:

```

static optional<vector<int>> hierholzer(const vector<vector<pair<int, int>>>& adj,
                                       int edgeCount, int start) {

    vector<int> iter(adj.size());
    vector<bool> used(edgeCount);
    vector<int> stack{start};
    vector<int> path;
    while(!stack.empty()) {
        int u = stack.back();
        while(iter[u] < int(adj[u].size()) && used[adj[u][iter[u]].second]) {
            iter[u]++;
        }
        if(iter[u] == int(adj[u].size())) {
            path.push_back(u);
            stack.pop_back();
            continue;
        }
        auto [v, id] = adj[u][iter[u]++];
        used[id] = true;
        stack.push_back(v);
    }
}

```

```

    }
    if(int(path.size()) != edgeCount + 1) {
        return nullopt;
    }
    reverse(path.begin(), path.end());
    return path;
}
};

```

二分图最大匹配

左部向右部加边; `matchLeft/matchRight` 未匹配为 0。

```

#include <bits/stdc++.h>
using namespace std;

struct BipartiteMatching {
    int leftSize, rightSize;
    vector<vector<int>> adj;
    vector<int> matchLeft, matchRight, visit;
    int timer = 0;

    BipartiteMatching(int leftSize_ = 0, int rightSize_ = 0)
        : leftSize(leftSize_), rightSize(rightSize_), adj(leftSize_ + 1),
          matchLeft(leftSize_ + 1), matchRight(rightSize_ + 1), visit(leftSize_ + 1) {}

    void addEdge(int u, int v) {
        adj[u].push_back(v);
    }

    int maxMatching() {
        int answer = 0;
        for(int u = 1; u <= leftSize; u++) {
            timer++;
            if(dfs(u)) {
                answer++;
            }
        }
        return answer;
    }

private:
    bool dfs(int u) {
        if(visit[u] == timer) {
            return false;
        }
        visit[u] = timer;
        for(int v : adj[u]) {
            if(matchRight[v] == 0 || dfs(matchRight[v])) {
                matchLeft[u] = v;
                matchRight[v] = u;
                return true;
            }
        }
    }
}

```

```

        return false;
    }
};

```

二分图最大权匹配 (Hungarian)

矩形最大权匹配，匹配 $\min(n,m)$ 对；返回 `rowToColumn`，行多于列时未匹配行为 0。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct Hungarian {
    static pair<i64, vector<int>> maximum(const vector<vector<i64>>& weight) {
        int n = int(weight.size()) - 1;
        int m = n ? int(weight[1].size()) - 1 : 0;
        if(n == 0 || m == 0) {
            return {0, vector<int>(n + 1)};
        }
        if(n <= m) {
            return solve(weight);
        }
        vector<vector<i64>> transposed(m + 1, vector<i64>(n + 1));
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j <= m; j++) {
                transposed[j][i] = weight[i][j];
            }
        }
        auto [value, columnToRow] = solve(transposed);
        vector<int> rowToColumn(n + 1);
        for(int column = 1; column <= m; column++) {
            rowToColumn[columnToRow[column]] = column;
        }
        return {value, rowToColumn};
    }

private:
    static pair<i64, vector<int>> solve(const vector<vector<i64>>& weight) {
        int n = int(weight.size()) - 1;
        int m = int(weight[1].size()) - 1;
        const i64 inf = numeric_limits<i64>::max() / 4;
        vector<i64> rowPotential(n + 1), columnPotential(m + 1);
        vector<int> matchColumn(m + 1), previous(m + 1);
        for(int row = 1; row <= n; row++) {
            matchColumn[0] = row;
            int column = 0;
            vector<i64> minValue(m + 1, inf);
            vector<bool> used(m + 1);
            do {
                used[column] = true;
                int currentRow = matchColumn[column];

```

```

        i64 delta = inf;
        int nextColumn = 0;
        for(int j = 1; j <= m; j++) {
            if(used[j]) {
                continue;
            }
            i64 current = -weight[currentRow][j]
                - rowPotential[currentRow] - columnPotential[j];
            if(current < minValue[j]) {
                minValue[j] = current;
                previous[j] = column;
            }
            if(minValue[j] < delta) {
                delta = minValue[j];
                nextColumn = j;
            }
        }
        for(int j = 0; j <= m; j++) {
            if(used[j]) {
                rowPotential[matchColumn[j]] += delta;
                columnPotential[j] -= delta;
            } else {
                minValue[j] -= delta;
            }
        }
        column = nextColumn;
    } while(matchColumn[column] != 0);
    do {
        int nextColumn = previous[column];
        matchColumn[column] = matchColumn[nextColumn];
        column = nextColumn;
    } while(column != 0);
}
vector<int> matchRow(n + 1);
for(int column = 1; column <= m; column++) {
    if(matchColumn[column]) {
        matchRow[matchColumn[column]] = column;
    }
}
i64 value = 0;
for(int row = 1; row <= n; row++) {
    value += weight[row][matchRow[row]];
}
return {value, matchRow};
}
};

```

最大流 (Dinic)

点 0-based; `minCut(s)` 在最大流后返回残量图中源点侧。

```
#include <bits/stdc++.h>
```

```

using namespace std;

using i64 = long long;

struct Dinic {
    struct Edge {
        int to, reverse;
        i64 capacity;
    };

    int n;
    vector<vector<Edge>> adj;
    vector<int> level, iter;

    Dinic(int n_ = 0) : n(n_), adj(n_), level(n_), iter(n_) {}
    int addEdge(int u, int v, i64 capacity) {
        int id = int(adj[u].size());
        int reverse = int(adj[v].size());
        adj[u].push_back({v, reverse, capacity});
        adj[v].push_back({u, id, 0});
        return id;
    }
    i64 maxFlow(int source, int sink,
                i64 limit = numeric_limits<i64>::max() / 4) {
        i64 flow = 0;
        while(flow < limit && bfs(source, sink)) {
            fill(iter.begin(), iter.end(), 0);
            while(flow < limit) {
                i64 pushed = dfs(source, sink, limit - flow);
                if(!pushed) {
                    break;
                }
                flow += pushed;
            }
        }
        return flow;
    }
    vector<bool> minCut(int source) const {
        vector<bool> visit(n);
        queue<int> q;
        visit[source] = true;
        q.push(source);
        while(!q.empty()) {
            int u = q.front();
            q.pop();
            for(const auto& edge : adj[u]) {
                if(edge.capacity && !visit[edge.to]) {
                    visit[edge.to] = true;
                    q.push(edge.to);
                }
            }
        }
        return visit;
    }
};

```

```

    }

private:
    bool bfs(int source, int sink) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[source] = 0;
        q.push(source);
        while(!q.empty()) {
            int u = q.front();
            q.pop();
            for(const auto& edge : adj[u]) {
                if(edge.capacity && level[edge.to] == -1) {
                    level[edge.to] = level[u] + 1;
                    q.push(edge.to);
                }
            }
        }
        return level[sink] != -1;
    }
    i64 dfs(int u, int sink, i64 limit) {
        if(u == sink) {
            return limit;
        }
        for(int& i = iter[u]; i < int(adj[u].size()); i++) {
            Edge& edge = adj[u][i];
            if(!edge.capacity || level[edge.to] != level[u] + 1) {
                continue;
            }
            i64 pushed = dfs(edge.to, sink, min(limit, edge.capacity));
            if(!pushed) {
                continue;
            }
            edge.capacity -= pushed;
            adj[edge.to][edge.reverse].capacity += pushed;
            return pushed;
        }
        return 0;
    }
};

```

最小费用流

点 0-based; 支持负费用边; 初始残量图中不能有 source 可达负环。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct MinCostFlow {
    struct Edge {

```

```

    int to, reverse;
    i64 capacity, cost;
};

static constexpr i64 inf = numeric_limits<i64>::max() / 4;
int n;
vector<vector<Edge>> adj;

MinCostFlow(int n_ = 0) : n(n_), adj(n_) {}
void addEdge(int u, int v, i64 capacity, i64 cost) {
    int uIndex = int(adj[u].size());
    int vIndex = int(adj[v].size());
    adj[u].push_back({v, vIndex, capacity, cost});
    adj[v].push_back({u, uIndex, 0, -cost});
}

pair<i64, i64> flow(int source, int sink, i64 limit = inf) {
    vector<i64> potential = initialPotential(source);
    vector<i64> distance(n);
    vector<int> previousVertex(n), previousEdge(n);
    i64 totalFlow = 0;
    i64 totalCost = 0;
    while(totalFlow < limit) {
        fill(distance.begin(), distance.end(), inf);
        priority_queue<pair<i64, int>, vector<pair<i64, int>>, greater<>> q;
        distance[source] = 0;
        q.push({0, source});
        while(!q.empty()) {
            auto [currentDistance, u] = q.top();
            q.pop();
            if(currentDistance != distance[u]) {
                continue;
            }
            for(int i = 0; i < int(adj[u].size()); i++) {
                const Edge& edge = adj[u][i];
                if(!edge.capacity) {
                    continue;
                }
                i64 nextDistance = currentDistance + edge.cost
                    + potential[u] - potential[edge.to];
                if(nextDistance < distance[edge.to]) {
                    distance[edge.to] = nextDistance;
                    previousVertex[edge.to] = u;
                    previousEdge[edge.to] = i;
                    q.push({nextDistance, edge.to});
                }
            }
        }
        if(distance[sink] == inf) {
            break;
        }
        for(int u = 0; u < n; u++) {
            if(distance[u] != inf) {
                potential[u] += distance[u];
            }
        }
    }
}

```

```

    }
}
i64 pushed = limit - totalFlow;
for(int v = sink; v != source; v = previousVertex[v]) {
    const Edge& edge = adj[previousVertex[v]][previousEdge[v]];
    pushed = min(pushed, edge.capacity);
}
for(int v = sink; v != source; v = previousVertex[v]) {
    Edge& edge = adj[previousVertex[v]][previousEdge[v]];
    totalCost += pushed * edge.cost;
    edge.capacity -= pushed;
    adj[v][edge.reverse].capacity += pushed;
}
totalFlow += pushed;
}
return {totalFlow, totalCost};
}

private:
vector<i64> initialPotential(int source) const {
    vector<i64> distance(n, inf);
    distance[source] = 0;
    for(int round = 1; round < n; round++) {
        bool changed = false;
        for(int u = 0; u < n; u++) {
            if(distance[u] == inf) {
                continue;
            }
            for(const Edge& edge : adj[u]) {
                if(edge.capacity && distance[edge.to] > distance[u] + edge.cost) {
                    distance[edge.to] = distance[u] + edge.cost;
                    changed = true;
                }
            }
        }
        if(!changed) {
            break;
        }
    }
    for(i64& value : distance) {
        if(value == inf) {
            value = 0;
        }
    }
    return distance;
}
};

```

有上下界网络流

点、原边编号均 0-based; Dinic 已内置; `feasibleCirculation()` 求可行环流; `maxFlow(s,t)` 求最大流及原边流量。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct Dinic {
    struct Edge {
        int to, reverse;
        i64 capacity;
    };

    int n;
    vector<vector<Edge>> adj;
    vector<int> level, iter;

    Dinic(int n_ = 0) : n(n_), adj(n_), level(n_), iter(n_) {}
    int addEdge(int u, int v, i64 capacity) {
        int id = int(adj[u].size());
        int reverse = int(adj[v].size());
        adj[u].push_back({v, reverse, capacity});
        adj[v].push_back({u, id, 0});
        return id;
    }
    i64 maxFlow(int source, int sink,
                i64 limit = numeric_limits<i64>::max() / 4) {
        i64 flow = 0;
        while(flow < limit && bfs(source, sink)) {
            fill(iter.begin(), iter.end(), 0);
            while(flow < limit) {
                i64 pushed = dfs(source, sink, limit - flow);
                if(!pushed) {
                    break;
                }
                flow += pushed;
            }
        }
        return flow;
    }
    vector<bool> minCut(int source) const {
        vector<bool> visit(n);
        queue<int> q;
        visit[source] = true;
        q.push(source);
        while(!q.empty()) {
            int u = q.front();
            q.pop();
            for(const auto& edge : adj[u]) {
                if(edge.capacity && !visit[edge.to]) {
                    visit[edge.to] = true;
                    q.push(edge.to);
                }
            }
        }
    }
};

```

```

        return visit;
    }

private:
    bool bfs(int source, int sink) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[source] = 0;
        q.push(source);
        while(!q.empty()) {
            int u = q.front();
            q.pop();
            for(const auto& edge : adj[u]) {
                if(edge.capacity && level[edge.to] == -1) {
                    level[edge.to] = level[u] + 1;
                    q.push(edge.to);
                }
            }
        }
        return level[sink] != -1;
    }

    i64 dfs(int u, int sink, i64 limit) {
        if(u == sink) {
            return limit;
        }
        for(int& i = iter[u]; i < int(adj[u].size()); i++) {
            Edge& edge = adj[u][i];
            if(!edge.capacity || level[edge.to] != level[u] + 1) {
                continue;
            }
            i64 pushed = dfs(edge.to, sink, min(limit, edge.capacity));
            if(!pushed) {
                continue;
            }
            edge.capacity -= pushed;
            adj[edge.to][edge.reverse].capacity += pushed;
            return pushed;
        }
        return 0;
    }
};

struct LowerBoundFlow {
    struct Edge {
        int from, to;
        i64 lower, upper;
    };
    struct MaxFlowResult {
        i64 value;
        vector<i64> edgeFlow;
    };
};

static constexpr i64 inf = numeric_limits<i64>::max() / 4;

```

```

int n;
vector<Edge> edges;

LowerBoundFlow(int n_ = 0) : n(n_) {}
int addEdge(int from, int to, i64 lower, i64 upper) {
    int id = int(edges.size());
    edges.push_back({from, to, lower, upper});
    return id;
}
optional<vector<i64>> feasibleCirculation() const {
    auto network = build(false, -1, -1);
    if(network.dinic.maxFlow(network.superSource, network.supersink) != network.demand)
{
        return nullopt;
    }
    return recover(network);
}
optional<MaxFlowResult> maxFlow(int source, int sink) const {
    auto network = build(true, source, sink);
    if(network.dinic.maxFlow(network.superSource, network.supersink) != network.demand)
{
        return nullopt;
    }
    auto& artificial = network.dinic.adj[sink][network.artificialEdge];
    i64 baseFlow = inf - artificial.capacity;
    network.dinic.adj[artificial.to][artificial.reverse].capacity = 0;
    artificial.capacity = 0;
    disableSupervertices(network);
    i64 extra = network.dinic.maxFlow(source, sink);
    return MaxFlowResult{baseFlow + extra, recover(network)};
}

private:
struct Network {
    Dinic dinic;
    int superSource, supersink;
    i64 demand = 0;
    vector<pair<int, int>> position;
    int artificialEdge = -1;
};

Network build(bool addArtificial, int source, int sink) const {
    Network network;
    network.dinic = Dinic(n + 2);
    network.superSource = n;
    network.supersink = n + 1;
    vector<i64> balance(n);
    for(int i = 0; i < int(edges.size()); i++) {
        const Edge& edge = edges[i];
        int index = network.dinic.addEdge(edge.from, edge.to, edge.upper - edge.lower);
        network.position.push_back({edge.from, index});
        balance[edge.from] -= edge.lower;
        balance[edge.to] += edge.lower;
    }
}

```

```

    }
    if(addArtificial) {
        network.artificialEdge = network.dinic.addEdge(sink, source, inf);
    }
    for(int u = 0; u < n; u++) {
        if(balance[u] > 0) {
            network.dinic.addEdge(network.superSource, u, balance[u]);
            network.demand += balance[u];
        } else if(balance[u] < 0) {
            network.dinic.addEdge(u, network.superSink, -balance[u]);
        }
    }
    return network;
}

vector<i64> recover(const Network& network) const {
    vector<i64> result(edges.size());
    for(int i = 0; i < int(edges.size()); i++) {
        auto [u, index] = network.position[i];
        const auto& edge = network.dinic.adj[u][index];
        result[i] = edges[i].upper - edge.capacity;
    }
    return result;
}

static void disablesSuperVertices(Network& network) {
    for(int u = 0; u < network.dinic.n; u++) {
        for(auto& edge : network.dinic.adj[u]) {
            if(u == network.superSource || u == network.superSink
                || edge.to == network.superSource || edge.to == network.superSink) {
                edge.capacity = 0;
            }
        }
    }
}

};

```

差分约束

统一成 $x[v] - x[u] \leq w$ ，建边 $u \rightarrow v (w)$ ；超级源连所有点。无负环时最短路势能是一组解，存在负环即无解。 $\geq$ 先移项取反，等式拆两边。

函数图

每个连通块 = 一个有向环 + 指向环的内向树。入度剥叶找环，反图标记入树；跳 k 步用倍增，环上距离按方向取模。

优化建图

用辅助结点复用批量边。点到区间连入向线段树，区间到点连出向线段树，树内边权为 0；偏序只需保可达性时只连 cover edge。检查方向、代价只计一次、扩展图规模。

增量容量最大流

只增容不删边：`addEdge` 返回正向边下标，`total += addCap(s,t,u,edgeId,delta)` 在残量网络上继续增广。减容/删边需重建。

```
int addEdge(int u, int v, int cap) {
    g[u].push_back({v, int(g[v].size()), cap});
    g[v].push_back({u, int(g[u].size()) - 1, 0});
    return int(g[u].size()) - 1;
}

int addCap(int s, int t, int u, int edgeId, int delta) {
    g[u][edgeId].cap += delta;
    int flow = 0;
    while(bfs(s, t)) {
        fill(it.begin(), it.end(), 0);
        while(int f = dfs(s, t, 1 << 30)) {
            flow += f;
        }
    }
    return flow;
}
```

严格次小生成树

Kruskal 建 MST 后倍增维护路径最大/严格次大边；每条非树边 (u,v,w) ： $w > M1$ 时 $cand = mst + w - M1$ ， $w == M1$ 且路径有第二条边时 $cand = mst + w - M2$ 。 $M2$ 无值用 $-INF$ 哨兵。

阈值激活事件堆

顶点需求 $a[u]$ 次到达才激活；小根堆处理最早事件， $--a[u]$ 到 0 才松弛出边。定时源用虚点批量清零依赖点需求。

```
while(!q.empty()) {
    auto [u, t] = q.top();
    q.pop();
    if(ans[u] != -1) {
        continue;
    }
    if(--a[u] > 0) {
        continue;
    }
    ans[u] = t;
    for(auto [v, w] : adj[u]) {
        if(ans[v] == -1) {
            q.push({v, t + w});
        }
    }
}
```

异或最小生成树

按最高异或位分治：组内递归建 MST，跨组最小异或对用 Trie（查询包含当前分位，跨组最高位恒为 1）。

```
long long solve(vector<int>& nums, int bit) {
    if(bit < 0 || int(nums.size()) <= 1) {
        return 0;
    }
    vector<int> l, r;
    for(int x : nums) {
        (x >> bit & 1 ? r : l).push_back(x);
    }
    long long res = solve(l, bit - 1) + solve(r, bit - 1);
    if(l.empty() || r.empty()) {
        return res;
    }
    trie.clear();
    trie.emplace_back();
    for(int x : r) {
        insert(x, bit);
    }
    int best = INT_MAX;
    for(int x : l) {
        best = min(best, minXor(x, bit));
    }
    return res + best;
}
```

最大权闭合子图

正权连 $S \rightarrow u$ 、负权连 $u \rightarrow T$ （容量为权值绝对值）、依赖 $u \rightarrow v$ 连 INF （大于正权和）；答案 = 正权和 - 最小割。

```
i64 INF = sumPos + 1;
for(int u = 1; u <= n; u++) {
    if(w[u] > 0) {
        din.addEdge(S, u, int(w[u]));
    } else if(w[u] < 0) {
        din.addEdge(u, T, int(-w[u]));
    }
    for(int v : dep[u]) {
        din.addEdge(u, v, int(INF));
    }
}
return sumPos - din.maxFlow(S, T);
```

隐式完全图 Boruvka

每轮对每个连通块用 oracle 求最小跨块边并合并，轮数 $O(\log n)$ ；oracle 是复杂度关键（排序双指针/Trie）。骨架即“DSU + 每块最小出边 + 统一合并”。

```

while(dsu.cnt > 1) {
    // 对每个块求最小跨块边 (u, v, w) (oracle)
    // 排序后依次 dsu.unite(u, v) 累加 w
}

```

指定拓扑序的稀疏约束

排列 a 用递减栈连“前一个更大”的边、排列 b 用递增栈连“前一个更小”的边；合并后大根堆 Kahn 应输出 b、小根堆 Kahn 应输出 a，否则无解。

```

stack<int> stk;
for(int x : a) {
    while(!stk.empty() && x > stk.top()) {
        stk.pop();
    }
    if(!stk.empty()) {
        adj[stk.top()].push_back(x);
    }
    stk.push(x);
}

```

3. 树上算法

树的直径

非负边权树；空树返回 {0,0,0}。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct TreeDiameter {
    struct Edge {
        int to;
        i64 w;
    };
    struct Result {
        int a, b;
        i64 distance;
    };
    static Result run(const vector<vector<Edge>>& adj) {
        if(adj.size() <= 1) {
            return {0, 0, 0};
        }
        auto farthest = [&](int s) {
            pair<i64, int> best{0, s};
            auto dfs = [&](auto&& self, int u, int p, i64 d) -> void {
                best = max(best, pair{d, u});
                for(auto [v, w] : adj[u]) {

```

```

        if(v != p) {
            self(self, v, u, d + w);
        }
    }
};
dfs(dfs, s, 0, 0);
return best;
};
int a = farthest(1).second;
auto [distance, b] = farthest(a);
return {a, b, distance};
}
};

```

树的重心

返回一个或两个重心。

```

#include <bits/stdc++.h>
using namespace std;

vector<int> treeCentroids(const vector<vector<int>>& adj) {
    int n = int(adj.size()) - 1;
    vector<int> size(adj.size()), ans;
    auto dfs = [&](auto&& self, int u, int p) -> void {
        size[u] = 1;
        int mx = 0;
        for(int v : adj[u]) {
            if(v != p) {
                self(self, v, u);
                size[u] += size[v];
                mx = max(mx, size[v]);
            }
        }
        mx = max(mx, n - size[u]);
        if(mx * 2 <= n) {
            ans.push_back(u);
        }
    };
    if(n > 0) {
        dfs(dfs, 1, 0);
    }
    return ans;
}

```

最小生成树重构树 (Kruskal)

原点 1-based; 新点为 `[n+1, nodeCount]`; `bottleneck(u,v)` 不连通返回 `nullopt`。

```

#include <bits/stdc++.h>
using namespace std;

```

```

using i64 = long long;

struct KruskalReconstructionTree {
    struct Edge {
        int u, v;
        i64 w;
    };
    int n, nodeCount, log;
    vector<i64> weight;
    vector<array<int, 2>> child;
    vector<int> depth, component;
    vector<vector<int>> up;

    KruskalReconstructionTree(int n_, vector<Edge> edges) : n(n_) {
        int cap = max(2, 2 * n + 1);
        vector<int> dsu(cap), root(cap);
        iota(dsu.begin(), dsu.end(), 0);
        iota(root.begin(), root.end(), 0);
        weight.assign(cap, 0);
        child.assign(cap, {0, 0});
        auto find = [&](auto&& self, int x) -> int {
            return dsu[x] == x ? x : dsu[x] = self(self, dsu[x]);
        };
        sort(edges.begin(), edges.end(), [](const auto& a, const auto& b) {
            return a.w < b.w;
        });
        nodeCount = n;
        for(auto [u, v, w] : edges) {
            int x = find(find, u);
            int y = find(find, v);
            if(x == y) {
                continue;
            }
            int z = ++nodeCount;
            weight[z] = w;
            child[z] = {root[x], root[y]};
            dsu[x] = dsu[y] = dsu[z] = z;
            root[z] = z;
        }
        weight.resize(nodeCount + 1);
        child.resize(nodeCount + 1);
        log = bit_width(static_cast<unsigned>(max(1, nodeCount)));
        depth.assign(nodeCount + 1, 0);
        component.assign(nodeCount + 1, 0);
        up.assign(log, vector<int>(nodeCount + 1, 0));
        for(int i = 1; i <= n; i++) {
            int r = find(find, i);
            if(component[r] != 0) {
                continue;
            }
            component[r] = r;
            auto dfs = [&](auto&& self, int u, int p) -> void {

```

```

        component[u] = r;
        up[0][u] = p;
        for(int j = 1; j < log; j++) {
            up[j][u] = up[j - 1][up[j - 1][u]];
        }
        for(int v : child[u]) {
            if(v != 0) {
                depth[v] = depth[u] + 1;
                self(self, v, u);
            }
        }
    };
    dfs(dfs, r, 0);
}

int lca(int u, int v) const {
    if(depth[u] < depth[v]) {
        swap(u, v);
    }
    int d = depth[u] - depth[v];
    for(int j = 0; j < log; j++) {
        if(d >> j & 1) {
            u = up[j][u];
        }
    }
    if(u == v) {
        return u;
    }
    for(int j = log - 1; j >= 0; j--) {
        if(up[j][u] != up[j][v]) {
            u = up[j][u];
            v = up[j][v];
        }
    }
    return up[0][u];
}

optional<i64> bottleneck(int u, int v) const {
    if(component[u] != component[v]) {
        return nullopt;
    }
    if(u == v) {
        return 0;
    }
    return weight[lca(u, v)];
}
};

```

最近公共祖先 (LCA)

1-based; `adj[0]` 不使用; `jump(u,k)` 超过根返回 0。

```
#include <bits/stdc++.h>
```

```

using namespace std;

struct LCA {
    int n, log, timer = 0;
    vector<int> depth, tin, tout, parent;
    vector<vector<int>> up;

    LCA(const vector<vector<int>>& adj, int root = 1)
        : n(int(adj.size()) - 1), log(bit_width(static_cast<unsigned>(max(1, n)))),
          depth(adj.size()), tin(adj.size()), tout(adj.size()), parent(adj.size()),
          up(log, vector<int>(adj.size())) {
        auto dfs = [&](auto&& self, int u, int p) -> void {
            parent[u] = up[0][u] = p;
            tin[u] = timer++;
            for(int j = 1; j < log; j++) {
                up[j][u] = up[j - 1][up[j - 1][u]];
            }
            for(int v : adj[u]) {
                if(v != p) {
                    depth[v] = depth[u] + 1;
                    self(self, v, u);
                }
            }
            tout[u] = timer;
        };
        if(n > 0) {
            dfs(dfs, root, 0);
        }
    }

    bool isAncestor(int u, int v) const {
        return tin[u] <= tin[v] && tout[v] <= tout[u];
    }

    int lca(int u, int v) const {
        if(isAncestor(u, v)) {
            return u;
        }
        if(isAncestor(v, u)) {
            return v;
        }
        for(int j = log - 1; j >= 0; j--) {
            if(up[j][u] != 0 && !isAncestor(up[j][u], v)) {
                u = up[j][u];
            }
        }
        return parent[u];
    }

    int jump(int u, int k) const {
        for(int j = 0; k && u != 0; j++, k >>= 1) {
            if(j == log) {
                return 0;
            }
            if(k & 1) {
                u = up[j][u];
            }
        }
    }
};

```

```

    }
}
return u;
}
int distance(int u, int v) const {
    int p = lca(u, v);
    return depth[u] + depth[v] - 2 * depth[p];
}
};

```

重链剖分 (HLD)

1-based; 构造传 `vertexCount+1`, 0 为父亲哨兵; 子树 DFS 序为 `[in[u],out[u]]`; 只调用一次 `work()`。

```

#include <bits/stdc++.h>
using namespace std;

struct HLD {
    int n, cur;
    vector<int> siz, top, dep, parent, in, out, seq;
    vector<vector<int>> adj;

    HLD() {}
    HLD(int n) {
        init(n);
    }
    void init(int n) {
        this->n = n;
        siz.assign(n, 0);
        top.assign(n, 0);
        dep.assign(n, 0);
        parent.assign(n, 0);
        in.assign(n, 0);
        out.assign(n, 0);
        seq.assign(n, 0);
        adj.assign(n, {});
        cur = 1;
    }
    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void work(int root = 1) {
        top[root] = root;
        dep[root] = 0;
        parent[root] = 0;
        dfs1(root);
        dfs2(root);
    }
    void dfs1(int u) {
        if(parent[u]) {
            auto it = find(adj[u].begin(), adj[u].end(), parent[u]);

```

```

        adj[u].erase(it);
    }
    siz[u] = 1;
    for(auto& v : adj[u]) {
        parent[v] = u;
        dep[v] = dep[u] + 1;
        dfs1(v);
        siz[u] += siz[v];
        if(siz[v] > siz[adj[u][0]]) {
            swap(v, adj[u][0]);
        }
    }
}

void dfs2(int u) {
    in[u] = cur++;
    seq[in[u]] = u;
    for(int v : adj[u]) {
        top[v] = v == adj[u][0] ? top[u] : v;
        dfs2(v);
    }
    out[u] = cur;
}

int lca(int u, int v) const {
    while(top[u] != top[v]) {
        if(dep[top[u]] > dep[top[v]]) {
            u = parent[top[u]];
        } else {
            v = parent[top[v]];
        }
    }
    return dep[u] < dep[v] ? u : v;
}

int dist(int u, int v) const {
    return dep[u] + dep[v] - 2 * dep[lca(u, v)];
}

int jump(int u, int k) const {
    if(dep[u] < k) {
        return 0;
    }
    int d = dep[u] - k;
    while(dep[top[u]] > d) {
        u = parent[top[u]];
    }
    return seq[in[u] - dep[u] + d];
}

bool isAncestor(int u, int v) const {
    return in[u] <= in[v] && in[v] < out[u];
}

int rootedParent(int root, int u) const {
    if(root == u) {
        return u;
    }
    if(!isAncestor(u, root)) {

```

```

        return parent[u];
    }
    auto it = upper_bound(adj[u].begin(), adj[u].end(), root,
        [&](int x, int y) {
            return in[x] < in[y];
        });
    return *prev(it);
}
int rootedSize(int root, int u) const {
    if(root == u) {
        return n - 1;
    }
    if(!isAncestor(u, root)) {
        return siz[u];
    }
    return n - 1 - siz[rootedParent(root, u)];
}
int rootedLca(int a, int b, int root) const {
    return lca(a, b) ^ lca(b, root) ^ lca(root, a);
}
};

```

树上差分

1-based; `adj[0]`、`diff[0]` 不使用；点差分与边差分最后按后序向父亲累加；边答案记在较深端点。

```

#include <bits/stdc++.h>
using namespace std;

struct TreeDifference {
    int n, log, timer = 0;
    vector<int> depth, tin, tout, parent;
    vector<vector<int>> up;

    TreeDifference(const vector<vector<int>>& adj, int root = 1)
        : n(int(adj.size()) - 1), log(bit_width(static_cast<unsigned>(max(1, n)))),
          depth(adj.size()), tin(adj.size()), tout(adj.size()), parent(adj.size()),
          up(log, vector<int>(adj.size())) {
        auto dfs = [&](auto&& self, int u, int p) -> void {
            parent[u] = up[0][u] = p;
            tin[u] = timer++;
            for(int j = 1; j < log; j++) {
                up[j][u] = up[j - 1][up[j - 1][u]];
            }
            for(int v : adj[u]) {
                if(v != p) {
                    depth[v] = depth[u] + 1;
                    self(self, v, u);
                }
            }
            tout[u] = timer;
        };
    };

```

```

        if(n > 0) {
            dfs(dfs, root, 0);
        }
    }
    bool isAncestor(int u, int v) const {
        return tin[u] <= tin[v] && tout[v] <= tout[u];
    }
    int lca(int u, int v) const {
        if(isAncestor(u, v)) {
            return u;
        }
        if(isAncestor(v, u)) {
            return v;
        }
        for(int j = log - 1; j >= 0; j--) {
            if(up[j][u] != 0 && !isAncestor(up[j][u], v)) {
                u = up[j][u];
            }
        }
        return parent[u];
    }
    template<class T>
    void addVertexPath(vector<T>& diff, int u, int v, T w) {
        int p = lca(u, v);
        diff[u] += w;
        diff[v] += w;
        diff[p] -= w;
        if(parent[p] != 0) {
            diff[parent[p]] -= w;
        }
    }
    template<class T>
    void addEdgePath(vector<T>& diff, int u, int v, T w) {
        int p = lca(u, v);
        diff[u] += w;
        diff[v] += w;
        diff[p] -= w + w;
    }
    template<class T>
    static void accumulate(vector<T>& diff, const vector<int>& fa,
                           const vector<int>& order) {
        for(int i = int(order.size()) - 1; i > 0; i--) {
            diff[fa[order[i]]] += diff[order[i]];
        }
    }
};

```

虚树

1-based; `adj[0]` 不使用; `build(关键点)` 返回虚树根与 `parent->child` 边, 空集根为 0; LCA 已内置。

```
#include <bits/stdc++.h>
```

```

using namespace std;

struct VirtualTree {
    int n, log, timer = 0;
    vector<int> tin, tout;
    vector<vector<int>> up;

    VirtualTree(const vector<vector<int>>& adj, int root = 1)
        : n(int(adj.size()) - 1), log(bit_width(static_cast<unsigned>(max(1, n)))),
          tin(adj.size()), tout(adj.size()), up(log, vector<int>(adj.size())) {
        auto dfs = [&](auto&& self, int u, int p) -> void {
            up[0][u] = p;
            tin[u] = timer++;
            for(int j = 1; j < log; j++) {
                up[j][u] = up[j - 1][up[j - 1][u]];
            }
            for(int v : adj[u]) {
                if(v != p) {
                    self(self, v, u);
                }
            }
            tout[u] = timer;
        };
        if(n > 0) {
            dfs(dfs, root, 0);
        }
    }

    bool isAncestor(int u, int v) const {
        return tin[u] <= tin[v] && tout[v] <= tout[u];
    }

    int lca(int u, int v) const {
        if(isAncestor(u, v)) {
            return u;
        }
        if(isAncestor(v, u)) {
            return v;
        }
        for(int j = log - 1; j >= 0; j--) {
            if(up[j][u] != 0 && !isAncestor(up[j][u], v)) {
                u = up[j][u];
            }
        }
        return up[0][u];
    }

    pair<int, vector<pair<int, int>>> build(vector<int> nodes) const {
        if(nodes.empty()) {
            return {0, {}};
        }
        auto cmp = [&](int u, int v) {
            return tin[u] < tin[v];
        };
        sort(nodes.begin(), nodes.end(), cmp);
        int k = int(nodes.size());

```

```

        for(int i = 1; i < k; i++) {
            nodes.push_back(lca(nodes[i - 1], nodes[i]));
        }
        sort(nodes.begin(), nodes.end(), cmp);
        nodes.erase(unique(nodes.begin(), nodes.end()), nodes.end());
        vector<pair<int, int>> edges;
        vector<int> st;
        for(int u : nodes) {
            while(!st.empty() && lca(st.back(), u) != st.back()) {
                st.pop_back();
            }
            if(!st.empty()) {
                edges.push_back({st.back(), u});
            }
            st.push_back(u);
        }
        return {nodes.front(), edges};
    }
};

```

倍增

`value[x]` 是从 `x` 出发一条转移的权; `jump(x,k)` 聚合前 `k` 条边, 不含落点。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

template<class T, class Op>
struct Doubling {
    int log;
    T identity;
    Op op;
    vector<vector<int>> next;
    vector<vector<T>> value;

    Doubling(const vector<int>& to, const vector<T>& weight, i64 maxStep,
              T identity_, Op op_ = Op())
        : log(1), identity(identity_), op(op_) {
        for(i64 x = maxStep; x > 1; x >>= 1) {
            log++;
        }
        next.assign(log, vector<int>(to.size()));
        value.assign(log, vector<T>(to.size()));
        next[0] = to;
        value[0] = weight;
        for(int j = 1; j < log; j++) {
            for(int x = 1; x < int(to.size()); x++) {
                next[j][x] = next[j - 1][next[j - 1][x]];
                value[j][x] = op(value[j - 1][x], value[j - 1][next[j - 1][x]]);
            }
        }
    }
};

```

```

    }
}
pair<int, T> jump(int x, i64 k) const {
    T res = identity;
    for(int j = 0; j < log; j++) {
        if(k >> j & 1) {
            res = op(res, value[j][x]);
            x = next[j][x];
        }
    }
    return {x, res};
}
};

```

树上启发式合并 (DSU on Tree)

`add(u,+1/-1)` 加入/移除点; `answer(u)` 调用时恰好保留 `u` 的整棵子树。

```

#include <bits/stdc++.h>
using namespace std;

struct DSUonTree {
    const vector<vector<int>>& adj;
    vector<int> parent, size, heavy, tin, tout, euler;
    int root, timer = 0;

    DSUonTree(const vector<vector<int>>& graph, int root_ = 1)
        : adj(graph), parent(adj.size()), size(adj.size()), heavy(adj.size()),
          tin(adj.size()), tout(adj.size()), euler(adj.size()), root(root_), timer(1) {
        if(adj.size() > 1) {
            dfsSize(root, 0);
        }
    }

    void dfsSize(int u, int p) {
        parent[u] = p;
        size[u] = 1;
        tin[u] = timer;
        euler[timer++] = u;
        for(int v : adj[u]) {
            if(v != p) {
                dfsSize(v, u);
                size[u] += size[v];
                if(heavy[u] == 0 || size[v] > size[heavy[u]]) {
                    heavy[u] = v;
                }
            }
        }
        tout[u] = timer - 1;
    }

    template<class Add, class Answer>
    void run(Add add, Answer answer) {
        auto dfs = [&](auto&& self, int u, bool keep) -> void {

```

```

        for(int v : adj[u]) {
            if(v != parent[u] && v != heavy[u]) {
                self(self, v, false);
            }
        }
        if(heavy[u] != 0) {
            self(self, heavy[u], true);
        }
        for(int v : adj[u]) {
            if(v == parent[u] || v == heavy[u]) {
                continue;
            }
            for(int i = tin[v]; i <= tout[v]; i++) {
                add(euler[i], 1);
            }
        }
        add(u, 1);
        answer(u);
        if(!keep) {
            for(int i = tin[u]; i <= tout[u]; i++) {
                add(euler[i], -1);
            }
        }
    };
    if(adj.size() > 1) {
        dfs(dfs, root, false);
    }
}
};

```

点分治

`parent[c]` 为点分树父亲，分治根父亲为 0。

```

#include <bits/stdc++.h>
using namespace std;

struct CentroidDecomposition {
    const vector<vector<int>>& adj;
    vector<int> parent, level, size;
    vector<bool> blocked;
    CentroidDecomposition(const vector<vector<int>>& graph)
        : adj(graph), parent(adj.size()), level(adj.size()), size(adj.size()),
        blocked(adj.size()) {
        if(adj.size() > 1) {
            build(1, 0, 0);
        }
    }
    int getSize(int u, int p) {
        size[u] = 1;
        for(int v : adj[u]) {
            if(v != p && !blocked[v]) {

```

```

        size[u] += getSize(v, u);
    }
}
return size[u];
}
int getCentroid(int u, int p, int n) {
    for(int v : adj[u]) {
        if(v != p && !blocked[v] && size[v] * 2 > n) {
            return getCentroid(v, u, n);
        }
    }
    return u;
}
void build(int entry, int p, int dep) {
    int c = getCentroid(entry, 0, getSize(entry, 0));
    parent[c] = p;
    level[c] = dep;
    blocked[c] = true;
    for(int v : adj[c]) {
        if(!blocked[v]) {
            build(v, c, dep + 1);
        }
    }
}
};

```

换根 DP

后序求 `down`；再把父侧贡献作为 `up` 传给儿子。传给 `v` 时排除 `v` 自身贡献：一般用前后缀；`min/max` 可记最优、次优及来源，`O(1)` 排除。整体 `O(n)`。

启发式合并 (Small-to-Large)

始终把小集合并入大集合；每次搬运后所在集合至少翻倍，单元素最多搬 `$O(\log n)$` 次。插入 `$O(\log n)$` 时总计通常 `$O(n \log^2 n)$` 。

图缩点合并邻接集合时同样小并大；同步替换邻居端代表元，并删自环、重边。

长链剖分优化深度 DP

选最高儿子为长儿子，父节点错位继承其深度数组，短儿子逐层并入；每点只作为短链合并一次，总 `$O(n)$` 。比赛用**池式共享数组**：链头分配整段，长链节点错位共享（`poolStart[heavy] = poolStart[u] + 1`）。每点答案必须是标量（共享槽会被父节点合并覆盖）。

```

if(poolStart[u] == -1) {
    poolStart[u] = int(pool.size());
    pool.resize(pool.size() + height[u], 0);
}
pool[poolStart[u]] = 1;
if(heavy[u]) {
    poolStart[heavy[u]] = poolStart[u] + 1; // 错位共享
    dfsDP(heavy[u], u);
}
// 短链并入: pool[poolStart[u] + 1 + d] += pool[poolStart[v] + d]

```

树上仿射约束传播

边约束 $x_u + x_v = w$: DFS 维护 $(coef, bias)$ 使 $x_u = coef \cdot x_{root} + bias$, $coef$ 每条边取反、 $bias$ 变 $w - bias$; 每个点的值域转成根值区间, 交集非空即可行, 取 lo 回代。

```

if(c == 1) lo = max(lo, 1 - b); // x_u >= 1
else      hi = min(hi, b - 1);
dfs(v, u, w - b, -c);

```

4. 动态规划

最长上升子序列 (LIS)

`strict=true` 最长严格上升 (`lower_bound`); `strict=false` 最长不下降 (`upper_bound`)。最长不上升/下降: 先把所有数取负, 再对负序列求不下降/严格上升。

```

#include <bits/stdc++.h>
using namespace std;

int lisLength(const vector<int>& a, bool strict = true) {
    vector<int> d;
    d.reserve(a.size());
    for(int x : a) {
        auto it = strict ? lower_bound(d.begin(), d.end(), x)
                        : upper_bound(d.begin(), d.end(), x);
        if(it == d.end()) {
            d.push_back(x);
        } else {
            *it = x;
        }
    }
    return int(d.size());
}

```

说明: `lower_bound` (第一个 $\geq x$) 保持 d 严格递增; `upper_bound` (第一个 $> x$) 允许相等, 得到非严格 LIS。最长不上升 = 对 $-x$ 求非严格 LIS; 最长下降 = 对 $-x$ 求严格 LIS。

多重背包（单调队列优化）

物品 1-based (w/v/m 下标 1..n) , 容量 0..C; O(nC)。

```
#include <bits/stdc++.h>
using namespace std;

int multiknapsack(int n, int capacity,
                 const vector<int>& w, const vector<int>& v, const vector<int>& m) {
    vector<vector<int>>> dp(2, vector<int>(capacity + 1, 0));
    for(int i = 1; i <= n; i++) {
        auto& pre = dp[(i - 1) % 2];
        auto& cur = dp[i % 2];
        for(int rem = 0; rem < w[i]; rem++) {
            deque<int> q;
            for(int j = rem; j <= capacity; j += w[i]) {
                while(!q.empty()
                    && pre[j] >= pre[q.back()] + (j - q.back()) / w[i] * v[i]) {
                    q.pop_back();
                }
                q.push_back(j);
                while(!q.empty() && q.front() < max(0, j - m[i] * w[i])) {
                    q.pop_front();
                }
                cur[j] = max(pre[j], pre[q.front()] + (j - q.front()) / w[i] * v[i]);
            }
        }
    }
    return dp[n % 2][capacity];
}
```

数位 DP

state 与合法性判断按题目改；只有 !lead && !lim 才记忆化；答案 f(r) - f(l-1)。

```
using i64 = long long;

i64 dfs(int pos, int state, bool lead, bool lim,
        const vector<int>& num, vector<vector<i64>>& memo) {
    if(pos == 0) return 满足条件(state) ? 1 : 0;
    if(!lead && !lim && memo[pos][state] != -1) return memo[pos][state];
    i64 ans = 0;
    int up = lim ? num[pos] : 9;
    for(int i = 0; i <= up; i++) {
        bool nextLead = lead && i == 0;
        int nextState = nextLead ? state : 转移(state, i);
        ans += dfs(pos - 1, nextState, nextLead, lim && i == up, num, memo);
    }
    if(!lead && !lim) memo[pos][state] = ans;
    return ans;
}
```

轮廓线 DP

逐格扫描，掩码位 $k < j$ 表示本行已处理格、 $k \geq j$ 表示上一行格；不选 = 清位 j 、选 = 置位 j （合法看位 j 与位 $j-1$ ）。换行无需移位。

```
int ns = s & ~(1 << j);           // 不选
ndp[ns] = max(ndp[ns], dp[s]);
bool ok = !(s >> j & 1) && (j == 0 || !(s >> (j - 1) & 1));
if(ok) {
    ndp[ns | (1 << j)] = max(ndp[ns | (1 << j)], dp[s] + a[i][j]);
}
```

中位数阈值 ± 1 变换

判定“中位数 $\geq x$ ”： $\geq x$ 记 +1、否则 -1，存在长度 $\geq k$ 且前缀和非负的窗口即可。上中位数用 ≥ 0 ；下中位数/“第 $(L+1)/2$ 大（上取整）”偶数长度用 ≥ 2 。对 x 二分。

```
mn = 0;
for(int i = k; i <= n; i++) {
    mn = min(mn, pre[i - k]);
    if(pre[i] - mn >= 0) return true;
}
```

稀疏事件线性 DP（矩阵幂）

状态小、步数大、事件点稀疏：普通段 $dp \times A^{len}$ ，事件步用特殊矩阵 B 。段长 = $e - prev - 1$ ，事件步单独算，末尾补 $A^{(S-prev)}$ 。

```
dp = vecMul(dp, powMat(A, e - prev - 1), MOD);
dp = vecMul(dp, B, MOD);
prev = e;
```

排列 LCS $\rightarrow$ LIS

两序列均为 $1..n$ 排列时： $pos[a[i]] = i$ ， $p[i] = pos[b[i]]$ ，对 p 求严格 LIS。

```
vector<int> pos(n + 1);
for(int i = 1; i <= n; i++) pos[a[i]] = i;
vector<int> p(n + 1);
for(int i = 1; i <= n; i++) p[i] = pos[b[i]];
int ans = lisLength(vector<int>(p.begin() + 1, p.end()));
```

背包

01 背包容量倒序，完全背包容量正序；多重背包用二进制拆分或按余数分组的单调队列。不可达状态用 $-INF$ ，转移前先判定可达。

单调队列优化 DP

转移是滑动窗口内的 max/min 时，把与决策有关的项入单调队列；先弹过期队首，再用队首转移。多重背包按容量模 w 分组。

区间 DP

$len \rightarrow l \rightarrow r \rightarrow k$ ；环拆成 $2n$ 链，答案取长度 n 窗口的最值。

```
for(int len = 2; len <= n; len++) {
    for(int l = 1; l + len - 1 <= n; l++) {
        int r = l + len - 1;
        dp[l][r] = INF;
        for(int k = 1; k < r; k++) {
            dp[l][r] = min(dp[l][r],
                           dp[l][k] + dp[k + 1][r] + cost(l, r));
        }
    }
}
```

树上背包

先选子树根再合并：进入孩子 v 前复制父状态加 v 的权，回溯后容量倒序合并。

```
auto dfs = [&](auto&& self, int u, int k) -> void {
    for(int v : adj[u]) {
        for(int i = 0; i < k; i++) dp[v][i] = dp[u][i] + w[v];
        self(self, v, k - 1);
        for(int i = 1; i <= k; i++) dp[u][i] = max(dp[u][i], dp[v][i - 1]);
    }
};
dfs(dfs, 0, m); // 虚拟根 0
```

子集和 DP (SOS DP)

```
for(int i = 0; i < n; i++)
    for(int mask = 0; mask < (1 << n); mask++)
        if(mask >> i & 1) dp[mask] += dp[mask ^ (1 << i)]; // 子集
        // 超集: if(!(mask >> i & 1)) dp[mask] += dp[mask | (1 << i)];
```

必须生成全部 0/1/? 掩码时，逐维将 (v_0, v_1) 展开为 (v_0, v_1, v_0+v_1) ，总量 $O(3^n)$ ；普通子集和仍用 $O(n2^n)$ 。

分治优化 DP

决策单调时：

```
void solve(int l, int r, int optL, int optR, int seg) {
    if(l > r) return;
    int mid = (l + r) / 2, best = optL;
    dp[seg][mid] = INF;
    for(int k = optL; k <= min(mid - 1, optR); k++) {
        i64 val = dp[seg - 1][k] + cost(k + 1, mid);
        if(val < dp[seg][mid]) dp[seg][mid] = val, best = k;
    }
    solve(l, mid - 1, optL, best, seg);
    solve(mid + 1, r, best, optR, seg);
}
```

状压 DP / Gray code

预处理合法状态与转移；枚举子集用 `for(s=mask; s; s=(s-1)&mask)`。Gray code 相邻状态只变化一位，统计量支持单元素增删时为 $O(2^n)$ 。

```
int pre = 0;
for(int s = 1; s < (1 << n); s++) {
    int cur = s ^ (s >> 1);
    int i = __builtin_ctz(cur ^ pre);
    if(cur >> i & 1) add(i);
    else remove(i);
    pre = cur;
}
```

四边形不等式优化 (Knuth)

区间 DP 且 cost 满足四边形不等式时，k 只在 `[opt[i][j-1], opt[i+1][j]]` 枚举。

带权二分 (WQS)

“恰好选 k 个”且 $F(k)$ 凸：二分附加惩罚 lambda，内层 $O(n)$ DP 得 (val, cnt)，答案 `val - lambda * k`；先验证凸性。

位集优化 (Bitset)

可达性：`dp |= dp << w`；Shift-And：`F = ((F<<1)|1) & mask[c]`；遍历置位用 `_Find_first/_Find_next`。

5. 数学

模整数 (MInt)

MOD 需为质数；对 0 取逆为未定义行为。

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
```

```

template <int MOD>
struct MInt {
    int x;

    constexpr MInt(i64 v = 0) : x(norm(v)) {}
    constexpr int val() const { return x; }
    constexpr static int getMod() { return MOD; }

    constexpr int norm(i64 v) const {
        v %= MOD;
        if(v < 0) {
            v += MOD;
        }
        return int(v);
    }

    constexpr MInt operator-() const { return MInt(x ? MOD - x : 0); }

    constexpr MInt& operator+=(MInt rhs) {
        x += rhs.x;
        if(x >= MOD) {
            x -= MOD;
        }
        return *this;
    }

    constexpr MInt& operator-=(MInt rhs) {
        x -= rhs.x;
        if(x < 0) {
            x += MOD;
        }
        return *this;
    }

    constexpr MInt& operator*=(MInt rhs) {
        x = int(1LL * x * rhs.x % MOD);
        return *this;
    }

    constexpr MInt& operator/=(MInt rhs) { return *this *= rhs.inv(); }

    constexpr MInt inv() const {
        assert(x != 0);
        return power(*this, MOD - 2);
    }

    friend constexpr MInt operator+(MInt a, MInt b) { return a += b; }
    friend constexpr MInt operator-(MInt a, MInt b) { return a -= b; }
    friend constexpr MInt operator*(MInt a, MInt b) { return a *= b; }
    friend constexpr MInt operator/(MInt a, MInt b) { return a /= b; }
    friend constexpr bool operator==(MInt a, MInt b) { return a.x == b.x; }
    friend constexpr bool operator!=(MInt a, MInt b) { return a.x != b.x; }
    friend constexpr bool operator<(MInt a, MInt b) { return a.x < b.x; }
};

template <int MOD>

```

```
constexpr MInt<MOD> power(MInt<MOD> a, i64 b) {
    MInt<MOD> res = 1;
    while(b > 0) {
        if(b & 1) {
            res *= a;
        }
        a *= a;
        b >>= 1;
    }
    return res;
}
```

扩展欧几里得 / 中国剩余定理 (CRT)

模数 > 0; `invMod` 要求 `gcd(a,m)=1`; 不相容方程组返回 `nullopt`。

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
using i128 = __int128_t;

i64 exgcd(i64 a, i64 b, i64& x, i64& y) {
    if(b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    i64 g = exgcd(b, a % b, y, x);
    y -= (a / b) * x;
    return g;
}

optional<i64> invMod(i64 a, i64 m) {
    if(m <= 0) {
        return nullopt;
    }
    a %= m;
    if(a < 0) {
        a += m;
    }
    i64 x, y;
    i64 g = exgcd(a, m, x, y);
    if(g != 1) {
        return nullopt;
    }
    x %= m;
    if(x < 0) {
        x += m;
    }
    return x;
}
```

```

// 合并  $x \equiv r_1 \pmod{m_1}$  与  $x \equiv r_2 \pmod{m_2}$ , 返回  $(r, \text{lcm}(m_1, m_2))$ 。
optional<pair<i64, i64>> crtMerge(i64 r1, i64 m1, i64 r2, i64 m2) {
    if(m1 <= 0 || m2 <= 0) {
        return nullopt;
    }
    r1 %= m1;
    if(r1 < 0) {
        r1 += m1;
    }
    r2 %= m2;
    if(r2 < 0) {
        r2 += m2;
    }
    i64 g = std::gcd(m1, m2);
    i128 diff = i128(r2) - r1;
    if(diff % g != 0) {
        return nullopt;
    }
    i128 m1g = m1 / g;
    i128 m2g = m2 / g;
    i128 d = ((diff / g) % m2g + m2g) % m2g;
    auto inv = invMod(i64(m1g % m2g), i64(m2g));
    if(!inv) {
        return nullopt;
    }
    i128 t = d * *inv % m2g;
    i128 lcm = i128(m1) * (m2 / g);
    i128 r = (i128(r1) + i128(m1) * t) % lcm;
    return pair<i64, i64>{i64(r), i64(lcm)};
}

// 方程组:  $x \equiv r_i \pmod{m_i}$ , 返回  $(r, \text{lcm})$ ; 无解返回 nullopt。
optional<pair<i64, i64>> crt(const vector<pair<i64, i64>>& equations) {
    i64 r = 0;
    i64 m = 1;
    for(auto [ri, mi] : equations) {
        auto merged = crtMerge(r, m, ri, mi);
        if(!merged) {
            return nullopt;
        }
        r = merged->first;
        m = merged->second;
    }
    return pair<i64, i64>{r, m};
}

```

组合数

p 为质数; 构造时预处理到 $\min(n, p-1)$, 调用 `C(x, k)` 需满足 x 不超过该上界; $x \geq p$ 返回 0 (大组合数用 Lucas)。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct Combination {
    int mod;
    vector<int> fac, ifac;

    Combination(int n, int p) : mod(p) {
        int limit = min(n, p - 1);
        fac.assign(limit + 1, 1);
        for(int i = 1; i <= limit; i++) {
            fac[i] = int(1LL * fac[i - 1] * i % p);
        }
        ifac.assign(limit + 1, 1);
        ifac[limit] = powerMod(fac[limit], p - 2, p);
        for(int i = limit; i >= 1; i--) {
            ifac[i - 1] = int(1LL * ifac[i] * i % p);
        }
    }

    static int powerMod(int a, i64 e, int p) {
        int res = 1;
        while(e > 0) {
            if(e & 1) {
                res = int(1LL * res * a % p);
            }
            a = int(1LL * a * a % p);
            e >>= 1;
        }
        return res;
    }

    int C(i64 n, i64 k) const {
        if(k < 0 || k > n) {
            return 0;
        }
        if(n >= mod) {
            return 0;
        }
        int nn = int(n);
        int kk = int(k);
        return int(1LL * fac[nn] * ifac[kk] % mod * ifac[nn - kk] % mod);
    }
};

```

卢卡斯定理 (Lucas)

p 为小质数; $0 \leq k \leq n$ 。

```

#include <bits/stdc++.h>

```

```

using namespace std;

using i64 = long long;

struct Lucas {
    int p;
    vector<int> fac, ifac;

    Lucas(int p_) : p(p_), fac(p_), ifac(p_) {
        fac[0] = ifac[0] = 1;
        for(int i = 1; i < p; i++) {
            fac[i] = int(1LL * fac[i - 1] * i % p);
        }
        ifac[p - 1] = powerMod(fac[p - 1], p - 2, p);
        for(int i = p - 2; i >= 1; i--) {
            ifac[i] = int(1LL * ifac[i + 1] * (i + 1) % p);
        }
    }

    static int powerMod(int a, i64 e, int p) {
        int res = 1;
        while(e > 0) {
            if(e & 1) {
                res = int(1LL * res * a % p);
            }
            a = int(1LL * a * a % p);
            e >>= 1;
        }
        return res;
    }

    int smallC(int n, int k) const {
        if(k < 0 || k > n) {
            return 0;
        }
        return int(1LL * fac[n] * ifac[k] % p * ifac[n - k] % p);
    }

    int C(i64 n, i64 k) const {
        if(k < 0 || k > n) {
            return 0;
        }
        if(n < p) {
            return smallC(int(n), int(k));
        }
        return int(1LL * smallC(int(n % p), int(k % p)) * C(n / p, k / p) % p);
    }
};

```

拉格朗日插值

p 为质数且 $n < p$; x 落在 $[0, n]$ 时直接返回 $y[x]$ 。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

int powerMod(int a, i64 e, int p) {
    int res = 1;
    while(e > 0) {
        if(e & 1) {
            res = int(1LL * res * a % p);
        }
        a = int(1LL * a * a % p);
        e >>= 1;
    }
    return res;
}

int lagrangeInterpolate(const vector<int>& y, i64 x, int p) {
    int n = int(y.size()) - 1;
    if(n < 0) {
        return 0;
    }
    if(0 <= x && x <= n) {
        return y[int(x)];
    }
    i64 v = x % p;
    if(v < 0) {
        v += p;
    }

    vector<int> fac(n + 1, 1), ifac(n + 1);
    for(int i = 1; i <= n; i++) {
        fac[i] = int(1LL * fac[i - 1] * i % p);
    }
    ifac[n] = powerMod(fac[n], p - 2, p);
    for(int i = n; i >= 1; i--) {
        ifac[i - 1] = int(1LL * ifac[i] * i % p);
    }

    vector<int> pre(n + 1, 1), suf(n + 2, 1);
    for(int i = 1; i <= n; i++) {
        int d = int(v - (i - 1));
        if(d < 0) {
            d += p;
        }
        pre[i] = int(1LL * pre[i - 1] * d % p);
    }
    for(int i = n - 1; i >= 0; i--) {
        int d = int(v - (i + 1));
        if(d < 0) {
            d += p;
        }
        suf[i] = int(1LL * suf[i + 1] * d % p);
    }
}

```

```

    }

    int res = 0;
    for(int i = 0; i <= n; i++) {
        int cur = int(1LL * y[i] * pre[i] % p * suf[i] % p * ifac[i] % p * ifac[n - i] %
p);
        if((n - i) & 1) {
            res -= cur;
            if(res < 0) {
                res += p;
            }
        } else {
            res += cur;
            if(res >= p) {
                res -= p;
            }
        }
    }
    return res;
}

```

线性筛

`phi[1]=mu[1]=1;` `phi[0]=mu[0]=0。`

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct LinearSieve {
    int n;
    vector<int> primes, minp, phi, mu;

    LinearSieve(int n_ = 0) : n(n_), minp(n_ + 1), phi(n_ + 1), mu(n_ + 1) {
        if(n < 1) {
            return;
        }
        phi[1] = 1;
        mu[1] = 1;
        for(int i = 2; i <= n; i++) {
            if(minp[i] == 0) {
                minp[i] = i;
                primes.push_back(i);
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for(int p : primes) {
                i64 v = 1LL * i * p;
                if(v > n) {
                    break;
                }
            }
        }
    }
}

```

```

        int vv = int(v);
        minp[vv] = p;
        if(i % p == 0) {
            phi[vv] = phi[i] * p;
            mu[vv] = 0;
            break;
        }
        phi[vv] = phi[i] * (p - 1);
        mu[vv] = -mu[i];
    }
}
};

```

莫比乌斯 / 欧拉函数前缀和

`phiPrefix(i)` / `muPrefix(i)` 返回 $\sum_{j=1..i}$, $O(1)$ 查询。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct MobiusPhi {
    int n;
    vector<int> primes, minp, phi, mu;
    vector<i64> prefPhi, prefMu;

    MobiusPhi(int n_ = 0)
        : n(n_), minp(n_ + 1), phi(n_ + 1), mu(n_ + 1),
          prefPhi(n_ + 1), prefMu(n_ + 1) {
        if(n < 1) {
            return;
        }
        phi[1] = 1;
        mu[1] = 1;
        for(int i = 2; i <= n; i++) {
            if(minp[i] == 0) {
                minp[i] = i;
                primes.push_back(i);
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for(int p : primes) {
                i64 v = 1LL * i * p;
                if(v > n) {
                    break;
                }
                int vv = int(v);
                minp[vv] = p;
                if(i % p == 0) {
                    phi[vv] = phi[i] * p;

```

```

        mu[vv] = 0;
        break;
    }
    phi[vv] = phi[i] * (p - 1);
    mu[vv] = -mu[i];
}
}
for(int i = 1; i <= n; i++) {
    prefPhi[i] = prefPhi[i - 1] + phi[i];
    prefMu[i] = prefMu[i - 1] + mu[i];
}
}

i64 phiPrefix(int x) const { return prefPhi[x]; }
i64 muPrefix(int x) const { return prefMu[x]; }
};

```

米勒-拉宾素性测试

u64 确定性判素 ($n < 2^{64}$) ; 0/1 返回 false。

```

#include <bits/stdc++.h>
using namespace std;

using u64 = unsigned long long;
using u128 = __uint128_t;

u64 mulMod(u64 a, u64 b, u64 m) {
    return u64(u128(a) * b % m);
}

u64 powerMod(u64 a, u64 e, u64 m) {
    u64 res = 1 % m;
    while(e > 0) {
        if(e & 1) {
            res = mulMod(res, a, m);
        }
        a = mulMod(a, a, m);
        e >>= 1;
    }
    return res;
}

bool isPrime(u64 n) {
    if(n < 2) {
        return false;
    }
    for(u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL, 29ULL, 31ULL,
37ULL}) {
        if(n % p == 0) {
            return n == p;
        }
    }
}

```

```

}
u64 d = n - 1;
int s = 0;
while((d & 1) == 0) {
    d >>= 1;
    s++;
}
for(u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL}) {
    if(a % n == 0) {
        continue;
    }
    u64 x = powerMod(a % n, d, n);
    if(x == 1 || x == n - 1) {
        continue;
    }
    bool composite = true;
    for(int r = 1; r < s; r++) {
        x = mulMod(x, x, n);
        if(x == n - 1) {
            composite = false;
            break;
        }
    }
    if(composite) {
        return false;
    }
}
return true;
}

```

波拉德-ρ 大整数分解

`factorize(n)` 返回升序质因数（含重复），`n <= 1` 返回空；随机失败自动换参数重跑。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
using u64 = unsigned long long;
using u128 = __uint128_t;

struct PollardRho {
    static u64 mulMod(u64 a, u64 b, u64 m) {
        return u64(u128(a) * b % m);
    }

    static u64 powerMod(u64 a, u64 e, u64 m) {
        u64 res = 1 % m;
        while(e > 0) {
            if(e & 1) {
                res = mulMod(res, a, m);
            }
        }
    }
}

```

```

        a = mulMod(a, a, m);
        e >>= 1;
    }
    return res;
}

static bool isPrime(u64 n) {
    if(n < 2) {
        return false;
    }
    for(u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL, 29ULL,
31ULL, 37ULL}) {
        if(n % p == 0) {
            return n == p;
        }
    }
    u64 d = n - 1;
    int s = 0;
    while((d & 1) == 0) {
        d >>= 1;
        s++;
    }
    for(u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL})
{
        if(a % n == 0) {
            continue;
        }
        u64 x = powerMod(a % n, d, n);
        if(x == 1 || x == n - 1) {
            continue;
        }
        bool composite = true;
        for(int r = 1; r < s; r++) {
            x = mulMod(x, x, n);
            if(x == n - 1) {
                composite = false;
                break;
            }
        }
        if(composite) {
            return false;
        }
    }
    return true;
}

static i64 f(i64 x, i64 c, i64 n) {
    return i64((u128(x) * x + u64(c)) % u64(n));
}

static i64 pollard(i64 n, mt19937_64& rng) {
    if(n % 2 == 0) {
        return 2;
    }

```

```

    }
    while(true) {
        i64 c = i64(rng() % (n - 1)) + 1;
        i64 x = i64(rng() % (n - 1)) + 1;
        i64 y = x;
        i64 d = 1;
        while(d == 1) {
            x = f(x, c, n);
            y = f(f(y, c, n), c, n);
            i64 diff = x > y ? x - y : y - x;
            d = std::gcd(diff, n);
        }
        if(d != n) {
            return d;
        }
    }
}

static void factorRec(i64 n, mt19937_64& rng, vector<i64>& out) {
    if(n <= 1) {
        return;
    }
    if(isPrime(u64(n))) {
        out.push_back(n);
        return;
    }
    if(n % 2 == 0) {
        out.push_back(2);
        factorRec(n / 2, rng, out);
        return;
    }
    i64 d = pollard(n, rng);
    factorRec(d, rng, out);
    factorRec(n / d, rng, out);
}

static vector<i64> factorize(i64 n) {
    vector<i64> out;
    if(n <= 1) {
        return out;
    }
    mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    factorRec(n, rng, out);
    sort(out.begin(), out.end());
    return out;
}

static vector<i64> factorize(i64 n, u64 seed) {
    vector<i64> out;
    if(n <= 1) {
        return out;
    }
    mt19937_64 rng(seed);

```

```

        factorRec(n, rng, out);
        sort(out.begin(), out.end());
        return out;
    }
};

```

整除分块

$\text{floor}(n/i)$ 取值 $O(\sqrt{n})$ 种; $R = n / (n / L)$ 。双参数 $\text{floor}(n/i) * \text{floor}(m/i)$ 时右端取 $\min(n/(n/L), m/(m/L))$; $n/L == 0$ 直接收尾; 区间和先转 $i128$ 防溢出。

```

using i64 = long long;
using i128 = __int128_t;

i64 sumFloorDiv(i64 n, i64 k) {
    i128 ans = 0;
    for(i64 l = 1, r; l <= n; l = r + 1) {
        i64 v = k / l;
        if(v == 0) r = n;
        else r = min(n, k / v);
        ans += i128(r - l + 1) * v;
    }
    return i64(ans);
}

```

类欧几里得 (Floor Sum)

$\text{floorSum}(n, a, b, c)$ 令 $q_i = \text{floor}((a*i+b)/c)$ ($i=0..n$) , 返回 $\{f, g, h\}$, 其中 $f = \sum q_i$ 、 $g = \sum i * q_i$ 、 $h = \sum q_i^2$ 。支持负数 a/b ; 要求 $c > 0$ 。若题目按 $[0, n)$ 定义, 传 $n-1$ 。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
using i128 = __int128_t;

struct FloorSumResult {
    i64 f, g, h;
};

FloorSumResult floorSumCalc(i128 n, i128 a, i128 b, i128 c) {
    if(n < 0) {
        return {0, 0, 0};
    }
    i128 A = a / c;
    if(a % c != 0 && a < 0) {
        A--;
    }
    i128 B = b / c;
    if(b % c != 0 && b < 0) {
        B--;
    }
}

```

```

}
i128 ra = a - A * c;
i128 rb = b - B * c;

if(A != 0 || B != 0) {
    FloorSumResult t = floorSumCalc(n, ra, rb, c);
    i128 s0 = n + 1;
    i128 s1 = n * (n + 1) / 2;
    i128 s2 = n * (n + 1) * (2 * n + 1) / 6;
    return {
        i64(A * s1 + B * s0 + t.f),
        i64(A * s2 + B * s1 + t.g),
        i64(A * A * s2 + 2 * A * B * s1 + B * B * s0
            + 2 * A * t.g + 2 * B * t.f + t.h)
    };
}

if(a == 0) {
    return {0, 0, 0};
}
i128 m = (a * n + b) / c;
if(m == 0) {
    return {0, 0, 0};
}
FloorSumResult t = floorSumCalc(m - 1, c, c - b - 1, a);
return {
    i64(n * m - t.f),
    i64(m * n * (n + 1) / 2 - (t.f + t.h) / 2),
    i64(n * m * m - t.f - 2 * t.g)
};
}

FloorSumResult floorSum(i64 n, i64 a, i64 b, i64 c) {
    if(c <= 0) {
        return {0, 0, 0};
    }
    return floorSumCalc(n, a, b, c);
}

```

离散对数 (BSGS)

求一个非负 x 满足 $a^x \equiv b \pmod{m}$; 要求 $\gcd(a, m) = 1$ (模数可合数), 非互质返回 `nullopt`; $b == 1$ 返回 0, $a == 0$ 时 $b == 0$ 返回 1; 不保证返回最小解。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
using i128 = __int128_t;

struct BSGS {
    static i64 mulMod(i64 a, i64 b, i64 m) {

```

```

        return i64(i128(a) * b % m);
    }

    static i64 powerMod(i64 a, i64 e, i64 m) {
        i64 res = 1 % m;
        while(e > 0) {
            if(e & 1) {
                res = mulMod(res, a, m);
            }
            a = mulMod(a, a, m);
            e >>= 1;
        }
        return res;
    }

    static optional<i64> solve(i64 a, i64 b, i64 m) {
        if(m <= 0) {
            return nullopt;
        }
        if(m == 1) {
            return 0;
        }
        a %= m;
        if(a < 0) {
            a += m;
        }
        b %= m;
        if(b < 0) {
            b += m;
        }
        if(b == 1) {
            return 0;
        }
        if(a == 0) {
            return b == 0 ? optional<i64>(1) : nullopt;
        }
        if(std::gcd(a, m) != 1) {
            return nullopt;
        }

        i64 n = i64(ceil(sqrt((long double)m))) + 1;
        unordered_map<i64, i64> table;
        table.reserve(size_t(2 * n));
        i64 e = 1;
        for(i64 j = 0; j < n; j++) {
            if(!table.count(e)) {
                table[e] = j;
            }
            e = mulMod(e, a, m);
        }

        i64 an = powerMod(a, n, m);
        i64 x, y;

```

```

    exgcd(an, m, x, y);
    x %= m;
    if(x < 0) {
        x += m;
    }

    i64 cur = b;
    for(i64 i = 0; i <= n; i++) {
        auto it = table.find(cur);
        if(it != table.end()) {
            return i * n + it->second;
        }
        cur = mulMod(cur, x, m);
    }
    return nullopt;
}

private:
    static i64 exgcd(i64 a, i64 b, i64& x, i64& y) {
        if(b == 0) {
            x = 1;
            y = 0;
            return a;
        }
        i64 g = exgcd(b, a % b, y, x);
        y -= (a / b) * x;
        return g;
    }
};

```

矩阵快速幂（模质数）

1-based; 矩阵与向量下标 0 不使用; MOD 可改; 乘法要求 `a.col == b.row`, `power` 要求方阵。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

constexpr int MOD = 998244353;

struct Matrix {
    int row, col;
    vector<int> mat;

    Matrix(int row_ = 0, int col_ = 0) : row(row_), col(col_), mat((row_ + 1) * (col_ + 1)) {}

    int& operator()(int i, int j) {
        return mat[i * (col + 1) + j];
    }

    const int& operator()(int i, int j) const {
        return mat[i * (col + 1) + j];
    }
};

```

```

    }
    static Matrix identity(int n) {
        Matrix res(n, n);
        for(int i = 1; i <= n; i++) {
            res(i, i) = 1;
        }
        return res;
    }
};

Matrix operator*(const Matrix& a, const Matrix& b) {
    assert(a.col == b.row);
    Matrix res(a.row, b.col);
    for(int i = 1; i <= a.row; i++) {
        for(int k = 1; k <= a.col; k++) {
            int x = a(i, k);
            if(x == 0) {
                continue;
            }
            for(int j = 1; j <= b.col; j++) {
                res(i, j) = int((res(i, j) + 1LL * x * b(k, j)) % MOD);
            }
        }
    }
    return res;
}

Matrix power(Matrix a, int64 b) {
    assert(a.row == a.col);
    Matrix res = Matrix::identity(a.row);
    while(b > 0) {
        if(b & 1) {
            res = res * a;
        }
        a = a * a;
        b >>= 1;
    }
    return res;
}

vector<int> operator*(const Matrix& a, const vector<int>& v) {
    assert(a.col + 1 == int(v.size()));
    vector<int> res(a.row + 1, 0);
    for(int i = 1; i <= a.row; i++) {
        for(int j = 1; j <= a.col; j++) {
            res[i] = int((res[i] + 1LL * a(i, j) * v[j]) % MOD);
        }
    }
    return res;
}

```

杜教筛 (phi / mu 前缀和)

`get(n)` 返回 $\{\sum\phi(1..n), \sum\mu(1..n)\}$; 内部 i128 防溢出, 要求 $\sum\phi$ 不溢出 i64 (约 $n < 4.3e9$)。

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
using i128 = __int128_t;

struct DuJiaoSieve {
    int limit;
    vector<i64> prefPhi, prefMu;
    unordered_map<i64, pair<i64, i64>> memo;

    DuJiaoSieve(int limit_ = 2000000) : limit(limit_) {
        vector<int> minp(limit + 1), phi(limit + 1), mu(limit + 1);
        prefPhi.assign(limit + 1, 0);
        prefMu.assign(limit + 1, 0);
        if(limit < 1) {
            return;
        }
        phi[1] = 1;
        mu[1] = 1;
        vector<int> primes;
        for(int i = 2; i <= limit; i++) {
            if(!minp[i]) {
                minp[i] = i;
                primes.push_back(i);
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for(int p : primes) {
                i64 v = 1LL * i * p;
                if(v > limit) {
                    break;
                }
                int vv = int(v);
                minp[vv] = p;
                if(i % p == 0) {
                    phi[vv] = phi[i] * p;
                    mu[vv] = 0;
                    break;
                }
                phi[vv] = phi[i] * (p - 1);
                mu[vv] = -mu[i];
            }
        }
        for(int i = 1; i <= limit; i++) {
            prefPhi[i] = prefPhi[i - 1] + phi[i];
            prefMu[i] = prefMu[i - 1] + mu[i];
        }
    }
};
```

```

}

pair<i64, i64> get(i64 n) {
    if(n <= limit) {
        return {prefPhi[n], prefMu[n]};
    }
    auto it = memo.find(n);
    if(it != memo.end()) {
        return it->second;
    }
    i128 sumPhi = i128(n) * (n + 1) / 2;
    i64 sumMu = 1;
    for(i64 l = 2, r; l <= n; l = r + 1) {
        r = n / (n / l);
        auto [x, y] = get(n / l);
        sumPhi -= i128(x) * (r - l + 1);
        sumMu -= y * (r - l + 1);
    }
    return memo[n] = {i64(sumPhi), sumMu};
}
};

```

亚线性筛 (Min_25)

默认 $f(p^e) = p^e * (p^e - 1)$ (P5325) ; 其它题目改 `fpe()` 与 `sumFp/sumFpPrefix` 中的多项式系数。当前实现用费马逆元求 $1/2, 1/6$, 要求 `mod` 为与 6 互质的质数。 `solve()` 返回 $\sum_{i=1..n} f(i) \bmod \text{mod}$ 。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct Min25Sieve {
    i64 n;
    int mod;
    int sq;
    vector<int> primes;
    vector<i64> w;
    vector<int> id1, id2;
    vector<i64> g0, g1, g2;
    vector<i64> sp0, sp1, sp2;

    Min25Sieve(i64 n_, int mod_) : n(n_), mod(mod_) {
        sq = int(sqrt((long double)n));
        id1.assign(sq + 2, -1);
        id2.assign(sq + 2, -1);
        initsieve(sq);
        build();
    }

    // ---- 按题目修改: f(p^e) ----
    i64 fpe(i64 p, int e) const {

```

```

// 默认  $f(p^e) = p^e * (p^e - 1)$  (Luogu P5325)
i64 pe = powerMod(p % mod, e, mod);
return pe * ((pe - 1 + mod) % mod) % mod;
}
//  $f(p) = a_0 + a_1*p + a_2*p^2$ , 默认  $a_0=0, a_1=-1, a_2=1$  (即  $p^2-p$ )
i64 sumFp(i64 x) const {
    int k = get(x);
    return (g2[k] - g1[k] + mod) % mod;
}
i64 sumFpPrefix(int j) const {
    return (sp2[j] - sp1[j] + mod) % mod;
}

i64 solve() {
    return (S(n, 0) + 1) % mod;
}

```

private:

```

i64 powerMod(i64 a, i64 e, int m) const {
    i64 res = 1 % m;
    a %= m;
    while(e > 0) {
        if(e & 1) {
            res = res * a % m;
        }
        a = a * a % m;
        e >>= 1;
    }
    return res;
}

void initSieve(int lim) {
    vector<int> minp(lim + 1);
    sp0.assign(lim + 2, 0);
    sp1.assign(lim + 2, 0);
    sp2.assign(lim + 2, 0);
    for(int i = 2; i <= lim; i++) {
        if(!minp[i]) {
            minp[i] = i;
            primes.push_back(i);
            int j = int(primes.size());
            sp0[j] = sp0[j - 1] + 1;
            sp1[j] = (sp1[j - 1] + i) % mod;
            sp2[j] = (sp2[j - 1] + 1LL * i * i) % mod;
        }
        for(int p : primes) {
            i64 v = 1LL * i * p;
            if(v > lim) {
                break;
            }
            minp[int(v)] = p;
            if(minp[i] == p) {
                break;
            }
        }
    }
}

```

```

    }
}
}
void build() {
    for(i64 l = 1, r; l <= n; l = r + 1) {
        r = n / (n / l);
        i64 v = n / l;
        w.push_back(v);
        if(v <= sq) {
            id1[int(v)] = int(w.size()) - 1;
        } else {
            id2[int(n / v)] = int(w.size()) - 1;
        }
    }
    i64 inv2 = powerMod(2, mod - 2, mod);
    i64 inv6 = powerMod(6, mod - 2, mod);
    g0.assign(w.size(), 0);
    g1.assign(w.size(), 0);
    g2.assign(w.size(), 0);
    for(int i = 0; i < int(w.size()); i++) {
        i64 x = w[i] % mod;
        g0[i] = (x - 1 + mod) % mod;
        g1[i] = (x * ((x + 1) % mod) % mod * inv2 % mod - 1 + mod) % mod;
        i64 t = x * ((x + 1) % mod) % mod * ((2 * x + 1) % mod) % mod;
        g2[i] = (t * inv6 % mod - 1 + mod) % mod;
    }
    for(int j = 0; j < int(primes.size()); j++) {
        i64 p = primes[j];
        i64 p2 = p * p;
        for(int i = 0; i < int(w.size()); i++) {
            if(w[i] < p2) {
                break;
            }
            int k = get(w[i] / p);
            g0[i] = (g0[i] - (g0[k] - sp0[j] + mod) % mod + mod) % mod;
            g1[i] = (g1[i] - p * ((g1[k] - sp1[j] + mod) % mod) % mod + mod) % mod;
            g2[i] = (g2[i] - p2 % mod * ((g2[k] - sp2[j] + mod) % mod) % mod + mod) %
mod;
        }
    }
}
int get(i64 x) const {
    return x <= sq ? id1[int(x)] : id2[int(n / x)];
}
i64 s(i64 x, int j) {
    if(x <= 1 || (j < int(primes.size()) && primes[j] > x)) {
        return 0;
    }
    i64 ans = (sumFp(x) - sumFpPrefix(j) + mod) % mod;
    for(int i = j; i < int(primes.size()); i++) {
        i64 p = primes[i];
        if(p * p > x) {
            break;
        }
    }
}

```

```

    }
    i64 pe = p;
    int e = 1;
    while(pe * p <= x) {
        ans = (ans + fpe(p, e) * S(x / pe, i + 1)) % mod;
        ans = (ans + fpe(p, e + 1)) % mod;
        pe *= p;
        e++;
    }
}
return ans;
}
};

```

约数触发更新

每次频次 cnt 增加时枚举其约数并更新聚合；约数表用调和级数预处理 $O(N \log N)$ 。更新顺序必须是“先删旧贡献再插入新贡献”。

```

for(int d : fac[cnt]) {
    ans ^= w[d] * d;
    w[d]++;
    ans ^= w[d] * d;
}

```

半环矩阵幂

max-plus (或 min-plus) 矩阵乘法 $c[i][j] = \max_k a[i][k] + b[k][j]$ ；单位阵对角线 0、其余 $-\text{INF}$ ；快速幂同普通矩阵。 $-\text{INF}$ 项跳过防溢出。

同余按 gcd 分类计数

$xy \equiv c \pmod n$ 的解数 $= \sum_{d|n, d|c} d \cdot \phi(n/d)$ ；前缀求和 $\sum_{d|n} d \cdot \phi(n/d) \cdot ([m/g(d)]+1)$ ，其中 $g(d) = \prod p^{\lfloor e/2 \rfloor}$ 满足 $d|k^2 \Leftrightarrow g(d)|k$ 。

二进制逐位提升 (2-adic)

奇数底数求 $a^x \equiv b \pmod{2^k}$ 的最小 x ：逐位检查低 i 位，不匹配则 $x += p$ ； $\text{cur} *= a^p$ ，同时 $a^p = a^{p^2}$ ； $p \ll 1$ 。k=64 时掩码特判 $x == y$ 。

```

if(!sameLow(cur, b, i)) {
    x += p;
    cur *= ap;
}
if(!sameLow(cur, b, i)) return false;
if(!sameLow(ap, 1, i)) {
    p <<= 1;
    ap *= ap;
}

```

欧拉降幂幂塔

$a^b \bmod m$ 非互质时必须带“值 $\geq m$ ”标记: `tagPow` 返回 `res + (flag ? m : 0)`; `m == 1` 返回 1; 每层模数取 `phi(m)`。

欧几里得批量跳跃

连续 `x -= y` 到变号需 `t = (x+y-1)/y - 1` 次; 一次 `x -= y*t` 整批跳过。概率版每批乘 `p^t` (快速幂)。

6. 多项式

快速傅里叶变换 (FFT, 整数卷积)

系数 0-based; 支持负数, 四舍五入取整; 结果约 $1e14$ 内安全。

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
using db = double;
using cdb = complex<db>;

void fft(vector<cdb>& a, bool invert) {
    int n = int(a.size());
    for(int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for(; j & bit; bit >>= 1) {
            j ^= bit;
        }
        j ^= bit;
        if(i < j) {
            swap(a[i], a[j]);
        }
    }
    for(int len = 2; len <= n; len <= 1) {
        db ang = 2 * acos(-1.0) / len * (invert ? -1 : 1);
        cdb wlen(cos(ang), sin(ang));
        for(int i = 0; i < n; i += len) {
            cdb w(1);
            for(int j = 0; j < len / 2; j++) {
                cdb u = a[i + j];
                cdb v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w *= wlen;
            }
        }
    }
    if(invert) {
        for(cdb& x : a) {
            x /= n;
        }
    }
}
```

```

    }
}

vector<i64> convolutionFft(const vector<int>& a, const vector<int>& b) {
    if(a.empty() || b.empty()) {
        return {};
    }
    int need = int(a.size() + b.size() - 1);
    int n = 1;
    while(n < need) {
        n <<= 1;
    }
    vector<cd> A(n), B(n);
    for(int i = 0; i < int(a.size()); i++) {
        A[i] = a[i];
    }
    for(int i = 0; i < int(b.size()); i++) {
        B[i] = b[i];
    }
    fft(A, false);
    fft(B, false);
    for(int i = 0; i < n; i++) {
        A[i] *= B[i];
    }
    fft(A, true);
    vector<i64> res(need);
    for(int i = 0; i < need; i++) {
        res[i] = llround(A[i].real());
    }
    return res;
}

```

数论变换 (NTT, 模 998244353)

系数 0-based; 长度须为 2 的幂且 $(\text{MOD}-1) \% n == 0$ 。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

constexpr int MOD = 998244353;
constexpr int G = 3;
constexpr int MAX_NTT_SIZE = 1 << 23;

int normalize(i64 x) {
    x %= MOD;
    if(x < 0) {
        x += MOD;
    }
    return int(x);
}

```

```

int powerMod(int a, int b) {
    a = normalize(a);
    int res = 1;
    while(b > 0) {
        if(b & 1) {
            res = int(1LL * res * a % MOD);
        }
        a = int(1LL * a * a % MOD);
        b >>= 1;
    }
    return res;
}

void ntt(vector<int>& a, bool invert) {
    int n = int(a.size());
    if(n <= 1) {
        return;
    }
    assert((n & (n - 1)) == 0);
    assert(n <= MAX_NTT_SIZE);
    assert((MOD - 1) % n == 0);
    for(int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        while(j & bit) {
            j ^= bit;
            bit >>= 1;
        }
        j ^= bit;
        if(i < j) {
            swap(a[i], a[j]);
        }
    }
    for(int len = 2; len <= n; len <= 1) {
        int wlen = powerMod(G, (MOD - 1) / len);
        if(invert) {
            wlen = powerMod(wlen, MOD - 2);
        }
        int half = len >> 1;
        for(int left = 0; left < n; left += len) {
            i64 w = 1;
            for(int j = 0; j < half; j++) {
                int u = a[left + j];
                int v = int(w * a[left + j + half] % MOD);
                int sum = u + v;
                if(sum >= MOD) {
                    sum -= MOD;
                }
                int diff = u - v;
                if(diff < 0) {
                    diff += MOD;
                }
                a[left + j] = sum;
            }
        }
    }
}

```

```

        a[left + j + half] = diff;
        w = w * wlen % MOD;
    }
}
}
if(invert) {
    int invN = powerMod(n, MOD - 2);
    for(int& x : a) {
        x = int(1LL * x * invN % MOD);
    }
}
}

vector<int> convolutionNaive(const vector<int>& a, const vector<int>& b) {
    if(a.empty() || b.empty()) {
        return {};
    }
    vector<int> c(a.size() + b.size() - 1);
    for(int i = 0; i < int(a.size()); i++) {
        for(int j = 0; j < int(b.size()); j++) {
            c[i + j] = int((c[i + j] + 1LL * a[i] * b[j]) % MOD);
        }
    }
    return c;
}

vector<int> convolution(vector<int> a, vector<int> b) {
    if(a.empty() || b.empty()) {
        return {};
    }
    for(int& x : a) {
        x = normalize(x);
    }
    for(int& x : b) {
        x = normalize(x);
    }
    int resSize = int(a.size() + b.size() - 1);
    if(1LL * a.size() * b.size() <= 4096) {
        return convolutionNaive(a, b);
    }
    assert(resSize <= MAX_NTT_SIZE);
    int n = 1;
    while(n < resSize) {
        n <= 1;
    }
    a.resize(n);
    b.resize(n);
    ntt(a, false);
    ntt(b, false);
    for(int i = 0; i < n; i++) {
        a[i] = int(1LL * a[i] * b[i] % MOD);
    }
    ntt(a, true);
}

```

```

    a.resize(ressize);
    return a;
}

vector<int> convolutionTrunc(vector<int> a, vector<int> b, int limit) {
    if(limit <= 0) {
        return {};
    }
    if(int(a.size()) > limit) {
        a.resize(limit);
    }
    if(int(b.size()) > limit) {
        b.resize(limit);
    }
    vector<int> c = convolution(a, b);
    if(int(c.size()) > limit) {
        c.resize(limit);
    }
    return c;
}

```

快速沃尔什变换 (FWT, OR / AND / XOR 卷积)

掩码 0-based; 输入长度补齐为 2 的幂; `convOr/convAnd/convXor(a, b)` 返回同长卷积, `fwt*(a, inv)` 可单独使用。P 需为奇素数 (XOR 逆需要 2 的幂可逆)。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

constexpr int P = 998244353;

int powerMod(int a, i64 b) {
    int res = 1;
    while(b > 0) {
        if(b & 1) {
            res = int(1LL * res * a % P);
        }
        a = int(1LL * a * a % P);
        b >>= 1;
    }
    return res;
}

// OR 卷积变换: 正变换 y += x, 逆变换 y -= x
void fwtOr(vector<int>& a, bool inv) {
    int n = int(a.size());
    for(int len = 1; len < n; len <= 1) {
        for(int i = 0; i < n; i += len << 1) {
            for(int j = 0; j < len; j++) {
                int& x = a[i + j];

```

```
void fwtXor(vector<int>& a, bool inv) {
    int n = int(a.size());
    for(int len = 1; len < n; len <= 1) {
        for(int i = 0; i < n; i += len < 1) {
            for(int j = 0; j < len; j++) {
                int x = a[i + j];
                int y = a[i + j + len];
                a[i + j] = x + y >= P ? x + y - P : x + y;
                a[i + j + len] = x - y < 0 ? x - y + P : x - y;
            }
        }
    }
}
```

```

    }
    if(inv) {
        int invN = powerMod(n, P - 2);
        for(int& v : a) {
            v = int(1LL * v * invN % P);
        }
    }
}

vector<int> convOr(vector<int> a, vector<int> b) {
    int n = 1;
    while(n < int(max(a.size(), b.size()))) {
        n <= 1;
    }
    a.resize(n);
    b.resize(n);
    fwtOr(a, false);
    fwtOr(b, false);
    for(int i = 0; i < n; i++) {
        a[i] = int(1LL * a[i] * b[i] % P);
    }
    fwtOr(a, true);
    return a;
}

vector<int> convAnd(vector<int> a, vector<int> b) {
    int n = 1;
    while(n < int(max(a.size(), b.size()))) {
        n <= 1;
    }
    a.resize(n);
    b.resize(n);
    fwtAnd(a, false);
    fwtAnd(b, false);
    for(int i = 0; i < n; i++) {
        a[i] = int(1LL * a[i] * b[i] % P);
    }
    fwtAnd(a, true);
    return a;
}

vector<int> convXor(vector<int> a, vector<int> b) {
    int n = 1;
    while(n < int(max(a.size(), b.size()))) {
        n <= 1;
    }
    a.resize(n);
    b.resize(n);
    fwtXor(a, false);
    fwtXor(b, false);
    for(int i = 0; i < n; i++) {
        a[i] = int(1LL * a[i] * b[i] % P);
    }
}

```

```

    fwtXor(a, true);
    return a;
}

```

形式幂级数：逆 / ln / exp

系数 0-based; `polyInv` 要求 `a[0]!=0`, `polyLog` 要求 `a[0]!=0` (通常 1), `polyExp` 要求 `a[0]==0`; NTT 内嵌。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

constexpr int FPS_MOD = 998244353;
constexpr int FPS_G = 3;

int fpsPower(int a, int b) {
    a %= FPS_MOD;
    if(a < 0) {
        a += FPS_MOD;
    }
    int res = 1;
    while(b > 0) {
        if(b & 1) {
            res = int(1LL * res * a % FPS_MOD);
        }
        a = int(1LL * a * a % FPS_MOD);
        b >>= 1;
    }
    return res;
}

void fpsNtt(vector<int>& a, bool invert) {
    int n = int(a.size());
    if(n <= 1) {
        return;
    }
    assert((n & (n - 1)) == 0);
    assert((FPS_MOD - 1) % n == 0);
    for(int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        while(j & bit) {
            j ^= bit;
            bit >>= 1;
        }
        j ^= bit;
        if(i < j) {
            swap(a[i], a[j]);
        }
    }
    for(int len = 2; len <= n; len <<= 1) {

```

```

    int wlen = fpsPower(FPS_G, (FPS_MOD - 1) / len);
    if(invert) {
        wlen = fpsPower(wlen, FPS_MOD - 2);
    }
    int half = len >> 1;
    for(int left = 0; left < n; left += len) {
        i64 w = 1;
        for(int j = 0; j < half; j++) {
            int u = a[left + j];
            int v = int(w * a[left + j + half] % FPS_MOD);
            int sum = u + v;
            if(sum >= FPS_MOD) {
                sum -= FPS_MOD;
            }
            int diff = u - v;
            if(diff < 0) {
                diff += FPS_MOD;
            }
            a[left + j] = sum;
            a[left + j + half] = diff;
            w = w * wlen % FPS_MOD;
        }
    }
}

if(invert) {
    int invN = fpsPower(n, FPS_MOD - 2);
    for(int& x : a) {
        x = int(1LL * x * invN % FPS_MOD);
    }
}

}

vector<int> fpsConvTrunc(vector<int> a, vector<int> b, int limit) {
    if(limit <= 0) {
        return {};
    }
    if(int(a.size()) > limit) {
        a.resize(limit);
    }
    if(int(b.size()) > limit) {
        b.resize(limit);
    }
    if(a.empty() || b.empty()) {
        return vector<int>(limit, 0);
    }
    int resSize = int(a.size() + b.size() - 1);
    if(1LL * a.size() * b.size() <= 4096) {
        vector<int> c(min(resSize, limit), 0);
        for(int i = 0; i < int(a.size()); i++) {
            for(int j = 0; j < int(b.size()) && i + j < limit; j++) {
                c[i + j] = int((c[i + j] + 1LL * a[i] * b[j]) % FPS_MOD);
            }
        }
    }
}

```

```

        return c;
    }
    int n = 1;
    while(n < min(resSize, limit)) {
        n <= 1;
    }
    a.resize(n);
    b.resize(n);
    fpsNtt(a, false);
    fpsNtt(b, false);
    for(int i = 0; i < n; i++) {
        a[i] = int(1LL * a[i] * b[i] % FPS_MOD);
    }
    fpsNtt(a, true);
    a.resize(min(resSize, limit));
    return a;
}

//  $a(x)^{-1} \bmod x^{\text{limit}}$ , 要求  $a[0] \neq 0$ 
vector<int> polyInv(const vector<int>& a, int limit) {
    if(limit <= 0) {
        return {};
    }
    assert(!a.empty() && a[0] != 0);
    vector<int> b{fpsPower(a[0], FPS_MOD - 2)};
    while(int(b.size()) < limit) {
        int len = min(2 * int(b.size()), limit);
        vector<int> f(min(int(a.size()), len));
        copy(a.begin(), a.begin() + int(f.size()), f.begin());
        vector<int> t = fpsConvTrunc(f, b, len);
        t.resize(len);
        for(int& x : t) {
            if(x != 0) {
                x = FPS_MOD - x;
            }
        }
        t[0] += 2;
        if(t[0] >= FPS_MOD) {
            t[0] -= FPS_MOD;
        }
        b = fpsConvTrunc(b, t, len);
        b.resize(len);
    }
    return b;
}

//  $\ln(a(x)) \bmod x^{\text{limit}}$ , 要求  $a[0] \neq 0$  (通常为 1)
vector<int> polyLog(const vector<int>& a, int limit) {
    if(limit <= 0) {
        return {};
    }
    assert(!a.empty() && a[0] != 0);
    vector<int> inv = polyInv(a, limit);

```

```

vector<int> der(min(int(a.size()) - 1, limit));
for(int i = 0; i < int(der.size()); i++) {
    der[i] = int(1LL * a[i + 1] * (i + 1) % FPS_MOD);
}
vector<int> prod = fpsConvTrunc(der, inv, limit);
prod.resize(limit, 0);
vector<int> invInt(limit + 1, 1);
for(int i = 2; i <= limit; i++) {
    invInt[i] = int(1LL * (FPS_MOD - FPS_MOD / i) * invInt[FPS_MOD % i] % FPS_MOD);
}
vector<int> res(limit, 0);
for(int i = 1; i < limit; i++) {
    res[i] = int(1LL * prod[i - 1] * invInt[i] % FPS_MOD);
}
return res;
}

// exp(a(x)) mod x^limit, 要求 a[0] == 0
vector<int> polyExp(const vector<int>& a, int limit) {
    if(limit <= 0) {
        return {};
    }
    assert(a.empty() || a[0] == 0);
    vector<int> b{1};
    while(int(b.size()) < limit) {
        int len = min(2 * int(b.size()), limit);
        vector<int> lb = polyLog(b, len);
        vector<int> f(len, 0);
        for(int i = 0; i < len && i < int(a.size()); i++) {
            f[i] = a[i];
        }
        for(int i = 0; i < len; i++) {
            f[i] = (f[i] - lb[i] + FPS_MOD) % FPS_MOD;
        }
        f[0] = (f[0] + 1) % FPS_MOD; // 1 + a - ln(b)
        b = fpsConvTrunc(b, f, len);
        b.resize(len);
    }
    return b;
}

```

7. 线性代数

高斯消元 (Gauss-Jordan)

1-based; `a[0]` 与 `a[i][0]` 不使用; 最后一列视为 RHS 时不选主元; 返回秩, `pivotCol[col]` 为主元行 (无主元 0) 。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

```

```

using db = double;

constexpr db EPS = 1e-9;

int gaussDouble(vector<vector<db>>& a, vector<int>& pivotCol, db eps = EPS) {
    int n = int(a.size()) - 1;
    int m = n > 0 ? int(a[1].size()) - 1 : 0;
    pivotCol.assign(m + 1, 0);
    int row = 1;
    int limit = (m == n + 1 ? m - 1 : m);
    for(int col = 1; col <= limit && row <= n; col++) {
        int best = row;
        for(int i = row + 1; i <= n; i++) {
            if(fabs(a[i][col]) > fabs(a[best][col])) {
                best = i;
            }
        }
        if(fabs(a[best][col]) < eps) {
            continue;
        }
        swap(a[best], a[row]);
        db inv = 1.0 / a[row][col];
        for(int j = col; j <= m; j++) {
            a[row][j] *= inv;
        }
        for(int i = 1; i <= n; i++) {
            if(i == row) {
                continue;
            }
            db fac = a[i][col];
            for(int j = col; j <= m; j++) {
                a[i][j] -= fac * a[row][j];
            }
        }
        pivotCol[col] = row++;
    }
    return row - 1;
}

int gaussMod(vector<vector<int>>& a, int mod, vector<int>& pivotCol) {
    int n = int(a.size()) - 1;
    int m = n > 0 ? int(a[1].size()) - 1 : 0;
    pivotCol.assign(m + 1, 0);
    int row = 1;
    auto powerMod = [&](int x, int e) {
        int res = 1;
        while(e > 0) {
            if(e & 1) {
                res = int(1LL * res * x % mod);
            }
            x = int(1LL * x * x % mod);
            e >>= 1;
        }
    };

```

```

        return res;
    };
    int limit = (m == n + 1 ? m - 1 : m);
    for(int col = 1; col <= limit && row <= n; col++) {
        int best = 0;
        for(int i = row; i <= n; i++) {
            if(a[i][col] != 0) {
                best = i;
                break;
            }
        }
        if(best == 0) {
            continue;
        }
        swap(a[best], a[row]);
        int inv = powerMod(a[row][col], mod - 2);
        for(int j = col; j <= m; j++) {
            a[row][j] = int(1LL * a[row][j] * inv % mod);
        }
        for(int i = 1; i <= n; i++) {
            if(i == row) {
                continue;
            }
            int fac = a[i][col];
            for(int j = col; j <= m; j++) {
                a[i][j] = int((a[i][j] - 1LL * fac * a[row][j] % mod + mod) % mod);
            }
        }
        pivotCol[col] = row++;
    }
    return row - 1;
}

```

```

int gaussXor(vector<vector<int>>& a, vector<int>& pivotCol) {
    int n = int(a.size()) - 1;
    int m = n > 0 ? int(a[1].size()) - 1 : 0;
    pivotCol.assign(m + 1, 0);
    int row = 1;
    int limit = (m == n + 1 ? m - 1 : m);
    for(int col = 1; col <= limit && row <= n; col++) {
        int best = 0;
        for(int i = row; i <= n; i++) {
            if(a[i][col]) {
                best = i;
                break;
            }
        }
        if(best == 0) {
            continue;
        }
        swap(a[best], a[row]);
        for(int i = 1; i <= n; i++) {
            if(i != row && a[i][col]) {

```

```

        for(int j = col; j <= m; j++) {
            a[i][j] ^= a[row][j];
        }
    }
    pivotCol[col] = row++;
}
return row - 1;
}

```

异或线性基 (u64)

bit 63 是数据位; `kthXor` 1-based, 0 可由非空子集得到时第 1 小为 0; 越界/空基返回 `ULLONG_MAX`。

```

#include <bits/stdc++.h>
using namespace std;

using u64 = unsigned long long;

struct XorBasis {
    array<u64, 64> b{};
    int num = 0;
    bool canBeZero = false;
    vector<u64> basis;

    bool insert(u64 v) {
        for(int i = 63; i >= 0; i--) {
            if(!(v >> i)) {
                continue;
            }
            if(b[i] == 0) {
                b[i] = v;
                num++;
                basis.clear();
                return true;
            }
            v ^= b[i];
        }
        canBeZero = true;
        return false;
    }

    bool decompose(u64 v) const {
        for(int i = 63; i >= 0; i--) {
            if(!(v >> i)) {
                continue;
            }
            if(b[i] == 0) {
                return false;
            }
            v ^= b[i];
        }
    }
}

```

```

        return true;
    }

    u64 maxXor() const {
        u64 res = 0;
        for(int i = 63; i >= 0; i--) {
            res = max(res, res ^ b[i]);
        }
        return res;
    }

    u64 maxXorWithVal(u64 v) const {
        u64 res = v;
        for(int i = 63; i >= 0; i--) {
            res = max(res, res ^ b[i]);
        }
        return res;
    }

    u64 minXor() const {
        if(canBeZero) {
            return 0;
        }
        for(int i = 0; i < 64; i++) {
            if(b[i]) {
                return b[i];
            }
        }
        return numeric_limits<u64>::max();
    }

    void initOnce() {
        if(!basis.empty()) {
            return;
        }
        vector<u64> tmp(b.begin(), b.end());
        for(int i = 0; i < 64; i++) {
            if(tmp[i] == 0) {
                continue;
            }
            for(int j = i - 1; j >= 0; j--) {
                if(tmp[i] >> j & 1) {
                    tmp[i] ^= tmp[j];
                }
            }
            basis.push_back(tmp[i]);
        }
    }

    u64 kthXor(u64 k) {
        initOnce();
        if(canBeZero) {
            k--;
        }
    }

```

```

    }
    if(basis.size() < 64 && k >= (u64(1) << basis.size())) {
        return numeric_limits<u64>::max();
    }
    u64 res = 0;
    for(int i = 0; i < int(basis.size()); i++) {
        if(k >> i & 1) {
            res ^= basis[i];
        }
    }
    return res;
}

void merge(const XorBasis& other) {
    canBeZero = canBeZero || other.canBeZero;
    for(int i = 63; i >= 0; i--) {
        if(other.b[i]) {
            insert(other.b[i]);
        }
    }
}

};

```

二元域多右端项线性方程组 (GF(2))

`m + p <= MAXC`; `setCoeff(row, col, v)` 填系数, `setRhs(row, k, v)` 填右端项; `solve()` 无解返回 false, 有解时 `answer()[var][k]` 为第 k 个右端项下该变量的 0/1 解, 自由变量取 0。

```

#include <bits/stdc++.h>
using namespace std;

// 行向量 a[i]: 低 m 位是变量系数, 第 m+k 位是第 k 个右端项
template <int MAXC>
struct XorLinearSystem {
    using B = bitset<MAXC>;

    int n, m, p;
    vector<B> a;
    vector<int> where; // 主元变量所在行, -1 表示自由变量

    XorLinearSystem(int n_, int m_, int p_) : n(n_), m(m_), p(p_), a(n_), where(m_, -1) {}

    void setCoeff(int row, int col, bool v) {
        a[row].set(col, v);
    }

    void setRhs(int row, int k, bool v) {
        a[row].set(m + k, v);
    }

    bool solve() {
        where.assign(m, -1);
    }
};

```

```

int r = 0;
for(int c = 0; c < m && r < n; c++) {
    int s = -1;
    for(int i = r; i < n; i++) {
        if(a[i][c]) {
            s = i;
            break;
        }
    }
    if(s == -1) {
        continue;
    }
    swap(a[r], a[s]);
    where[c] = r;
    for(int i = 0; i < n; i++) {
        if(i != r && a[i][c]) {
            a[i] ^= a[r];
        }
    }
    r++;
}
for(int i = 0; i < n; i++) {
    bool lhs = false;
    bool rhs = false;
    for(int j = 0; j < m; j++) {
        lhs |= a[i][j];
    }
    for(int k = 0; k < p; k++) {
        rhs |= a[i][m + k];
    }
    if(!lhs && rhs) {
        return false;
    }
}
return true;
}

```

// 仅在 solve() 返回 true 后调用; x[var][k] 为第 k 个右端项下该变量的取值

```

vector<vector<int>> answer() const {
    vector<vector<int>> x(m, vector<int>(p, 0));
    for(int c = 0; c < m; c++) {
        if(where[c] == -1) {
            continue;
        }
        int row = where[c];
        for(int k = 0; k < p; k++) {
            x[c][k] = a[row][m + k] ? 1 : 0;
        }
    }
    return x;
}
};

```

8. 字符串

前缀函数匹配 (KMP)

1-based; `kmpMatch` 返回所有匹配起点的 1-based 位置 (升序) ; `pattern` 为空返回空。

```
#include <bits/stdc++.h>
using namespace std;

vector<int> getNext(const string& pattern) {
    int m = int(pattern.size());
    string p = " " + pattern;
    vector<int> next(m + 1, 0);
    for(int i = 2; i <= m; i++) {
        int j = next[i - 1];
        while(j && p[i] != p[j + 1]) {
            j = next[j];
        }
        if(p[i] == p[j + 1]) {
            j++;
        }
        next[i] = j;
    }
    return next;
}

vector<int> kmpMatch(const string& text, const string& pattern) {
    vector<int> next = getNext(pattern);
    int n = int(text.size());
    int m = int(pattern.size());
    vector<int> result;
    if(m == 0) {
        return result;
    }
    string t = " " + text;
    string p = " " + pattern;
    int j = 0;
    for(int i = 1; i <= n; i++) {
        while(j && t[i] != p[j + 1]) {
            j = next[j];
        }
        if(t[i] == p[j + 1]) {
            j++;
        }
        if(j == m) {
            result.push_back(i - m + 1);
            j = next[j];
        }
    }
    return result;
}
```

Z 函数 / 扩展 KMP

1-based; `z[1] = n`; `exKmp` 的 `ext[i]` 是 `text[i..]` 与 `pattern` 的 LCP。

```
#include <bits/stdc++.h>
using namespace std;

vector<int> zFunction(const string& s) {
    string t = " " + s;
    int n = int(s.size());
    vector<int> z(n + 1);
    z[1] = n;
    for(int i = 2, l = 0, r = 0; i <= n; i++) {
        if(i <= r) {
            z[i] = min(z[i - l + 1], r - i + 1);
        }
        while(i + z[i] <= n && t[1 + z[i]] == t[i + z[i]]) {
            z[i]++;
        }
        if(i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}

vector<int> exKmp(const string& text, const string& pattern) {
    vector<int> z = zFunction(pattern);
    string t = " " + text;
    string p = " " + pattern;
    int n = int(text.size());
    int m = int(pattern.size());
    vector<int> ext(n + 1);
    for(int i = 1, l = 0, r = 0; i <= n; i++) {
        if(i <= r) {
            ext[i] = min(z[i - l + 1], r - i + 1);
        }
        while(i + ext[i] <= n && ext[i] < m && p[1 + ext[i]] == t[i + ext[i]]) {
            ext[i]++;
        }
        if(i + ext[i] - 1 > r) {
            l = i;
            r = i + ext[i] - 1;
        }
    }
    return ext;
}
```

滚动哈希

1-based; `get(l, r)` 是闭区间 `[l, r]` 子串的 u64 自然溢出哈希；固定 base 有构造碰撞风险，正式比赛建议随机 base 或双哈希。

```
#include <bits/stdc++.h>
using namespace std;

using u64 = unsigned long long;

struct RollingHash {
    u64 base;
    vector<u64> pw, h;

    RollingHash(const string& s, u64 base_ = 19260817) : base(base_) {
        string t = " " + s;
        int n = int(s.size());
        pw.assign(n + 1, 1);
        h.assign(n + 1, 0);
        for(int i = 1; i <= n; i++) {
            pw[i] = pw[i - 1] * base;
            h[i] = h[i - 1] * base + u64(t[i]);
        }
    }

    u64 get(int l, int r) const {
        return h[r] - h[l - 1] * pw[r - l + 1];
    }
};
```

子序列自动机 (next-occurrence)

建表 $O(n\sigma)$ 、查询 $O(|t|)$ ；`isSubsequence(t)` 判断 `t` 是否为 `s` 的子序列；`nxt[i][c]` 为 $\geq i$ 的首个字符 `c`（不存在为 `n+1`）。非 'a' 字符域用 `offset` 参数。

```
#include <bits/stdc++.h>
using namespace std;

struct SubsequenceAutomaton {
    int n, sigma;
    vector<vector<int>> nxt; // nxt[i][c]: 从位置 i (含) 起第一个字符 c, 无则 n + 1

    SubsequenceAutomaton(const string& s, int sigma_, char offset = 'a')
        : n(int(s.size())), sigma(sigma_), nxt(n + 2, vector<int>(sigma_, n + 1)) {
        for(int i = n; i >= 1; i--) {
            nxt[i] = nxt[i + 1];
            nxt[i][s[i - 1] - offset] = i;
        }
    }

    bool isSubsequence(const string& t, char offset = 'a') const {
        int cur = 0;
```

```

        for(char c : t) {
            cur = nxt[cur + 1][c - offset];
            if(cur == n + 1) {
                return false;
            }
        }
        return true;
    }
};

```

字典树 (Trie)

结点 0 为根；字符域由模板参数控制（默认 26 个 'a' 起）。countPrefix 是前缀出现次数。

```

#include <bits/stdc++.h>
using namespace std;

template <int SIGMA = 26, char BASE = 'a'>
struct Trie {
    struct Node {
        array<int, SIGMA> ch{};
        int cnt = 0;
        int end = 0;
    };
    vector<Node> tr;

    Trie(int n = 0) {
        if(n) {
            tr.reserve(n);
        }
        tr.emplace_back();
    }
    int newNode() {
        tr.emplace_back();
        return int(tr.size()) - 1;
    }
    int id(char c) const {
        return c - BASE;
    }
    int size() const {
        return int(tr.size());
    }
    void clear() {
        tr.clear();
        tr.emplace_back();
    }
    int go(int u, char c) {
        int x = id(c);
        if(!tr[u].ch[x]) {
            tr[u].ch[x] = newNode();
        }
        return tr[u].ch[x];
    }
};

```

```

}
int next(int u, char c) const {
    int x = id(c);
    int v = tr[u].ch[x];
    return v ? v : -1;
}
int insert(const string& s) {
    int u = 0;
    tr[u].cnt++;
    for(char c : s) {
        u = go(u, c);
        tr[u].cnt++;
    }
    tr[u].end++;
    return u;
}
int find(const string& s) const {
    int u = 0;
    for(char c : s) {
        int x = id(c);
        if(!tr[u].ch[x]) {
            return -1;
        }
        u = tr[u].ch[x];
    }
    return u;
}
int count(const string& s) const {
    int u = find(s);
    return u == -1 ? 0 : tr[u].end;
}
int countPrefix(const string& s) const {
    int u = find(s);
    return u == -1 ? 0 : tr[u].cnt;
}
bool contains(const string& s) const {
    return count(s) > 0;
}
bool erase(const string& s) {
    int u = find(s);
    if(u == -1 || tr[u].end == 0) {
        return false;
    }
    tr[0].cnt--;
    u = 0;
    for(char c : s) {
        u = tr[u].ch[id(c)];
        tr[u].cnt--;
    }
    tr[u].end--;
    return true;
}
};

```

AC 自动机

结点 1 为根；add 返回终点，mark 标记出现次数，work 后遍历文本：p = next(p, c-'a')，沿 out 链累加 cnt。

```
#include <bits/stdc++.h>
using namespace std;

struct AhoCorasick {
    static const int ALPHABET = 26;
    struct Node {
        int len;
        int link;
        int out;
        array<int, ALPHABET> next;
        int cnt;
        Node() : len{0}, link{0}, out{0}, next{}, cnt{0} {}
    };

    vector<Node> t;

    AhoCorasick() {
        init();
    }
    void init() {
        t.assign(2, Node());
        t[0].next.fill(1);
        t[0].len = -1;
    }
    int newNode() {
        t.emplace_back();
        return int(t.size()) - 1;
    }
    int add(const string& s) {
        int p = 1;
        for(char c : s) {
            int x = c - 'a';
            if(t[p].next[x] == 0) {
                t[p].next[x] = newNode();
                t[t[p].next[x]].len = t[p].len + 1;
            }
            p = t[p].next[x];
        }
        return p;
    }
    void mark(int p) {
        t[p].cnt++;
    }
    void work() {
        queue<int> q;
        q.push(1);
        while(!q.empty()) {
```

```

        int x = q.front();
        q.pop();
        for(int c = 0; c < ALPHABET; c++) {
            int y = t[x].next[c];
            if(t[x].next[c] == 0) {
                t[x].next[c] = t[t[x].link].next[c];
            } else {
                t[y].link = t[t[x].link].next[c];
                int f = t[y].link;
                t[y].out = t[f].cnt ? f : t[f].out;
                q.push(y);
            }
        }
    }
}

int next(int p, int x) {
    return t[p].next[x];
}

int link(int p) {
    return t[p].link;
}

int out(int p) {
    return t[p].out;
}

int cnt(int p) {
    return t[p].cnt;
}

int len(int p) {
    return t[p].len;
}

int size() {
    return int(t.size());
}

};

```

后缀自动机 (SAM)

结点 1 为初始状态; `extend(p, c, offset)` 支持任意字符域; `work` 返回 fail 树邻接表; `len[v]-len[link[v]]` 求和为不同子串数。

```

#include <bits/stdc++.h>
using namespace std;

struct SAM {
    static constexpr int ALPHABET_SIZE = 26;
    struct Node {
        int len;
        int link;
        int cnt;
        array<int, ALPHABET_SIZE> next;
        Node() : len{}, link{}, cnt{}, next{} {}
    };
};

```

```

vector<Node> t;

SAM() {
    init();
}

void init() {
    t.assign(2, Node());
    t[0].next.fill(1);
    t[0].len = -1;
}

int newNode() {
    t.emplace_back();
    return int(t.size()) - 1;
}

int extend(int p, int c) {
    if(t[p].next[c]) {
        int q = t[p].next[c];
        if(t[q].len == t[p].len + 1) {
            return q;
        }
        int r = newNode();
        t[r].len = t[p].len + 1;
        t[r].link = t[q].link;
        t[r].next = t[q].next;
        t[q].link = r;
        while(t[p].next[c] == q) {
            t[p].next[c] = r;
            p = t[p].link;
        }
        return r;
    }
    int cur = newNode();
    t[cur].cnt = 1;
    t[cur].len = t[p].len + 1;
    while(!t[p].next[c]) {
        t[p].next[c] = cur;
        p = t[p].link;
    }
    t[cur].link = extend(p, c);
    return cur;
}

int extend(int p, char c, char offset = 'a') {
    return extend(p, c - offset);
}

int next(int p, int x) {
    return t[p].next[x];
}

int next(int p, char c, char offset = 'a') {
    return next(p, c - offset);
}

int link(int p) {
    return t[p].link;
}

```

```

int len(int p) {
    return t[p].len;
}
int size() {
    return int(t.size());
}
int count(int p) {
    return t[p].cnt;
}
vector<vector<int>> work() {
    int n = size();
    vector<vector<int>> adj(n);
    for(int i = 2; i < n; i++) {
        adj[link(i)].push_back(i);
    }
    return adj;
}
};

```

后缀数组 (SA)

1-based; `sa[1..n]` 排序后的后缀起点, `rk[1..n]` 位置排名, `height[1]=0`, `height[i]=LCP(sa[i-1],sa[i])`; 输入不含 0 字节。

```

#include <bits/stdc++.h>
using namespace std;

struct SuffixArray {
    int n;
    vector<int> sa, rk, height;

    SuffixArray(const string& str) {
        n = int(str.size());
        sa.assign(n + 1, 0);
        rk.assign(n + 1, 0);
        height.assign(n + 1, 0);
        if(n == 0) {
            return;
        }
        // 追加最小哨兵, 使循环移位排序等价于后缀排序
        string t = str + char(0);
        int N = n + 1;
        vector<int> p(N), c(N);
        int alphabet = 256;
        vector<int> cnt(max(alphabet, N), 0);
        for(int i = 0; i < N; i++) {
            cnt[(unsigned char)t[i]]++;
        }
        for(int i = 1; i < alphabet; i++) {
            cnt[i] += cnt[i - 1];
        }
        for(int i = 0; i < N; i++) {

```

```

        p[--cnt[(unsigned char)t[i]]] = i;
    }
    c[p[0]] = 0;
    int classes = 1;
    for(int i = 1; i < N; i++) {
        if(t[p[i]] != t[p[i - 1]]) {
            classes++;
        }
        c[p[i]] = classes - 1;
    }
    vector<int> pn(N), cn(N);
    for(int h = 0; (1 << h) < N; h++) {
        int step = 1 << h;
        for(int i = 0; i < N; i++) {
            pn[i] = p[i] - step;
            if(pn[i] < 0) {
                pn[i] += N;
            }
        }
        fill(cnt.begin(), cnt.begin() + classes, 0);
        for(int i = 0; i < N; i++) {
            cnt[c[pn[i]]]++;
        }
        for(int i = 1; i < classes; i++) {
            cnt[i] += cnt[i - 1];
        }
        for(int i = N - 1; i >= 0; i--) {
            p[--cnt[c[pn[i]]]] = pn[i];
        }
        cn[p[0]] = 0;
        int newClasses = 1;
        for(int i = 1; i < N; i++) {
            pair<int, int> cur = {c[p[i]], c[(p[i] + step) % N]};
            pair<int, int> prev = {c[p[i - 1]], c[(p[i - 1] + step) % N]};
            if(cur != prev) {
                newClasses++;
            }
            cn[p[i]] = newClasses - 1;
        }
        c.swap(cn);
        classes = newClasses;
    }
    int k = 0;
    for(int i = 0; i < N; i++) {
        if(p[i] == n) {
            continue;
        }
        sa[++k] = p[i] + 1;
    }
    for(int i = 1; i <= n; i++) {
        rk[sa[i]] = i;
    }
    k = 0;

```

```

        for(int i = 0; i < n; i++) {
            int pos = rk[i + 1];
            if(pos == 1) {
                k = 0;
                continue;
            }
            int j = sa[pos - 1] - 1;
            while(i + k < n && j + k < n && str[i + k] == str[j + k]) {
                k++;
            }
            height[pos] = k;
            if(k) {
                k--;
            }
        }
    }
};

```

最小循环同构

返回字典序最小循环同构的 1-based 起始位置；空串返回 0。

```

#include <bits/stdc++.h>
using namespace std;

int minimalRotation(const string& s) {
    int n = int(s.size());
    if(n == 0) {
        return 0;
    }
    string t = s + s;
    int i = 0, j = 1, k = 0;
    while(i < n && j < n && k < n) {
        char a = t[i + k];
        char b = t[j + k];
        if(a == b) {
            k++;
            continue;
        }
        if(a > b) {
            i += k + 1;
            if(i == j) {
                j++;
            }
        } else {
            j += k + 1;
            if(i == j) {
                j++;
            }
        }
    }
    k = 0;
}

```

```
    return min(i, j) + 1;
}
```

回文半径 (Manacher)

`manacherRadius` 返回变换串回文半径 (P[0] 未用) ; `longestPalindromicSubstring` 返回最长回文子串长度。

```
#include <bits/stdc++.h>
using namespace std;

vector<int> manacherRadius(const string& s) {
    string t = "$#";
    for(char c : s) {
        t += c;
        t += '#';
    }
    t += '&';
    int n = int(t.size());
    vector<int> P(n);
    int R = 0, C = 0;
    for(int i = 1; i < n - 1; i++) {
        P[i] = i < R ? min(P[2 * C - i], P[C] + C - i) : 1;
        while(t[i + P[i]] == t[i - P[i]]) {
            P[i]++;
        }
        if(P[i] + i > R) {
            R = P[i] + i;
            C = i;
        }
    }
    return P;
}

int longestPalindromicSubstring(const string& s) {
    vector<int> P = manacherRadius(s);
    int ans = 1;
    for(int x : P) {
        ans = max(ans, x);
    }
    return ans - 1;
}
```

回文自动机 (Eertree)

结点 0 奇根 (len=-1) 、1 偶根 (len=0) ; `extend(pos)` 逐字符构建, `propagateCount` 后 `cnt` 为出现次数, `distinctCount` 为不同回文子串数。

```
#include <bits/stdc++.h>
using namespace std;

struct PalindromicTree {
```

```

struct Node {
    int len, link;
    array<int, 26> next{};
    int cnt = 0;
    Node() : len(0), link(0), cnt(0) {}
};

vector<Node> tr;
string s;
int last;

PalindromicTree(const string& str) : s(str), last(1) {
    tr.resize(2);
    tr[0].len = -1;
    tr[0].link = 0;
    tr[1].len = 0;
    tr[1].link = 0;
    for(int i = 0; i < int(s.size()); i++) {
        extend(i);
    }
}

int extend(int pos) {
    int c = s[pos] - 'a';
    int cur = last;
    while(true) {
        int curlen = tr[cur].len;
        if(pos - 1 - curlen >= 0 && s[pos - 1 - curlen] == s[pos]) {
            break;
        }
        cur = tr[cur].link;
    }
    if(tr[cur].next[c]) {
        last = tr[cur].next[c];
        tr[last].cnt++;
        return last;
    }
    int now = int(tr.size());
    tr.emplace_back();
    tr[now].len = tr[cur].len + 2;
    tr[cur].next[c] = now;
    tr[now].cnt = 1;
    if(tr[now].len == 1) {
        tr[now].link = 1;
    } else {
        int link = tr[cur].link;
        while(true) {
            int linklen = tr[link].len;
            if(pos - 1 - linklen >= 0 && s[pos - 1 - linklen] == s[pos]) {
                break;
            }
            link = tr[link].link;
        }
        tr[now].link = tr[link].next[c];
    }
}

```

```

        last = now;
        return now;
    }
    void propagateCount() {
        vector<int> order(tr.size());
        iota(order.begin(), order.end(), 0);
        sort(order.begin(), order.end(), [&](int a, int b) {
            return tr[a].len > tr[b].len;
        });
        for(int v : order) {
            if(v >= 2) {
                tr[tr[v].link].cnt += tr[v].cnt;
            }
        }
    }
    int distinctCount() const {
        return int(tr.size()) - 2;
    }
    int size() const {
        return int(tr.size());
    }
};

```

9. 计算几何

点 (Point)

`polarSort` 以 `o` 为极心按极角升序（同角按距离近优先）；整数/浮点坐标均可。

```

#include <bits/stdc++.h>
using namespace std;

using db = double;

template<class T>
struct Point {
    T x;
    T y;
    Point(const T& x_ = 0, const T& y_ = 0) : x(x_), y(y_) {}
    Point& operator+=(const Point& p) {
        x += p.x;
        y += p.y;
        return *this;
    }
    Point& operator-=(const Point& p) {
        x -= p.x;
        y -= p.y;
        return *this;
    }
    Point& operator*=(const T& v) {
        x *= v;
        y *= v;
    }
};

```

```

        return *this;
    }
    Point& operator/=(const T& v) {
        x /= v;
        y /= v;
        return *this;
    }
    Point operator-() const {
        return Point(-x, -y);
    }
    friend Point operator+(Point a, const Point& b) {
        return a += b;
    }
    friend Point operator-(Point a, const Point& b) {
        return a -= b;
    }
    friend Point operator*(Point a, const T& b) {
        return a *= b;
    }
    friend Point operator/(Point a, const T& b) {
        return a /= b;
    }
    friend Point operator*(const T& a, Point b) {
        return b *= a;
    }
    friend bool operator==(const Point& a, const Point& b) {
        return a.x == b.x && a.y == b.y;
    }
    friend bool operator<(const Point& a, const Point& b) {
        return a.x < b.x || (a.x == b.x && a.y < b.y);
    }
};

template<class T>
T dot(const Point<T>& a, const Point<T>& b) {
    return a.x * b.x + a.y * b.y;
}

template<class T>
T cross(const Point<T>& a, const Point<T>& b) {
    return a.x * b.y - a.y * b.x;
}

template<class T>
T square(const Point<T>& p) {
    return dot(p, p);
}

template<class T>
db length(const Point<T>& p) {
    return sqrt(db(square(p)));
}

```

```

template<class T>
db distance(const Point<T>& a, const Point<T>& b) {
    return length(a - b);
}

template<class T>
Point<db> normalize(const Point<T>& p) {
    return Point<db>(db(p.x), db(p.y)) / length(p);
}

template<class T>
Point<T> rotate(const Point<T>& a) {
    return Point<T>(-a.y, a.x);
}

template<class T>
int sgn(const Point<T>& a) {
    return a.y > 0 || (a.y == 0 && a.x > 0) ? 1 : -1;
}

// 以 o 为极心的极角排序（同角按距离近优先），叉积用平移后的向量。
template<class T>
void polarSort(vector<Point<T>>& p, const Point<T>& o = Point<T>()) {
    sort(p.begin(), p.end(), [&](const Point<T>& a, const Point<T>& b) {
        Point<T> ad = a - o;
        Point<T> bd = b - o;
        int sa = sgn(ad);
        int sb = sgn(bd);
        if(sa != sb) {
            return sa > sb;
        }
        T c = cross(ad, bd);
        if(c != 0) {
            return c > 0;
        }
        return square(ad) < square(bd);
    });
}

template<class T>
vector<int> getPolarOrder(const vector<Point<T>>& pts, const Point<T>& o = Point<T>()) {
    int n = int(pts.size());
    vector<int> p(n);
    iota(p.begin(), p.end(), 0);
    sort(p.begin(), p.end(), [&](int i, int j) {
        Point<T> ad = pts[i] - o;
        Point<T> bd = pts[j] - o;
        int sa = sgn(ad);
        int sb = sgn(bd);
        if(sa != sb) {
            return sa > sb;
        }
        T c = cross(ad, bd);
    });
}

```

```

        if(c != 0) {
            return c > 0;
        }
        return square(ad) < square(bd);
    });
    return p;
}

```

直线 (Line)

交点按 double 计算；整型坐标输入时返回 `Point<double>`，不要再强转回整数坐标。

```

#include <bits/stdc++.h>
using namespace std;

using db = double;

template<class T>
struct Point {
    T x;
    T y;
    Point(const T& x_ = 0, const T& y_ = 0) : x(x_), y(y_) {}
    template<class U>
    operator Point<U>() const {
        return Point<U>(U(x), U(y));
    }
    Point& operator+=(const Point& p) {
        x += p.x;
        y += p.y;
        return *this;
    }
    Point& operator-=(const Point& p) {
        x -= p.x;
        y -= p.y;
        return *this;
    }
    Point& operator*=(const T& v) {
        x *= v;
        y *= v;
        return *this;
    }
    Point& operator/=(const T& v) {
        x /= v;
        y /= v;
        return *this;
    }
    Point operator-() const {
        return Point(-x, -y);
    }
    friend Point operator+(Point a, const Point& b) {
        return a += b;
    }
}

```

```

    friend Point operator-(Point a, const Point& b) {
        return a -= b;
    }
    friend Point operator*(Point a, const T& b) {
        return a *= b;
    }
    friend Point operator/(Point a, const T& b) {
        return a /= b;
    }
    friend Point operator*(const T& a, Point b) {
        return b *= a;
    }
    friend bool operator==(const Point& a, const Point& b) {
        return a.x == b.x && a.y == b.y;
    }
};

template<class T>
T dot(const Point<T>& a, const Point<T>& b) {
    return a.x * b.x + a.y * b.y;
}

template<class T>
T cross(const Point<T>& a, const Point<T>& b) {
    return a.x * b.y - a.y * b.x;
}

template<class T>
db length(const Point<T>& p) {
    return sqrt(db(dot(p, p)));
}

template<class T>
struct Line {
    Point<T> a;
    Point<T> b;
    Line(const Point<T>& a_ = Point<T>(), const Point<T>& b_ = Point<T>()) : a(a_), b(b_)
{}
};

template<class T>
db length(const Line<T>& l) {
    return length(l.a - l.b);
}

template<class T>
bool parallel(const Line<T>& l1, const Line<T>& l2) {
    return cross(l1.b - l1.a, l2.b - l2.a) == 0;
}

template<class T>
bool pointOnLineLeft(const Point<T>& p, const Line<T>& l) {
    return cross(l.b - l.a, p - l.a) > 0;
}

```

```

}

template<class T>
bool pointOnLine(const Point<T>& p, const Line<T>& l) {
    return cross(p - l.a, l.b - l.a) == 0;
}

// 整型坐标返回 Point<db>, 浮点坐标返回原类型。
template<class T>
auto lineIntersection(const Line<T>& l1, const Line<T>& l2) {
    using RT = typename conditional<is_integral_v<T>, db, T>::type;
    RT num = RT(cross(l2.b - l2.a, l1.a - l2.a));
    RT den = RT(cross(l2.b - l2.a, l1.a - l1.b));
    RT t = num / den;
    return Point<RT>(RT(l1.a.x) + RT(l1.b.x - l1.a.x) * t,
                    RT(l1.a.y) + RT(l1.b.y - l1.a.y) * t);
}

template<class T>
db distancePL(const Point<T>& p, const Line<T>& l) {
    if(l.a == l.b) {
        return length(p - l.a);
    }
    return db(abs(cross(l.a - l.b, l.a - p))) / length(l);
}

```

线段 (Segment)

`segmentIntersection` 返回 {0 不相交, 1 严格相交, 2 重叠, 3 端点相交}; 输入可为整数, 交点统一以 `Point<double>` 返回, 避免非整数交点被截断。

```

#include <bits/stdc++.h>
using namespace std;

using db = double;

template<class T>
struct Point {
    T x;
    T y;
    Point(const T& x_ = 0, const T& y_ = 0) : x(x_), y(y_) {}
    Point& operator+=(const Point& p) {
        x += p.x;
        y += p.y;
        return *this;
    }
    Point& operator-=(const Point& p) {
        x -= p.x;
        y -= p.y;
        return *this;
    }
    Point& operator*=(const T& v) {

```

```

        x *= v;
        y *= v;
        return *this;
    }
    Point& operator/=(const T& v) {
        x /= v;
        y /= v;
        return *this;
    }
    Point operator-() const {
        return Point(-x, -y);
    }
    friend Point operator+(Point a, const Point& b) {
        return a += b;
    }
    friend Point operator-(Point a, const Point& b) {
        return a -= b;
    }
    friend Point operator*(Point a, const T& b) {
        return a *= b;
    }
    friend Point operator/(Point a, const T& b) {
        return a /= b;
    }
    friend Point operator*(const T& a, Point b) {
        return b *= a;
    }
    friend bool operator==(const Point& a, const Point& b) {
        return a.x == b.x && a.y == b.y;
    }
    friend bool operator<(const Point& a, const Point& b) {
        return a.x < b.x || (a.x == b.x && a.y < b.y);
    }
};

template<class T>
T dot(const Point<T>& a, const Point<T>& b) {
    return a.x * b.x + a.y * b.y;
}

template<class T>
T cross(const Point<T>& a, const Point<T>& b) {
    return a.x * b.y - a.y * b.x;
}

template<class T>
db length(const Point<T>& p) {
    return sqrt(db(dot(p, p)));
}

template<class T>
struct Line {
    Point<T> a;

```

```

    Point<T> b;
    Line(const Point<T>& a_ = Point<T>(), const Point<T>& b_ = Point<T>()) : a(a_), b(b_)
{}
};

template<class T>
db length(const Line<T>& l) {
    return length(l.a - l.b);
}

template<class T>
db distance(const Point<T>& a, const Point<T>& b) {
    return length(a - b);
}

template<class T>
db distancePL(const Point<T>& p, const Line<T>& l) {
    if(l.a == l.b) {
        return distance(p, l.a);
    }
    return db(abs(cross(l.a - l.b, l.a - p))) / length(l);
}

template<class T>
bool pointOnSegment(const Point<T>& p, const Line<T>& l) {
    return cross(p - l.a, l.b - l.a) == 0
        && min(l.a.x, l.b.x) <= p.x && p.x <= max(l.a.x, l.b.x)
        && min(l.a.y, l.b.y) <= p.y && p.y <= max(l.a.y, l.b.y);
}

template<class T>
db distancePS(const Point<T>& p, const Line<T>& l) {
    if(dot(p - l.a, l.b - l.a) < 0) {
        return distance(p, l.a);
    }
    if(dot(p - l.b, l.a - l.b) < 0) {
        return distance(p, l.b);
    }
    return distancePL(p, l);
}

template<class T>
tuple<int, Point<db>, Point<db>> segmentIntersection(const Line<T>& l1, const Line<T>& l2) {
    auto castPoint = [](const Point<T>& p) {
        return Point<db>(db(p.x), db(p.y));
    };
    auto emptyPoint = []() {
        return Point<db>();
    };
    if(l1.a == l1.b) {
        if(l2.a == l2.b) {
            return l1.a == l2.a
                ? tuple<int, Point<db>, Point<db>>{3, castPoint(l1.a), castPoint(l1.a)}

```

```

        : tuple<int, Point<db>, Point<db>>>{0, emptyPoint(), emptyPoint()};
    }
    return pointOnSegment(l1.a, l2)
        ? tuple<int, Point<db>, Point<db>>>{3, castPoint(l1.a), castPoint(l1.a)}
        : tuple<int, Point<db>, Point<db>>>{0, emptyPoint(), emptyPoint()};
}
if(l2.a == l2.b) {
    return pointOnSegment(l2.a, l1)
        ? tuple<int, Point<db>, Point<db>>>{3, castPoint(l2.a), castPoint(l2.a)}
        : tuple<int, Point<db>, Point<db>>>{0, emptyPoint(), emptyPoint()};
}
if(max(l1.a.x, l1.b.x) < min(l2.a.x, l2.b.x)
    || min(l1.a.x, l1.b.x) > max(l2.a.x, l2.b.x)
    || max(l1.a.y, l1.b.y) < min(l2.a.y, l2.b.y)
    || min(l1.a.y, l1.b.y) > max(l2.a.y, l2.b.y)) {
    return {0, emptyPoint(), emptyPoint()};
}
if(cross(l1.b - l1.a, l2.b - l2.a) == 0) {
    if(cross(l1.b - l1.a, l2.a - l1.a) != 0) {
        return {0, emptyPoint(), emptyPoint()};
    }
    Point<T> p1(max(min(l1.a.x, l1.b.x), min(l2.a.x, l2.b.x)),
                max(min(l1.a.y, l1.b.y), min(l2.a.y, l2.b.y)));
    Point<T> p2(min(max(l1.a.x, l1.b.x), max(l2.a.x, l2.b.x)),
                min(max(l1.a.y, l1.b.y), max(l2.a.y, l2.b.y)));
    if(!pointOnSegment(p1, l1)) {
        swap(p1.y, p2.y);
    }
    if(p1 == p2) {
        return {3, castPoint(p1), castPoint(p2)};
    }
    return {2, castPoint(p1), castPoint(p2)};
}
T cp1 = cross(l2.a - l1.a, l2.b - l1.a);
T cp2 = cross(l2.a - l1.b, l2.b - l1.b);
T cp3 = cross(l1.a - l2.a, l1.b - l2.a);
T cp4 = cross(l1.a - l2.b, l1.b - l2.b);
if((cp1 > 0 && cp2 > 0) || (cp1 < 0 && cp2 < 0)
    || (cp3 > 0 && cp4 > 0) || (cp3 < 0 && cp4 < 0)) {
    return {0, emptyPoint(), emptyPoint()};
}
vector<Point<T>> shared;
for(Point<T> q : {l1.a, l1.b, l2.a, l2.b}) {
    if(pointOnSegment(q, l1) && pointOnSegment(q, l2)) {
        shared.push_back(q);
    }
}
sort(shared.begin(), shared.end());
shared.erase(unique(shared.begin(), shared.end()), shared.end());
if(!shared.empty()) {
    Point<db> p = castPoint(shared[0]);
    return {3, p, p};
}

```

```

    db num = db(cross(l2.b - l2.a, l1.a - l2.a));
    db den = db(cross(l2.b - l2.a, l1.a - l1.b));
    db t = num / den;
    Point<db> p(db(l1.a.x) + db(l1.b.x - l1.a.x) * t,
               db(l1.a.y) + db(l1.b.y - l1.a.y) * t);
    return {1, p, p};
}

template<class T>
db distanceSS(const Line<T>& l1, const Line<T>& l2) {
    if(get<0>(segmentIntersection(l1, l2)) != 0) {
        return 0.0;
    }
    return min({distancePS(l1.a, l2), distancePS(l1.b, l2),
               distancePS(l2.a, l1), distancePS(l2.b, l1)});
}

```

凸包 (Andrew 单调链)

≤ 0 弹出共线点 (不保留共线边界顶点), 返回逆时针顶点序列。

```

#include <bits/stdc++.h>
using namespace std;

using db = double;

template<class T>
struct Point {
    T x;
    T y;
    Point(const T& x_ = 0, const T& y_ = 0) : x(x_), y(y_) {}
    Point& operator+=(const Point& p) {
        x += p.x;
        y += p.y;
        return *this;
    }
    Point& operator-=(const Point& p) {
        x -= p.x;
        y -= p.y;
        return *this;
    }
    Point& operator*=(const T& v) {
        x *= v;
        y *= v;
        return *this;
    }
    Point& operator/=(const T& v) {
        x /= v;
        y /= v;
        return *this;
    }
    Point operator-() const {

```

```

        return Point(-x, -y);
    }
    friend Point operator+(Point a, const Point& b) {
        return a += b;
    }
    friend Point operator-(Point a, const Point& b) {
        return a -= b;
    }
    friend Point operator*(Point a, const T& b) {
        return a *= b;
    }
    friend Point operator/(Point a, const T& b) {
        return a /= b;
    }
    friend Point operator*(const T& a, Point b) {
        return b *= a;
    }
    friend bool operator==(const Point& a, const Point& b) {
        return a.x == b.x && a.y == b.y;
    }
    friend bool operator<(const Point& a, const Point& b) {
        return a.x < b.x || (a.x == b.x && a.y < b.y);
    }
};

template<class T>
T cross(const Point<T>& a, const Point<T>& b) {
    return a.x * b.y - a.y * b.x;
}

template<class T>
vector<Point<T>> convexHull(vector<Point<T>> p) {
    int n = int(p.size());
    if(n <= 2) {
        return p;
    }
    sort(p.begin(), p.end());
    vector<Point<T>> h;
    for(int i = 0; i < n; i++) {
        while(int(h.size()) >= 2
            && cross(h.back() - h[int(h.size()) - 2], p[i] - h.back()) <= 0) {
            h.pop_back();
        }
        h.push_back(p[i]);
    }
    int k = int(h.size());
    for(int i = n - 2; i >= 0; i--) {
        while(int(h.size()) > k
            && cross(h.back() - h[int(h.size()) - 2], p[i] - h.back()) <= 0) {
            h.pop_back();
        }
        h.push_back(p[i]);
    }
}

```

```

    h.pop_back();
    return h;
}

// 多边形面积（鞋带公式，逆时针为正）。
template<class T>
db polygonArea(const vector<Point<T>>& p) {
    db area = 0;
    int n = int(p.size());
    for(int i = 0; i < n; i++) {
        area += db(cross(p[i], p[(i + 1) % n]));
    }
    return area / 2.0;
}

```

点在多边形内（射线法）

边界点算在内；支持凹多边形。

```

#include <bits/stdc++.h>
using namespace std;

template<class T>
struct Point {
    T x;
    T y;
    Point(const T& x_ = 0, const T& y_ = 0) : x(x_), y(y_) {}
    Point& operator+=(const Point& p) {
        x += p.x;
        y += p.y;
        return *this;
    }
    Point& operator-=(const Point& p) {
        x -= p.x;
        y -= p.y;
        return *this;
    }
    Point& operator*=(const T& v) {
        x *= v;
        y *= v;
        return *this;
    }
    Point& operator/=(const T& v) {
        x /= v;
        y /= v;
        return *this;
    }
    Point operator-() const {
        return Point(-x, -y);
    }
    friend Point operator+(Point a, const Point& b) {
        return a += b;
    }
}

```

```

    }
    friend Point operator-(Point a, const Point& b) {
        return a -= b;
    }
    friend Point operator*(Point a, const T& b) {
        return a *= b;
    }
    friend Point operator/(Point a, const T& b) {
        return a /= b;
    }
    friend Point operator*(const T& a, Point b) {
        return b *= a;
    }
    friend bool operator==(const Point& a, const Point& b) {
        return a.x == b.x && a.y == b.y;
    }
};

template<class T>
T cross(const Point<T>& a, const Point<T>& b) {
    return a.x * b.y - a.y * b.x;
}

template<class T>
struct Line {
    Point<T> a;
    Point<T> b;
    Line(const Point<T>& a_ = Point<T>(), const Point<T>& b_ = Point<T>()) : a(a_), b(b_)
{}
};

template<class T>
bool pointOnSegment(const Point<T>& p, const Line<T>& l) {
    return cross(p - l.a, l.b - l.a) == 0
        && min(l.a.x, l.b.x) <= p.x && p.x <= max(l.a.x, l.b.x)
        && min(l.a.y, l.b.y) <= p.y && p.y <= max(l.a.y, l.b.y);
}

template<class T>
bool pointOnLineLeft(const Point<T>& p, const Line<T>& l) {
    return cross(l.b - l.a, p - l.a) > 0;
}

template<class T>
bool pointInPolygon(const Point<T>& a, const vector<Point<T>>& p) {
    int n = int(p.size());
    for(int i = 0; i < n; i++) {
        if(pointOnSegment(a, Line(p[i], p[(i + 1) % n]))) {
            return true;
        }
    }
    int t = 0;
    for(int i = 0; i < n; i++) {

```

```

    Point<T> u = p[i];
    Point<T> v = p[(i + 1) % n];
    if(u.x < a.x && v.x >= a.x && pointOnLineLeft(a, Line(v, u))) {
        t ^= 1;
    }
    if(u.x >= a.x && v.x < a.x && pointOnLineLeft(a, Line(u, v))) {
        t ^= 1;
    }
}
return t == 1;
}

```

最近点对（分治）

返回最小距离平方； $n < 2$ 返回 `4e18`。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct Point {
    i64 x, y;
};

i64 dist2(const Point& p1, const Point& p2) {
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
}

i64 closestPairSquared(vector<Point> a) {
    int n = int(a.size());
    sort(a.begin(), a.end(), [](const Point& p1, const Point& p2) {
        return p1.x < p2.x;
    });
    constexpr i64 INF = (i64)4e18;
    auto dfs = [&](auto&& self, int l, int r) -> i64 {
        if(r - l <= 1) {
            return INF;
        }
        if(r - l == 2) {
            return dist2(a[l], a[l + 1]);
        }
        int mid = (l + r) >> 1;
        i64 midX = a[mid].x;
        i64 d = min(self(self, l, mid), self(self, mid, r));
        vector<Point> candidates;
        for(int i = l; i < r; i++) {
            i64 dx = a[i].x - midX;
            if(dx * dx < d) {
                candidates.push_back(a[i]);
            }
        }
    };
}

```

```

        sort(candidates.begin(), candidates.end(), [](const Point& p1, const Point& p2) {
            return p1.y < p2.y;
        });
        for(int i = 0; i < int(candidates.size()); i++) {
            for(int j = i + 1; j < int(candidates.size()); j++) {
                i64 dy = candidates[j].y - candidates[i].y;
                if(dy * dy >= d) {
                    break;
                }
                d = min(d, dist2(candidates[i], candidates[j]));
            }
        }
        return d;
    };
    return dfs(dfs, 0, n);
}

```

旋转卡壳（凸多边形直径）

输入逆时针凸包；返回直径平方。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

template<class T>
struct Point {
    T x;
    T y;
    Point(const T& x_ = 0, const T& y_ = 0) : x(x_), y(y_) {}
    Point& operator+=(const Point& p) {
        x += p.x;
        y += p.y;
        return *this;
    }
    Point& operator-=(const Point& p) {
        x -= p.x;
        y -= p.y;
        return *this;
    }
    Point& operator*=(const T& v) {
        x *= v;
        y *= v;
        return *this;
    }
    Point& operator/=(const T& v) {
        x /= v;
        y /= v;
        return *this;
    }
    Point operator-() const {

```

```

        return Point(-x, -y);
    }
    friend Point operator+(Point a, const Point& b) {
        return a += b;
    }
    friend Point operator-(Point a, const Point& b) {
        return a -= b;
    }
    friend Point operator*(Point a, const T& b) {
        return a *= b;
    }
    friend Point operator*(const T& a, Point b) {
        return b *= a;
    }
    friend bool operator==(const Point& a, const Point& b) {
        return a.x == b.x && a.y == b.y;
    }
};

template<class T>
T cross(const Point<T>& a, const Point<T>& b) {
    return a.x * b.y - a.y * b.x;
}

template<class T>
T dist2(const Point<T>& a, const Point<T>& b) {
    return (a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y);
}

template<class T>
T rotatingCalipersDiameterSquared(const vector<Point<T>>& hull) {
    int n = int(hull.size());
    if(n < 2) {
        return 0;
    }
    T best = 0;
    int j = 1;
    for(int i = 0; i < n; i++) {
        int ni = (i + 1) % n;
        while(cross(hull[ni] - hull[i], hull[(j + 1) % n] - hull[j]) > 0) {
            j = (j + 1) % n;
        }
        best = max(best, dist2(hull[i], hull[j]));
        best = max(best, dist2(hull[ni], hull[j]));
    }
    return best;
}

```

圓（外接圓 / 兩圓交点）

```

#include <bits/stdc++.h>
using namespace std;

```

```

using db = double;

struct Point {
    db x;
    db y;
    Point(db x_ = 0, db y_ = 0) : x(x_), y(y_) {}
    Point operator+(const Point& p) const {
        return Point(x + p.x, y + p.y);
    }
    Point operator-(const Point& p) const {
        return Point(x - p.x, y - p.y);
    }
    Point operator*(db v) const {
        return Point(x * v, y * v);
    }
    Point operator/(db v) const {
        return Point(x / v, y / v);
    }
    friend Point operator*(db a, const Point& p) {
        return p * a;
    }
};

db dot(const Point& a, const Point& b) {
    return a.x * b.x + a.y * b.y;
}

db cross(const Point& a, const Point& b) {
    return a.x * b.y - a.y * b.x;
}

db dist(const Point& a, const Point& b) {
    return hypot(a.x - b.x, a.y - b.y);
}

struct Circle {
    Point c;
    db r;
    Circle(const Point& c_ = Point(), db r_ = 0) : c(c_), r(r_) {}
};

// 三点外接圆：共线返回 nullopt
optional<Circle> circumcircle(const Point& a, const Point& b, const Point& c) {
    Point ab = b - a;
    Point ac = c - a;
    db d = 2 * cross(ab, ac);
    if(fabs(d) < 1e-12) {
        return nullopt;
    }
    db ab2 = dot(ab, ab);
    db ac2 = dot(ac, ac);
    Point o = a + Point(ab2 * ac.y - ac2 * ab.y,

```

```

        ab.x * ac2 - ac.x * ab2) / d;
    return Circle(o, dist(o, a));
}

// 两圆交点（最多 2 个）
vector<Point> intersectCircleCircle(const Circle& c1, const Circle& c2) {
    Point d = c2.c - c1.c;
    db dist2v = dot(d, d);
    if(dist2v < 1e-18) {
        return {};
    }
    db dlen = sqrt(dist2v);
    if(dlen > c1.r + c2.r + 1e-12 || dlen < fabs(c1.r - c2.r) - 1e-12) {
        return {};
    }
    db a = (c1.r * c1.r - c2.r * c2.r + dist2v) / (2 * dlen);
    db h2 = c1.r * c1.r - a * a;
    if(h2 < 0) {
        h2 = 0;
    }
    db h = sqrt(h2);
    Point mid = c1.c + d * (a / dlen);
    Point perp(-d.y, d.x);
    perp = perp * (h / dlen);
    vector<Point> res;
    res.push_back(mid + perp);
    if(h > 1e-12) {
        res.push_back(mid - perp);
    }
    return res;
}

```

半平面交（直线左侧，需有界）

坐标类型 `T` 必须为浮点类型；整数输入先转为 `double/long double`，避免交点回写时截断。

```

#include <bits/stdc++.h>
using namespace std;

using db = double;

template<class T>
struct Point {
    T x;
    T y;
    Point(const T& x_ = 0, const T& y_ = 0) : x(x_), y(y_) {}
    template<class U>
    operator Point<U>() const {
        return Point<U>(U(x), U(y));
    }
    Point& operator+=(const Point& p) {
        x += p.x;

```

```

        y += p.y;
        return *this;
    }
    Point& operator--(const Point& p) {
        x -= p.x;
        y -= p.y;
        return *this;
    }
    Point& operator*=(const T& v) {
        x *= v;
        y *= v;
        return *this;
    }
    Point& operator/=(const T& v) {
        x /= v;
        y /= v;
        return *this;
    }
    Point operator-() const {
        return Point(-x, -y);
    }
    friend Point operator+(Point a, const Point& b) {
        return a += b;
    }
    friend Point operator-(Point a, const Point& b) {
        return a -= b;
    }
    friend Point operator*(Point a, const T& b) {
        return a *= b;
    }
    friend Point operator*(const T& a, Point b) {
        return b *= a;
    }
    friend bool operator==(const Point& a, const Point& b) {
        return a.x == b.x && a.y == b.y;
    }
};

template<class T>
T dot(const Point<T>& a, const Point<T>& b) {
    return a.x * b.x + a.y * b.y;
}

template<class T>
T cross(const Point<T>& a, const Point<T>& b) {
    return a.x * b.y - a.y * b.x;
}

template<class T>
int sgn(const Point<T>& a) {
    return a.y > 0 || (a.y == 0 && a.x > 0) ? 1 : -1;
}

```

```

template<class T>
struct Line {
    Point<T> a;
    Point<T> b;
    Line(const Point<T>& a_ = Point<T>(), const Point<T>& b_ = Point<T>()) : a(a_), b(b_)
{}
};

template<class T>
bool pointOnLineLeft(const Point<T>& p, const Line<T>& l) {
    return cross(l.b - l.a, p - l.a) > 0;
}

template<class T>
auto lineIntersection(const Line<T>& l1, const Line<T>& l2) {
    using RT = typename conditional<is_integral_v<T>, db, T>::type;
    RT num = RT(cross(l2.b - l2.a, l1.a - l2.a));
    RT den = RT(cross(l2.b - l2.a, l1.a - l1.b));
    RT t = num / den;
    return Point<RT>(RT(l1.a.x) + RT(l1.b.x - l1.a.x) * t,
                    RT(l1.a.y) + RT(l1.b.y - l1.a.y) * t);
}

template<class T>
vector<Point<T>> halfPlaneIntersection(vector<Line<T>> lines) {
    static_assert(is_floating_point_v<T>);
    sort(lines.begin(), lines.end(), [](const Line<T>& l1, const Line<T>& l2) {
        Point<T> d1 = l1.b - l1.a;
        Point<T> d2 = l2.b - l2.a;
        if(sgn(d1) != sgn(d2)) {
            return sgn(d1) == 1;
        }
        return cross(d1, d2) > 0;
    });
    deque<Line<T>> ls;
    deque<Point<T>> ps;
    for(auto l : lines) {
        if(ls.empty()) {
            ls.push_back(l);
            continue;
        }
        while(!ps.empty() && !pointOnLineLeft(ps.back(), l)) {
            ps.pop_back();
            ls.pop_back();
        }
        while(!ps.empty() && !pointOnLineLeft(ps.front(), l)) {
            ps.pop_front();
            ls.pop_front();
        }
        if(cross(l.b - l.a, ls.back().b - ls.back().a) == 0) {
            if(dot(l.b - l.a, ls.back().b - ls.back().a) > 0) {
                // 同向平行：保留更靠左的那条
                if(!pointOnLineLeft(ls.back().a, l)) {

```

```

        ls.back() = 1;
    }
    continue;
}
return {};
}
ps.push_back(lineIntersection(ls.back(), 1));
ls.push_back(1);
}
while(!ps.empty() && !pointOnLineLeft(ps.back(), ls[0])) {
    ps.pop_back();
    ls.pop_back();
}
if(int(ls.size()) <= 2) {
    return {};
}
ps.push_back(lineIntersection(ls[0], ls.back()));
return vector<Point<T>>(ps.begin(), ps.end());
}

```

闵可夫斯基和

输入逆时针凸多边形，输出逆时针 Minkowski 和。

```

#include <bits/stdc++.h>
using namespace std;

template<class T>
struct Point {
    T x;
    T y;
    Point(const T& x_ = 0, const T& y_ = 0) : x(x_), y(y_) {}
    Point& operator+=(const Point& p) {
        x += p.x;
        y += p.y;
        return *this;
    }
    Point& operator-=(const Point& p) {
        x -= p.x;
        y -= p.y;
        return *this;
    }
    Point& operator*=(const T& v) {
        x *= v;
        y *= v;
        return *this;
    }
    Point& operator/=(const T& v) {
        x /= v;
        y /= v;
        return *this;
    }
}

```

```

Point operator-() const {
    return Point(-x, -y);
}
friend Point operator+(Point a, const Point& b) {
    return a += b;
}
friend Point operator-(Point a, const Point& b) {
    return a -= b;
}
friend Point operator*(Point a, const T& b) {
    return a *= b;
}
friend Point operator*(const T& a, Point b) {
    return b *= a;
}
friend bool operator==(const Point& a, const Point& b) {
    return a.x == b.x && a.y == b.y;
}
};

template<class T>
T cross(const Point<T>& a, const Point<T>& b) {
    return a.x * b.y - a.y * b.x;
}

template<class T>
int sgn(const Point<T>& a) {
    return a.y > 0 || (a.y == 0 && a.x > 0) ? 1 : -1;
}

template<class T>
vector<Point<T>> minkowskiSum(const vector<Point<T>>& a, const vector<Point<T>>& b) {
    int n = int(a.size());
    int m = int(b.size());
    if(n == 0 || m == 0) {
        return {};
    }
    auto lowest = [](const vector<Point<T>>& p) {
        int k = 0;
        for(int i = 1; i < int(p.size()); i++) {
            if(p[i].y < p[k].y || (p[i].y == p[k].y && p[i].x < p[k].x)) {
                k = i;
            }
        }
        return k;
    };
    auto edge = [](const vector<Point<T>>& p, int k) {
        return p[(k + 1) % int(p.size())] - p[k];
    };
    auto angleLess = [](const Point<T>& u, const Point<T>& v) {
        int su = sgn(u);
        int sv = sgn(v);
        if(su != sv) {

```

```

        return su > sv;
    }
    return cross(u, v) > 0;
};

int ia = lowest(a);
int ib = lowest(b);
vector<Point<T>> result;
result.push_back(a[ia] + b[ib]);
int i = 0, j = 0;
while(i < n || j < m) {
    Point<T> ea = i < n ? edge(a, (ia + i) % n) : Point<T>(0, 0);
    Point<T> eb = j < m ? edge(b, (ib + j) % m) : Point<T>(0, 0);
    Point<T> take;
    if(i == n) {
        take = eb;
        j++;
    } else if(j == m) {
        take = ea;
        i++;
    } else if(angleLess(ea, eb)) {
        take = ea;
        i++;
    } else if(angleLess(eb, ea)) {
        take = eb;
        j++;
    } else {
        take = ea + eb;
        i++;
        j++;
    }
    Point<T> nxt = result.back() + take;
    while(int(result.size()) >= 2
        && cross(result.back() - result[int(result.size()) - 2],
            nxt - result.back()) <= 0) {
        result.pop_back();
    }
    result.push_back(nxt);
}
while(int(result.size()) >= 2
    && cross(result.back() - result[int(result.size()) - 2],
        result[0] - result.back()) <= 0) {
    result.pop_back();
}
if(int(result.size()) >= 2 && result.front() == result.back()) {
    result.pop_back();
}
return result;
}

```

矩形面积并

矩形为 $[x_1, x_2) \times [y_1, y_2)$ ；返回 `__int128`。

```

#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct RectangleCoverTree {
    int n;
    const vector<i64>& ys;
    vector<int> cover;
    vector<i64> len;

    RectangleCoverTree(const vector<i64>& ys_)
        : n(int(ys_.size()) - 1), ys(ys_), cover(4 * max(1, n)), len(4 * max(1, n)) {}

    void pull(int p, int l, int r) {
        if(cover[p]) len[p] = ys[r + 1] - ys[l];
        else if(l == r) len[p] = 0;
        else len[p] = len[p << 1] + len[p << 1 | 1];
    }

    void add(int p, int l, int r, int x, int y, int v) {
        if(x <= l && r <= y) {
            cover[p] += v;
            pull(p, l, r);
            return;
        }
        int m = (l + r) >> 1;
        if(x <= m) add(p << 1, l, m, x, y, v);
        if(y > m) add(p << 1 | 1, m + 1, r, x, y, v);
        pull(p, l, r);
    }

    void add(int l, int r, int v) {
        if(l <= r) add(1, 0, n - 1, l, r, v);
    }

    i64 coveredLength() const {
        return n ? len[1] : 0;
    }
};

__int128 rectangleUnionArea(const vector<array<i64, 4>>& rects) {
    struct Event {
        i64 x, y1, y2;
        int v;
    };
    vector<Event> events;
    vector<i64> ys;
    for(auto [x1, y1, x2, y2] : rects) {
        if(x1 >= x2 || y1 >= y2) continue;
        events.push_back({x1, y1, y2, 1});
        events.push_back({x2, y1, y2, -1});
        ys.push_back(y1);
        ys.push_back(y2);
    }
    if(events.empty()) return 0;

```

```

sort(events.begin(), events.end(), [](const Event& a, const Event& b) {
    return a.x < b.x;
});
sort(ys.begin(), ys.end());
ys.erase(unique(ys.begin(), ys.end()), ys.end());

RectangleCoverTree seg(ys);
__int128 ans = 0;
i64 last = events[0].x;
for(int i = 0, j; i < int(events.size()); i = j) {
    ans += (__int128)seg.coveredLength() * (events[i].x - last);
    for(j = i; j < int(events.size()) && events[j].x == events[i].x; j++) {
        int l = int(lower_bound(ys.begin(), ys.end(), events[j].y1) - ys.begin());
        int r = int(lower_bound(ys.begin(), ys.end(), events[j].y2) - ys.begin()) - 1;
        seg.add(l, r, events[j].v);
    }
    last = events[i].x;
}
return ans;
}

```

10. 离线算法与经典技巧

二分答案

先写单调判定 `check(x)`；求最小可行值时维护“不行/可行”边界，求最大可行值反向。整数域注意终止与溢出；浮点固定迭代次数。

莫队

离线区间询问，当前区间支持 $O(1)$ 增删。按左端点块排序，块内右端点奇偶交替；维护闭区间 `[l, r]`，初始 `[1, 0]`。

离线分治 (CDQ)

按时间/第一维分治，只统计左半对右半的贡献；归并时按第二维排序，第三维用 Fenwick。每层加入的数据必须完整回滚。

整体二分

把所有修改与询问一起按答案域递归分组；判定落左侧的贡献临时加入 Fenwick，询问不足则扣掉计数进右侧，层末回滚。

线段树分治

把生效于时间闭区间 `[l, r]` 的操作挂到时间线段树；DFS 进入结点应用操作，离开时回滚。常配 Rollback DSU 处理离线动态连通。

根号分治 / 序列分块

阈值 B 将对象分成 $O(B)$ 暴力与 $O(n/B)$ 预处理两部分；或序列散块暴力、整块维护 tag/有序表。令两侧代价平衡。

度数定向 / 重轻点

阈值约 $\sqrt{2m}$ ：轻点枚举邻边，重点维护聚合表。枚举三角形等可把边从低度点定向到高度点，使出度 $O(\sqrt{m})$ 。

时间分块

按操作时间分块。每块只显式处理本块会修改的 $O(B)$ 个对象，其余对象视为静态并预处理 $Base$ ；查询 = $Base +$ 块内动态贡献。

离线倒序

把删除/断开倒过来变成添加/合并。先构造最终状态，再逆序应用操作并记录答案，最后反转。

双指针 / 滑动窗口

窗口可行性对右端点单调时，右指针只前进；每次扩右后收左到重新可行。增删操作必须对称。

折半搜索

$n \leq 40$ 的 2^n 搜索拆成两半，各枚举 $2^{n/2}$ 状态，再排序双指针/二分/哈希合并。

坐标压缩

排序去重后用 $lower_bound + 1$ 映射到 1-based；只保留相对顺序。需要区间长度时必须同时保留原坐标。

扫描线

把二维对象拆成进入/离开事件，按一维排序，另一维用 Fenwick/线段树维护。同坐标事件顺序与离散线段 $[y_i, y_{i+1})$ 要按题意确定。

对数段技巧 (Log Trick)

固定右端点时，所有子数组的 AND/OR/gcd 取值只有 $O(\log V)$ 段；更新全部旧状态后合并相同值，同时累计每段长度/次数。

前缀和 / 差分

闭区间 $[l, r]$ 加 v ： $d[l] += v, d[r+1] -= v$ 。等差段用二阶差分，最后做两次前缀和恢复。

贡献法 / 动态维护贡献

把总答案拆成每个对象的出现次数或局部贡献。修改时只重算受影响项，按“删旧贡献 $\rightarrow$ 改状态 $\rightarrow$ 加新贡献”；分组 DP 可用 $sum += new - old$ 。

有限群惰性标记

全局操作来自很小的置换/变换群时，预处理每个群元素的作用与合成表；状态只保存群编号，更新变成 $O(1)$ 合成。

容斥

按条件子集累加 $(-1)^{|S|}$ ；互质计数常用 $[\gcd(i,j)=1]=\sum_{d|\gcd(i,j)} \mu(d)$ ，再配整除分块。

反悔贪心

先接受当前选择，超出约束时用堆撤销代价最差的一项；或把“替换旧选择的收益差”入堆。

单调状态空间 Top-K

各维已排序时，从最优状态入堆；每次弹出并扩展各维 $+1$ 的邻居，用访问集合去重，直到取出 K 个。

最后出现位置单调栈

去重子序列字典序最优：当前元素更优时，只能弹出“后面仍会出现”的栈顶；`vis` 与栈同步撤销。

二进制对齐块分解

用 `lowbit` 把前缀拆成 2 的幂大小且左端点对齐的块。XOR mask 后块起点为 $(1 \wedge \text{mask}) \& \sim(\text{len}-1)$ ； $[0,m]$ 按 $[0,m+1)$ 分解。

相邻交换排序

目标可拆成相邻两项次序代价时，只比较 `xy` 与 `yx` 的交换差来定排序。交叉乘法用 `i128`，方向先用小规模全排列验证。

区间-点贪心匹配

区间按左端点排序，点递增扫描；把 $l \leq p$ 的区间加入 r 小根堆，删去 $r < p$ 的过期项，堆非空则匹配最小 r 。

可行性 Oracle 构造

逐位按目标顺序试候选；若 `ok(prefix)` 判断剩余仍可完成就固定。求第 k 个时把布尔 oracle 换成方案计数。

加权中位数

$\min \sum w_i |x-a_i|$ 的最优点是累计权首次达到总权一半的位置；动态场景用 Fenwick 找第 k 个。

固定半径空间哈希

桶边长取距离阈值 d ；距离小于 d 的点对只可能落在本桶或相邻桶。负坐标必须使用 `floor` 除法。

曼哈顿距离符号变换 (L1)

二维维护 $u=x+y, v=x-y$ 的四个极值；到点集最大 L1 距离为 $\max(u-u_{\min}, u_{\max}-u, v-v_{\min}, v_{\max}-v)$ 。

重置点分段

会清空/重排历史影响的操作作为分段点；段内独立计算，跨段只保留可组合的摘要。常把段内选择转成前 k 大收益。

频次排序记忆化

二元查询按频次规范化，使低频对象在前；遍历低频侧、在高频侧二分/查询并缓存。交换方向可由总对数减去已算答案。

预留坐标

已知最多 M 次插到最前：初始位置放到 $M+1..M+n$ ，第 i 次前插使用 $M-i+1$ ；Fenwick 前缀和维护排名。

势能剪枝线段树

区间取模等更新若结点 $mx < x$ 可整段跳过，否则下钻叶子。一次有效取模使值至少减半，单元有效修改 $O(\log V)$ 次。

布尔格对称链

把 $0/1$ 串视为括号匹配；将第一个未匹配 0 改为 1 ，可构造相邻层的单射匹配，用于布尔格分层配对与补集论证。

11. 公式与博弈

常用恒等式

- $\sum i = n(n+1)/2$, $\sum i^2 = n(n+1)(2n+1)/6$, $\sum i^3 = (n(n+1)/2)^2$ 。
- $\sum C(n,k)C(m,r-k) = C(n+m,r)$ 。
- $\sum_{d|n} \phi(d) = n$, $\sum_{d|n} \mu(d) = [n=1]$ 。
- $[gcd(i,j)=1] = \sum_{d|gcd(i,j)} \mu(d)$, $gcd(a,b) lcm(a,b) = ab$ 。

博弈 (SG / Nim)

公平组合游戏： $sg(x) = mex\{sg(y)\}$ ，独立子游戏 SG 异或为 0 时先手必败。Nim 直接异或各堆；反 Nim/非公平游戏不能直接套。